Post-Symbolic Programming

Universal State-Operator Calculus

Typed Multigraph Semantics · Constraint Lattice · Replayable Evidence

Solve / Gate / Coagula

Formal Specification

Master Document — Version 1.0.0

Sebastian Klemm

Contents

0.1 Part I — Foundations (P-I)

0.1.1 I.1 Scope, non-goals, and non-claims (P-I.C1)

F1. Purpose

Purpose. This node fixes the scope boundary of PSP as a deterministic, state-first programming discipline. It prevents scope drift by explicit non-goals and explicit non-claims. No downstream node may reinterpret PSP as a textual language, a metaphor, or an implementation-only style guide.

F2. Dependencies

Dependencies. None. This node is foundational.

F3. Definitions (Defined)

(Defined) PSP. Post-Symbolic Programming (PSP) is a formal discipline in which a "program" is an operator composition acting on a typed state space Σ. PSP semantics are defined by $(\Sigma, \Omega, \mathrm{RD})$: a state space, a typed operator calculus, and a determinism contract enabling replay and audit.

(Defined) PSP specification. The PSP master specification is a single frozen frame plus a finite fill-out that assigns every primitive, operator, gate, proof obligation, registry entry, and boundary condition a stable node ID.

(Defined) Conformance. An implementation or process is PSP-conformant iff it satisfies all *normative* clauses of this document and preserves the structural invariants SI1–SI7.

(Defined) Normative vs informative text. A clause is *normative* iff it is explicitly marked as such using the keywords **MUST**, **MUST NOT**, **SHOULD**, **MAY**. All other text is informative and cannot impose requirements.

F4. Derived statements

(Derived) Non-textuality. PSP does not require a textual surface syntax: any textual notation, if present, is merely an encoding of operator compositions and typed state objects.

Proof. By definition, PSP semantics are fixed by the triple $(\Sigma, \Omega, \mathrm{RD})$. A textual notation can only (i) name elements of Σ, (ii) denote operators in Ω, and (iii) specify RD. Therefore a textual syntax is not a semantic prerequisite but an optional representation layer. □

F5. Proof or status

All statements in this node are either **Defined** or **Derived** with explicit proofs above. No external theorems are required.

F6. Model notes

Model note. This node is model-agnostic. Model classes (e.g. Scorpio boundary calculus, AinSoft runtime) are forbidden to introduce foundational meaning here. Any model-specific content belongs to Part VII and must be labeled **Model-Specific**.

F7. Examples (non-definitional)

Example (informative). A "program" may be represented as a chain $\Omega_k \circ \cdots \circ \Omega_1$, or as an operator DAG/HDAG evaluated in topological order. These are representations, not foundations.

F8. Limitations

Limitations. This node does not define Σ, does not define any concrete operator set, does not define gates, and does not define convergence. It fixes only the scope boundary and the meaning of PSP.

F9. Local DoD

Local DoD. This node is complete iff: (i) PSP is defined as $(\Sigma, \Omega, \mathrm{RD})$ and not as a text language, (ii) conformance and normative keyword discipline are fixed, (iii) explicit non-goals/non-claims prevent scope drift into models or tooling.

0.1.2 I.2 PSP discipline definition: state-first programming (P-I.C2)

F1. Purpose

Purpose. This node fixes PSP as a *state-first* discipline: semantics are defined by the evolution of a typed state, not by the interpretation of symbolic surface forms. It establishes PSP's core commitments: determinism, auditability, and operator-defined meaning.

F2. Dependencies

Dependencies. Requires §0.1.1.

F3. Definitions (Defined)

(Defined) State-first semantics. A PSP system is state-first iff all semantic meaning is defined as a function of the state trajectory

$$\Sigma_0 \xrightarrow{\Omega_1} \Sigma_1 \xrightarrow{\Omega_2} \cdots \xrightarrow{\Omega_k} \Sigma_k,$$

where each Ω_i is a typed operator $\Omega_i : \Sigma \to \Sigma$ and each Σ_i is a well-formed state.

(Defined) PSP program. A PSP program is a finite operator composition $P := \Omega_k \circ \cdots \circ \Omega_1$ together with an evaluation policy that determines how the composition is executed (sequence evaluation or DAG/HDAG evaluation). A program is *well-typed* iff all operator domain/codomain constraints are satisfied along every evaluated path.

(Defined) Determinism contract. A PSP run is deterministic iff for fixed inputs and fixed run descriptor RD the produced trajectory and all emitted artifacts are identical. Formally, for the run function

$$\mathrm{Run}(\Sigma_0, P, \mathrm{RD}) := (\Sigma_k, \mathcal{T}, \mathcal{A}),$$

where $\mathcal{T}$ is the recorded trajectory and $\mathcal{A}$ the emitted artifact set, determinism means:

$$(\Sigma_0, P, \mathrm{RD}) = (\Sigma_0', P', \mathrm{RD}') \;\Rightarrow\; (\Sigma_k, \mathcal{T}, \mathcal{A}) = (\Sigma_k', \mathcal{T}', \mathcal{A}').$$

(Defined) Replay. A run is replayable iff RD contains sufficient information to reproduce the run deterministically, including canonicalized inputs, fixed parameters/constants, and explicit resolution rules for ties and nondeterminism sources.

(Defined) Auditability. A PSP system is auditable iff every emitted artifact $a \in \mathcal{A}$ admits a verifiable provenance claim linking a to a specific run $(\Sigma_0, P, \mathrm{RD})$ and to the operator subsequence (or subgraph) that produced it.

F4. Derived statements

(Derived) Textual representations are conservative. Any textual notation used to describe PSP computations is conservative over the state-first semantics: it cannot add semantic meaning beyond naming states, operators, and run descriptors.

Proof. By the definition of state-first semantics, meaning is fully determined by $(\Sigma_0, P, \mathrm{RD})$ and the operator-evaluation policy. A textual notation can only encode these inputs (or a representation thereof). Hence any notation is an encoding layer and cannot introduce additional semantics without changing $(\Sigma_0, P, \mathrm{RD})$, which would constitute a different PSP run rather than additional meaning. □

(Derived) Determinism implies reproducible artifacts. If a run is deterministic and auditable, then each emitted artifact has a reproducible provenance trace.

Proof. Determinism implies that repeating $\mathrm{Run}(\Sigma_0, P, \mathrm{RD})$ yields identical $(\mathcal{T}, \mathcal{A})$. Auditability provides a verifiable mapping from each artifact to its producing run and operator segment. Therefore each artifact's origin can be re-established by replay, yielding a reproducible provenance trace. □

F5. Proof or status

All non-definitional claims above are derived with explicit proofs. No external theorems are required.

F6. Model notes

Model note. This node is model-agnostic: it does not assume any particular internal geometry, logic base, or domain. Model families (e.g. boundary calculus, lattice dynamics, runtime meshes) may instantiate Σ and Ω later, but may not alter the state-first semantics defined here.

F7. Examples (non-definitional)

Example (informative). A program may be executed as a linear chain, or as a DAG of operators where nodes are operators and edges indicate state dependencies; evaluation proceeds in a deterministic topological order with a deterministic tie-break rule.

F8. Limitations

Limitations. This node does not define the internal structure of Σ, does not define concrete operators, and does not define convergence or correctness criteria; it fixes only the semantic discipline of PSP.

F9. Local DoD

Local DoD. This node is complete iff: (i) state-first semantics are fixed as state trajectory semantics, (ii) program, determinism, replay, and auditability are defined formally, (iii) textual representations are explicitly conservative over the formal semantics.

0.1.3 I.3 Normative language, proof status, and admissibility (P-I.C3)

F1. Purpose

Purpose. This node fixes the normative interpretation of requirements and the proof-status taxonomy used throughout PSP. It ensures that formal claims are explicitly classified and that no informal statement silently carries foundational weight.

F2. Dependencies

Dependencies. Requires §0.1.1 and §0.1.2.

F3. Definitions (Defined)

(Defined) Normative keywords. Statements containing the keywords **MUST**, **MUST NOT**, **SHOULD**, **MAY** are normative and define conformance requirements. All other statements are informative.

(Defined) Claim classes. A claim is any statement intended to be used as a premise for subsequent reasoning. Each claim MUST be assigned exactly one proof-status label from the set

$$\mathcal{S} := \{\text{Axiomatic},\ \text{Defined},\ \text{Derived},\ \text{Model-Specific},\ \text{External},\ \text{Open}\}.$$

(Defined) Axiomatic. A claim is *Axiomatic* iff it is adopted as a primitive assumption of PSP.

(Defined) Defined. A claim is *Defined* iff it introduces a new object, function, relation, operator, or predicate by an explicit formal definition.

(Defined) Derived. A claim is *Derived* iff it is proved using only Axiomatic and Defined material from earlier sections, together with previously Derived results whose proofs are contained in-document.

(Defined) Model-Specific. A claim is *Model-Specific* iff it holds only within an explicitly declared model class (e.g. a particular state representation, geometry, runtime, or boundary calculus). A Model-Specific claim MUST state the model boundary.

(Defined) External. A claim is *External* iff it relies on an external theorem, external standard result, or external reference not proved within this document. An External claim MUST explicitly name the dependency and MUST NOT be treated as Derived.

(Defined) Open. A claim is *Open* iff it is a known gap, conjecture, or unresolved bridge. Open claims MUST be quarantined and MUST NOT be used as premises for Derived claims.

(Defined) Admissibility. A section is admissible iff all claims it introduces are labeled by a proof-status in $\mathcal{S}$ and all dependencies used in proofs are admissible under the relevant status rules.

F4. Derived statements

(Derived) No silent upgrades. A claim labeled External or Open cannot be used to prove a Derived claim without either (i) importing the dependency explicitly as External in the derived claim, or (ii) providing an in-document proof that removes the dependency.

Proof. By definition, Derived claims must be proved only from Axiomatic/Defined/Derived material with in-document proofs. External and Open claims do not satisfy this requirement. Therefore, any proof that uses an External/Open claim cannot qualify as a Derived proof unless the dependency is removed by proof or the result is reclassified accordingly. □

(Derived) Model boundary preservation. A Model-Specific claim cannot be used to establish a foundational (model-agnostic) Derived claim without explicitly propagating the model boundary.

Proof. Model-Specific claims hold only under a model restriction. If such a claim were used to establish a model-agnostic Derived claim, the conclusion would inherit the model restriction. Therefore either the restriction is propagated or the proof is invalid as a model-agnostic derivation. □

F5. Proof or status

All non-definitional claims above are derived with explicit proofs. No external theorems are required.

F6. Model notes

Model note. This node is model-agnostic. It governs proof discipline and admissibility across all PSP model families.

F7. Examples (non-definitional)

Example (informative). If a convergence statement invokes Banach's fixed-point theorem but does not prove it, that statement must be labeled External. If the theorem is proved in-document (rare), the statement may be labeled Derived.

F8. Limitations

Limitations. This node does not decide which axioms are adopted. It only fixes the admissible status classes and how they may interact in proofs.

F9. Local DoD

Local DoD. This node is complete iff: (i) the proof-status taxonomy $\mathcal{S}$ is fixed, (ii) each status has an admissibility rule, (iii) silent status upgrades are forbidden by derived discipline, (iv) model-boundary leakage into foundations is forbidden.

0.1.4 I.4 Determinism, canonicalization, and replay contract (P-I.C4)

F1. Purpose

Purpose. This node fixes the determinism and replay contract required for PSP runs. It specifies what it means for inputs, states, operators, and artifacts to be replayable and auditably reproducible, and it defines the minimum canonicalization discipline required to make this property well-defined.

F2. Dependencies

Dependencies. Requires §0.1.2 and §0.1.3.

F3. Definitions (Defined)

(Defined) Run descriptor. A run descriptor is a finite record

$$\mathrm{RD} := (\mathrm{RD}_{\mathrm{ver}}, \text{seed}, \text{canon}, \text{params}, \text{env}, \text{tie}, \text{hash})$$

where: (i) $\mathrm{RD}_{\mathrm{ver}}$ is the PSP-spec version identifier, (ii) seed is the run seed (may be a constant, not necessarily random), (iii) canon specifies canonicalization rules for inputs/states, (iv) params lists all numeric constants and thresholds used, (v) env fixes execution-relevant environment constraints, (vi) tie specifies deterministic tie-breaking rules, (vii) hash specifies the digest algorithm(s) used for traceability.

(Defined) Canonicalization map. A canonicalization map is a deterministic function

$$\mathrm{Canon} : \mathcal{X} \to \mathcal{X}$$

such that $\mathrm{Canon}(\mathrm{Canon}(x)) = \mathrm{Canon}(x)$ (idempotence), and for any x, y, if x and y are equivalent under the chosen representation equivalence $\sim$, then $\mathrm{Canon}(x) = \mathrm{Canon}(y)$.

(Defined) Digest. A digest is a deterministic map

$$\mathrm{Dig} : \mathcal{X} \to \{0,1\}^{\ell}$$

with fixed output length ℓ and fixed algorithm identifier in RD. The digest is used only for identity/traceability, not as a semantic definition of equality.

(Defined) Trace. A trace is a finite record

$$\mathcal{T} := \big((i, \Omega_i, \mathrm{in}_i, \mathrm{out}_i, d_i)\big)_{i=1}^{k},$$

where $\mathrm{in}_i = \mathrm{Dig}(\mathrm{Canon}(\Sigma_{i-1}))$, $\mathrm{out}_i = \mathrm{Dig}(\mathrm{Canon}(\Sigma_i))$, and d_i is an operator-local evidence payload (may be empty). The trace is the minimal replay object.

(Defined) Replay. A run is replayable iff executing the same program P on the same canonicalized initial input $\mathrm{Canon}(\Sigma_0)$ under the same RD produces the same final state digest and the same artifact digests:

$$\mathrm{Run}(\mathrm{Canon}(\Sigma_0), P, \mathrm{RD}) \equiv (\mathrm{Dig}(\mathrm{Canon}(\Sigma_k)), \mathrm{Dig}(\mathcal{A})).$$

(Defined) Deterministic tie-break. A deterministic tie-break rule is a total order $\prec$ on candidate sets used by any selection step (e.g. top-k, Pareto selection, scheduling). The rule MUST be a pure function of canonicalized objects and RD.

F4. Derived statements

(Derived) Replay implies audit reproducibility. If (i) all operators in P are pure with respect to (Σ, RD), (ii) canonicalization is idempotent, and (iii) all selection steps use deterministic tie-break rules, then replay yields identical trace digests.

Proof. Purity implies Σ_i is determined by $(\Sigma_{i-1}, \Omega_i, \mathrm{RD})$. Idempotent canonicalization fixes representation variability. Deterministic tie-break removes nondeterminism in selections. By induction over $i = 1, \ldots, k$, $\mathrm{Canon}(\Sigma_i)$ is uniquely determined, hence each $\mathrm{Dig}(\mathrm{Canon}(\Sigma_i))$ is uniquely determined, and the full trace digest sequence is reproduced. □

(Derived) Canonicalization is required for cross-run comparability. If canonicalization is absent, representation-dependent variability can yield non-identical digests for semantically equivalent states, breaking replay comparability.

Proof. Without Canon, two equivalent representations $x \sim y$ may have $\mathrm{Dig}(x) \neq \mathrm{Dig}(y)$. Thus two runs can be semantically equivalent but non-identical in trace identity, violating the replay equality criterion. □

F5. Proof or status

All non-definitional claims above are derived with explicit proofs. No external theorems are required.

F6. Model notes

Model note. This node is model-agnostic. Any model-specific state representation MUST provide a compatible equivalence relation $\sim$ and a canonicalization map Canon satisfying the idempotence and equivalence-collapsing conditions above.

F7. Examples (non-definitional)

Example (informative). If a selection operator chooses among equal-scoring candidates, it must use a stable deterministic order derived from $(\mathrm{Dig}(\mathrm{Canon}(\cdot)), \mathrm{RD})$ rather than an ambient iteration order.

F8. Limitations

Limitations. This node does not specify a concrete digest algorithm, numeric format, or serialization format. It specifies the requirements such formats MUST satisfy to support determinism and replay.

F9. Local DoD

Local DoD. This node is complete iff: (i) RD is formally defined with mandatory fields, (ii) canonicalization, digest, trace, replay and tie-break rules are defined, (iii) replay determinism is established under explicit purity and tie-break assumptions.

0.1.5 I.5 Structural invariants and conformance boundary (P-I.C5)

F1. Purpose

Purpose. This node fixes the global invariants that every PSP-conformant system MUST satisfy. These invariants prevent hidden drift, untyped operator use, semantic leakage from model-specific layers into foundations, and non-auditable execution.

F2. Dependencies

Dependencies. Requires §0.1.2, §0.1.3, and §0.1.4.

F3. Definitions (Defined)

(Defined) Structural invariants. The structural invariants are the following requirements, denoted SI1–SI9.

SI1 (Typed state). PSP MUST define a typed state space Σ and a well-formedness predicate $\mathrm{WF} : \Sigma \to \{0,1\}$; every run MUST preserve well-formedness: $\mathrm{WF}(\Sigma_i) = 1$ for all i.

SI2 (Typed operators). Every operator $\Omega \in \otimes$ MUST have a declared type $\Omega : \Sigma \to \Sigma$ and an admissibility predicate $\mathrm{Adm}_\Omega(\Sigma, \mathrm{RD})$.

SI3 (Deterministic evaluation). All selection, scheduling, and tie resolution steps MUST be deterministic functions of canonicalized inputs and RD (cf. §0.1.4).

SI4 (Replay). A PSP run MUST be replayable: the trace and artifacts MUST be reproducible under the same canonicalized inputs and the same RD.

SI5 (Proof-status discipline). Every claim used as a premise MUST be labeled by exactly one proof status in $\mathcal{S}$ and MUST obey the admissibility rules (cf. §0.1.3).

SI6 (Model wall). Model-specific constructions MUST NOT be used to establish foundational claims unless the model boundary is explicitly propagated and the claim is labeled Model-Specific.

SI7 (No hidden channels). A PSP-conformant system MUST NOT depend on unstated external state such as ambient time, ambient randomness, undefined IO order, or undefined platform iteration order.

SI8 (Artifact provenance). Every emitted artifact MUST carry a provenance binding to the producing run $(\Sigma_0, P, \mathrm{RD})$ and to the producing operator subgraph/subsequence.

SI9 (Quarantine). Open claims and quarantined items MUST NOT influence gates, proofs, or derived correctness claims except by explicit labeling and explicit boundary conditions.

(Defined) Conformance. A system is PSP-conformant iff it satisfies SI1–SI9 and all normative requirements introduced in Parts I–IX.

F4. Derived statements

(Derived) SI1+SI2 imply semantic well-typedness of trajectories. If SI1 and SI2 hold and all executed operators satisfy their admissibility predicates, then every run defines a well-typed trajectory $\Sigma_0 \to \Sigma_1 \to \cdots \to \Sigma_k$.

Proof. By SI2 each executed operator maps Σ to Σ. By SI1 and preservation, each Σ_i remains well-formed. Admissibility ensures each operator application is permitted on the current state. Hence the full trajectory is typed and well-formed at every step. □

(Derived) SI3+SI4 imply stable artifact identity under replay. If SI3 and SI4 hold, then the digests of emitted artifacts are stable under replay.

Proof. SI4 gives replay equality of the run outputs under fixed canonical inputs and RD. SI3 removes nondeterminism from selections, hence the artifact-producing subcomputations are identical, yielding identical artifact digests. □

F5. Proof or status

All non-definitional claims above are derived with explicit proofs. No external theorems are required.

F6. Model notes

Model note. Model families may add additional invariants, but cannot weaken SI1–SI9.

F7. Examples (non-definitional)

Example (informative). If two candidates have identical scores, choosing the "first encountered" is non-conformant unless the encounter order is itself a deterministic function of canonicalized objects and RD.

F8. Limitations

Limitations. This node does not define concrete operator sets or concrete artifact formats; it fixes invariant constraints that any such choices must satisfy.

F9. Local DoD

Local DoD. This node is complete iff: (i) SI1–SI9 are fixed as normative invariants, (ii) conformance is defined in terms of SI1–SI9 plus the remainder of the document, (iii) derived consequences of SI1–SI4 are stated and proved.

0.1.6 I.6 Admissible model classes and boundary discipline (P-I.C6)

F1. Purpose

Purpose. This node defines what constitutes an admissible PSP model class and fixes the boundary rules by which model-specific constructions may instantiate PSP without altering the foundational semantics and invariants.

F2. Dependencies

Dependencies. Requires §0.1.1–§0.1.5.

F3. Definitions (Defined)

(Defined) Model class. A *model class* $\mathfrak{M}$ is a tuple

$$\mathfrak{M} := (\Sigma_{\mathfrak{M}},\, \Omega_{\mathfrak{M}},\, \mathrm{RD}_{\mathfrak{M}},\, \mathrm{Rep}_{\mathfrak{M}})$$

where: (i) $\Sigma_{\mathfrak{M}}$ is a concrete state representation, (ii) $\Omega_{\mathfrak{M}}$ is a concrete operator family acting on $\Sigma_{\mathfrak{M}}$, (iii) $\mathrm{RD}_{\mathfrak{M}}$ refines the run descriptor with model-specific parameters, (iv) $\mathrm{Rep}_{\mathfrak{M}}$ is a representation layer (optional) mapping external encodings to $\Sigma_{\mathfrak{M}}$.

(Defined) Instantiation. A model class $\mathfrak{M}$ instantiates PSP iff there exists a semantics-preserving embedding

$$\iota_{\mathfrak{M}} : \Sigma_{\mathfrak{M}} \hookrightarrow \Sigma \quad \text{and} \quad \kappa_{\mathfrak{M}} : \Omega_{\mathfrak{M}} \to \otimes$$

such that for every well-formed $\sigma \in \Sigma_{\mathfrak{M}}$ and every admissible operator $\omega \in \Omega_{\mathfrak{M}}$,

$$\iota_{\mathfrak{M}}(\omega(\sigma)) \equiv \kappa_{\mathfrak{M}}(\omega)(\iota_{\mathfrak{M}}(\sigma)),$$

where $\equiv$ denotes semantic equivalence under canonicalization.

(Defined) Admissible model class. A model class $\mathfrak{M}$ is *admissible* iff:

M1. it satisfies SI1–SI9 (possibly via model-specific implementations of WF, Canon, Dig),
M2. it declares all model-specific axioms/assumptions explicitly and labels them Model-Specific or External,
M3. it provides a deterministic canonicalization and deterministic tie-break rules compatible with §0.1.4,
M4. it provides a boundary statement: which claims hold only in $\mathfrak{M}$ and do not generalize.

(Defined) Boundary statement. A boundary statement is a finite list of declarations of the form

"Claim c is valid only under model class $\mathfrak{M}$."

A model class without a boundary statement is non-admissible.

(Defined) Model families. A *model family* is a named collection of admissible model classes sharing a common representation commitment. Typical families include:

- boundary-calculus models (message/header states, masking/phase operators),
- lattice-dynamics models (operator lattices, contraction/closure regimes),
- runtime-mesh models (digital twin, projection-driven dynamics),
- toolchain models (manifest/replay/audit orchestration).

Family names are informative; admissibility is decided by M1–M4.

F4. Derived statements

(Derived) Model wall yields portability of foundational results. If a result is proved as Derived using only Axiomatic/Defined material and does not invoke any Model-Specific premises, then it holds for every admissible model class.

Proof. By definition, a Derived proof uses only model-agnostic premises. Any admissible model class instantiates PSP while preserving semantics and invariants, hence the same proof steps apply under the instantiation maps. Therefore the result is preserved across all admissible models. □

(Derived) Boundary statements prevent semantic leakage. If all model-specific claims include boundary statements and are labeled Model-Specific, then no foundational claim can silently depend on them without violating §0.1.3.

Proof. A foundational claim is model-agnostic by construction. Using a Model-Specific premise would require propagating the model boundary and re-labeling the conclusion accordingly (by §0.1.3). If this is not done, the proof is inadmissible. Hence leakage is prevented. □

F5. Proof or status

All non-definitional claims above are derived with explicit proofs. No external theorems are required.

F6. Model notes

Model note. This node defines admissibility and boundaries but does not privilege any specific model family. Specific model instantiations are introduced later (Part VII).

F7. Examples (non-definitional)

Example (informative). A boundary-calculus model may define a boundary state Σ_B and masking operators. These are admissible if (i) they preserve determinism/replay and (ii) any claims about stealth or protocol mimicry are labeled Model-Specific and bounded.

F8. Limitations

Limitations. This node does not provide concrete model implementations. It specifies the admissibility contract a model must satisfy.

F9. Local DoD

Local DoD. This node is complete iff: (i) model class, instantiation, admissibility and boundary statement are defined, (ii) admissibility requires SI1–SI9 and determinism compatibility, (iii) portability and leakage-prevention properties are derived and proved.

0.1.7 I.7 Notation, equivalence, and typing conventions (P-I.C7)

F1. Purpose

Purpose. This node fixes the notational and typing conventions used throughout PSP, including equivalence, canonicalization conventions, and operator-composition conventions. It ensures that later formal statements are unambiguous and that representation choices cannot silently change semantics.

F2. Dependencies

Dependencies. Requires §0.1.3 and §0.1.4.

F3. Definitions (Defined)

(Defined) State space and well-formedness. Σ denotes the PSP state space. $\mathrm{WF}(\sigma) = 1$ denotes that $\sigma \in \Sigma$ is well-formed.

(Defined) Operator set and typing. $\otimes$ denotes the set of admissible PSP operators. Every $\Omega \in \otimes$ has type $\Omega : \Sigma \to \Sigma$ and admissibility predicate $\mathrm{Adm}_\Omega(\sigma, \mathrm{RD})$.

(Defined) Composition. For $\Omega_1, \Omega_2 : \Sigma \to \Sigma$, composition is written $(\Omega_2 \circ \Omega_1)(\sigma) := \Omega_2(\Omega_1(\sigma))$. A program is a finite composition $P = \Omega_k \circ \cdots \circ \Omega_1$.

(Defined) Evaluation policy. An evaluation policy $\mathcal{E}$ specifies how an operator composition is executed. Two canonical policies are: (i) *sequential* execution of P, and (ii) *DAG/HDAG* execution of an operator graph in a deterministic topological order. If $\mathcal{E}$ is not specified, sequential execution is the default.

(Defined) Representation equivalence. A representation equivalence relation $\sim$ on Σ is any equivalence relation such that $\sigma \sim \sigma'$ means "σ and σ' represent the same semantic state". Every admissible model class MUST declare its $\sim$ explicitly.

(Defined) Canonical form. A canonicalization map $\mathrm{Canon} : \Sigma \to \Sigma$ is a deterministic idempotent map satisfying: (i) $\mathrm{Canon}(\mathrm{Canon}(\sigma)) = \mathrm{Canon}(\sigma)$ and (ii) $\sigma \sim \sigma' \Rightarrow \mathrm{Canon}(\sigma) = \mathrm{Canon}(\sigma')$.

(Defined) Semantic equivalence. $\sigma \equiv \sigma'$ denotes semantic equivalence under canonicalization:

$$\sigma \equiv \sigma' \iff \mathrm{Canon}(\sigma) = \mathrm{Canon}(\sigma').$$

(Defined) Deterministic ordering. A deterministic ordering $\prec$ is a total order on a finite candidate set, defined as a pure function of canonicalized objects and RD.

F4. Derived statements

(Derived) Canonical equality is a congruence for deterministic operators. Assume $\Omega : \Sigma \to \Sigma$ is deterministic with respect to (Σ, RD) and respects semantic equivalence:

$$\sigma \equiv \sigma' \Rightarrow \Omega(\sigma) \equiv \Omega(\sigma').$$

Then $\equiv$ is preserved by Ω.

Proof. If $\sigma \equiv \sigma'$, then $\mathrm{Canon}(\sigma) = \mathrm{Canon}(\sigma')$. By the assumed equivalence-respecting property, $\Omega(\sigma) \equiv \Omega(\sigma')$, i.e. $\mathrm{Canon}(\Omega(\sigma)) = \mathrm{Canon}(\Omega(\sigma'))$. Thus $\equiv$ is preserved under Ω. □

(Derived) Deterministic evaluation eliminates representation-dependent nondeterminism. If (i) all operators are deterministic, (ii) all selections use $\prec$, and (iii) canonicalization is applied at each step (or at least at IO boundaries), then evaluation outcomes are invariant under representational variation within $\sim$.

Proof. Representational variation within $\sim$ is collapsed by Canon. Deterministic operators and deterministic selection remove residual nondeterminism. Therefore the canonical outcome depends only on the semantic input class, not on representation artifacts. □

F5. Proof or status

All non-definitional claims above are derived with explicit proofs. No external theorems are required.

F6. Model notes

Model note. Specific state encodings (graphs, tensors, boundary messages, lattices) are model-specific representations. This node fixes the abstract equivalence/canonicalization discipline they must satisfy.

F7. Examples (non-definitional)

Example (informative). If a state is represented by a graph, two isomorphic graphs may represent the same semantic state. Then $\sim$ may be graph isomorphism and Canon may be a canonical labeling procedure.

F8. Limitations

Limitations. This node does not define any particular $\sim$ or Canon implementation. It fixes the abstract constraints required for determinism and semantic stability.

F9. Local DoD

Local DoD. This node is complete iff: (i) composition and evaluation conventions are fixed, (ii) $\sim$, Canon, and $\equiv$ are defined and linked, (iii) deterministic ordering is defined, (iv) preservation properties are stated and proved.

0.1.8 I.8 Dual-fabric consistency principle and robustness discipline (P-I.C8)

F1. Purpose

Purpose. This node introduces the PSP robustness discipline that prevents one-sided operator narratives from becoming foundational truth. The core mechanism is *dual-fabric consistency*: a computation (or acceptance decision) is treated as stable only if it remains stable under two explicitly declared, independently evaluated operator realizations that share the same determinism contract.

F2. Dependencies

Dependencies. Requires §0.1.2–§0.1.7.

F3. Definitions (Defined)

(Defined) Fabric. A fabric is a pair $\mathcal{F} := (P, \mathcal{E})$ consisting of a program P (operator composition or operator DAG) and an evaluation policy $\mathcal{E}$ that deterministically executes P on Σ.

(Defined) Dual fabrics. Two fabrics $\mathcal{F}_1 = (P_1, \mathcal{E}_1)$ and $\mathcal{F}_2 = (P_2, \mathcal{E}_2)$ are *dual* for a task τ iff: (i) both are well-typed on Σ, (ii) both satisfy the same run descriptor RD discipline, (iii) both are intended to realize the same task-level predicate Q_τ over final states, but (iv) their internal operator decompositions differ (distinct operator chains or distinct operator graphs).

(Defined) Task predicate. A task predicate is a boolean map $Q_\tau : \Sigma \to \{0, 1\}$ (e.g. acceptance, gating, safety, admissibility).

(Defined) Dual-fabric agreement. Given initial state Σ_0, dual fabrics $\mathcal{F}_1, \mathcal{F}_2$ *agree* on τ iff, under the same RD,

$$Q_\tau\big(\mathrm{Run}(\Sigma_0, \mathcal{F}_1, \mathrm{RD}).\Sigma_k\big) = Q_\tau\big(\mathrm{Run}(\Sigma_0, \mathcal{F}_2, \mathrm{RD}).\Sigma_k\big).$$

(Defined) Mirror-consistency index (MCI). Let $m : \Sigma \to \mathbb{R}^d$ be a deterministic measurement map (a profile) and let $\|\cdot\|$ be a norm on $\mathbb{R}^d$. For dual fabrics producing final states $\Sigma_k^{(1)}$ and $\Sigma_k^{(2)}$, define

$$\mathrm{MCI}(\Sigma_0; \mathcal{F}_1, \mathcal{F}_2) := 1 - \min\Big(1,\ \frac{\|m(\Sigma_k^{(1)}) - m(\Sigma_k^{(2)})\|}{\epsilon_m}\Big),$$

for a fixed tolerance parameter $\epsilon_m > 0$ contained in RD. Thus $\mathrm{MCI} \in [0, 1]$ and $\mathrm{MCI} = 1$ means measurement-level equality within tolerance.

(Defined) Robust acceptance. A robust acceptance rule for task τ is a predicate

$$\mathrm{Accept}_\tau(\Sigma_0) := \big(Q_\tau(\Sigma_k^{(1)}) = 1\big)\ \wedge\ \big(Q_\tau(\Sigma_k^{(2)}) = 1\big)\ \wedge\ \big(\mathrm{MCI} \geq \eta\big),$$

for a threshold $\eta \in [0, 1]$ fixed in RD.

F4. Derived statements

(Derived) Dual-fabric discipline reduces representation-induced false positives. Assume (i) both fabrics are deterministic and replayable under the same RD, (ii) the measurement map m is compatible with semantic equivalence (i.e. $\sigma \equiv \sigma' \Rightarrow m(\sigma) = m(\sigma')$), and (iii) robust acceptance requires dual agreement plus $\mathrm{MCI} \geq \eta$. Then acceptance cannot be triggered purely by representation artifacts within $\sim$.

Proof. If two initial states differ only by representation within $\sim$, canonicalization collapses them to the same semantic class, so both fabrics compute semantically equivalent trajectories (by determinism and equivalence compatibility). By assumption on m, measurement outcomes cannot diverge solely due to representation artifacts. Therefore any acceptance requires agreement driven by semantic content rather than representational noise. □

(Derived) Robust acceptance is replay-stable. If SI3–SI4 hold and robust acceptance is defined only in terms of deterministic runs and deterministic measurements fixed by RD, then $\mathrm{Accept}_\tau(\Sigma_0)$ is invariant under replay.

Proof. By determinism and replay, both final states and all deterministic measurements are identical under replay. Hence the boolean conjunction defining Accept_τ yields the same value. □

F5. Proof or status

All non-definitional claims above are derived with explicit proofs. No external theorems are required.

F6. Model notes

Model note. Dual fabrics are a meta-discipline and do not fix any particular operator family. Concrete dualizations (e.g. swapping operator roles, alternate decompositions, alternate evaluation graphs) are model-specific constructions and must be declared where used.

F7. Examples (non-definitional)

Example (informative). A task may be evaluated both as a linear operator chain and as an operator DAG with different intermediate normal forms. Robust acceptance requires agreement of the acceptance predicate plus high measurement consistency.

F8. Limitations

Limitations. This node does not specify which dual fabrics must exist for a given task; it specifies the admissible contract for declaring and using dual fabrics when robustness is required.

F9. Local DoD

Local DoD. This node is complete iff: (i) fabrics and dual fabrics are defined, (ii) dual agreement and MCI are defined with RD-fixed tolerances, (iii) robust acceptance is defined, (iv) replay-stability and anti-artifact properties are derived and proved.

0.1.9 I.9 Termination, convergence, and closure notions (P-I.C9)

F1. Purpose

Purpose. This node fixes the minimal, model-agnostic notions of termination and convergence used throughout PSP. It defines what it means for an operator program to "finish" and what it means for iterative post-symbolic processes to stabilize, without committing to any particular geometry or metric beyond what is required.

F2. Dependencies

Dependencies. Requires §0.1.4 and §0.1.7.

F3. Definitions (Defined)

(Defined) Step index and budget. A PSP run executes a finite sequence of steps indexed by $i = 0, 1, \ldots, k$. A run budget is a tuple $\mathcal{B} := (k_{\max}, t_{\max}, \beta)$ where $k_{\max} \in \mathbb{N}$ is the maximal step count, $t_{\max} > 0$ is an optional time budget (model-specific), and β is an optional resource budget descriptor (e.g. memory/compute ceilings). All budgets MUST be fixed in RD.

(Defined) Termination. A run terminates iff at least one of the following holds:

T1. (*budget termination*) $k = k_{\max}$ or a budget constraint in $\mathcal{B}$ is exhausted,
T2. (*gate termination*) a designated termination gate $q_{\text{term}} : \Sigma \to \{0, 1\}$ evaluates to 1,
T3. (*convergence termination*) a convergence predicate $\text{Conv}_\varepsilon(\Sigma_{i-1}, \Sigma_i) = 1$ holds for a specified ε and persistence window length N (defined below).

(Defined) Convergence witness. A convergence witness is a tuple (d, ε, N) where: (i) $d : \Sigma \times \Sigma \to \mathbb{R}_{\geq 0}$ is a deterministic pseudo-distance, (ii) $\varepsilon > 0$ is a tolerance, (iii) $N \in \mathbb{N}$ is a persistence window length. All three MUST be fixed in RD.

(Defined) Convergence predicate. Given witness (d, ε, N), define

$$\text{Conv}_\varepsilon(\Sigma_{i-N}, \ldots, \Sigma_i) = 1 \iff \max_{j=i-N+1,\ldots,i} d(\Sigma_{j-1}, \Sigma_j) \leq \varepsilon.$$

If $i < N$, the predicate is undefined and MUST NOT be used.

(Defined) Fixed point (state-level). A state $\Sigma^\star$ is a fixed point of an operator Ω iff $\Omega(\Sigma^\star) \equiv \Sigma^\star$ (semantic equivalence per §0.1.7).

(Defined) Closure (program-level). A program P is *closed* with respect to a run class $\mathcal{R}$ iff for every run in $\mathcal{R}$, termination is guaranteed and all termination conditions are recorded in the trace.

F4. Derived statements

(Derived) Convergence termination is replay-stable. If d is deterministic, ε and N are fixed in RD, and the run is replayable, then convergence termination decisions are invariant under replay.

Proof. Replay yields the same canonicalized trajectory digests (SI3–SI4). Deterministic d applied to the same state sequence yields identical distances. Therefore the inequality checks for ε across the same window N yield identical truth values, hence the convergence termination decision is replay-stable. □

(Derived) Fixed point implies convergence for persistent contraction steps. Assume there exists i_0 such that for all $j \geq i_0$, $\Sigma_j \equiv \Sigma^\star$. Then for any witness (d, ε, N) with $d(\Sigma^\star, \Sigma^\star) = 0$ and any $\varepsilon > 0$, $\text{Conv}_\varepsilon(\Sigma_{i-N}, \ldots, \Sigma_i) = 1$ holds for all $i \geq i_0 + N$.

Proof. If $\Sigma_j \equiv \Sigma^\star$ for all $j \geq i_0$, then for $j \geq i_0 + 1$, $d(\Sigma_{j-1}, \Sigma_j) = 0$ by the assumption on d. Hence the maximum over any window fully contained in $\{i_0, \ldots\}$ is $0 \leq \varepsilon$, establishing convergence. □

F5. Proof or status

All non-definitional claims above are derived with explicit proofs. No external theorems are required.

F6. Model notes

Model note. Concrete choices of d (graph edit distance, tensor norms, profile distances) are model-specific. This node requires only determinism and compatibility with semantic equivalence where needed.

F7. Examples (non-definitional)

Example (informative). A system may use (i) a profile-based distance $d(\Sigma, \Sigma') = \|m(\Sigma) - m(\Sigma')\|$ with a fixed tolerance ε, and (ii) a termination gate that triggers when an acceptance predicate holds and MCI $\geq \eta$.

F8. Limitations

Limitations. This node does not guarantee termination for arbitrary operator families. It fixes the vocabulary and replay discipline for termination and convergence claims.

F9. Local DoD

Local DoD. This node is complete iff: (i) termination modes are defined and RD-bound, (ii) convergence witness and predicate are defined, (iii) fixed point and closure notions are defined, (iv) replay-stability of convergence termination is derived and proved.

0.2 Part II — State Algebra (P-II)

0.2.1 II.1 Core state space Σ and well-formedness (P-II.C1)

F1. Purpose

Purpose. This node fixes the canonical PSP state space Σ and the well-formedness discipline WF. It provides the minimal typed carrier on which all operators act.

F2. Dependencies

Dependencies. Requires Part I (in particular §0.1.2, §0.1.4, §0.1.5, §0.1.7).

F3. Definitions (Defined)

(Defined) Canonical PSP state. The canonical PSP state is a tuple

$$\Sigma := (G, z, \Pi, \mu, H, RD),$$

where:

- $G = (V, E, \tau)$ is a typed multigraph with node set V, edge set E, and typing function τ,
- $z \in \mathbb{D}^n$ is a global measurement vector over a fixed numeric domain $\mathbb{D}$ (e.g. fixed-point or real),
- Π is a finite set of projection/embedding maps used to move between representations,
- $\mu : \Sigma \to \mathbb{R}$ is a potential (or energy) functional used for monotone descent / stability claims,
- H is a finite history object (trace, events, proofs, deltas),
- RD is the run descriptor (determinism contract).

(Defined) State component domains. Each component has a declared domain:

$$G \in \mathcal{G}, \quad z \in \mathbb{D}^n, \quad \Pi \in \mathcal{P}, \quad \mu \in \mathcal{U}, \quad H \in \mathcal{H}, \quad RD \in \mathcal{R}.$$

(Defined) Well-formedness predicate. $\mathrm{WF} : \Sigma \to \{0, 1\}$ is defined by:

$$\mathrm{WF}(\Sigma) = 1 \iff \big(\mathrm{WF}_G(G) = 1\big) \wedge \big(\mathrm{WF}_z(z) = 1\big) \wedge \big(\mathrm{WF}_\Pi(\Pi) = 1\big) \wedge \big(\mathrm{WF}_H(H) = 1\big) \wedge \big(\mathrm{WF}_{RD}(RD) = 1\big),$$

where each sub-predicate is defined in the relevant chapters below.

(Defined) Semantic equality. Semantic equality of states is defined by canonicalization (Part I):

$$\Sigma \equiv \Sigma' \iff \mathrm{Canon}(\Sigma) = \mathrm{Canon}(\Sigma').$$

(Defined) Admissible state updates. An operator application $\Omega(\Sigma) = \Sigma'$ is admissible iff: (i) $\mathrm{Adm}_\Omega(\Sigma, RD) = 1$, and (ii) $\mathrm{WF}(\Sigma') = 1$.

F4. Derived statements

(Derived) State typing implies operator totality requirement. If PSP claims an operator $\Omega : \Sigma \to \Sigma$, then Ω must be total on well-formed states (or explicitly partial with an admissibility predicate).

Proof. By SI2 every operator has declared type and admissibility predicate. Totality on well-formed inputs is the default; otherwise admissibility restricts the domain. In either case, the semantics are well-defined only if the operator's domain over which it may be applied is explicit. □

F5. Proof or status

All statements are Defined or Derived with explicit proofs. No external theorems.

F6. Model notes

Model note. Model classes may choose different concrete encodings for G, z, Π, μ, and H, but MUST provide compatible well-formedness checks and canonicalization.

F7. Examples (non-definitional)

Example (informative). A model may take $\mathbb{D} = \mathbb{Q}_{16.16}$ for deterministic fixed-point execution, or $\mathbb{D} = \mathbb{R}$ for analysis, as long as RD fixes the choice and replay is preserved.

F8. Limitations

Limitations. This node does not define G typing rules, projection admissibility, the potential μ, or the history schema. It fixes only the canonical state tuple and its top-level well-formedness discipline.

F9. Local DoD

Local DoD. Complete iff: (i) $\Sigma = (G, z, \Pi, \mu, H, RD)$ is fixed, (ii) WF is defined compositionally, (iii) admissible state updates are defined, (iv) semantic equality refers to canonicalization.

0.2.2 II.2 Typed multigraph $G = (V, E, \tau)$ (P-II.C2)

F1. Purpose

Purpose. This node defines the PSP carrier graph G as a typed multigraph and fixes minimal typing rules required for operator correctness, auditing, and deterministic canonicalization.

F2. Dependencies

Dependencies. Requires §0.2.1 and §0.1.7.

F3. Definitions (Defined)

(Defined) Node and edge sets. V is a finite set of nodes. E is a finite multiset of directed edges. An edge is a triple $e = (u, v, k)$ with $u, v \in V$ and a key k distinguishing parallel edges.

(Defined) Type universe. Let $\mathcal{T}_V$ be the set of node types and $\mathcal{T}_E$ the set of edge types.

(Defined) Typing function. $\tau := (\tau_V, \tau_E)$ with $\tau_V : V \to \mathcal{T}_V$ and $\tau_E : E \to \mathcal{T}_E$.

(Defined) Typed adjacency. For an edge $e = (u, v, k)$, define $\mathrm{src}(e) = u$ and $\mathrm{tgt}(e) = v$. Typing constraints may restrict admissible pairs:

$$\mathrm{Allow}_E\big(\tau_V(u), \tau_V(v), \tau_E(e)\big) \in \{0, 1\}.$$

(Defined) Graph well-formedness. $\mathrm{WF}_G(G) = 1$ iff:

G1. V and E are finite,
G2. τ_V and τ_E are total on V and E,
G3. all edges satisfy $\mathrm{src}(e), \mathrm{tgt}(e) \in V$,
G4. all edges satisfy $\mathrm{Allow}_E(\tau_V(\mathrm{src}(e)), \tau_V(\mathrm{tgt}(e)), \tau_E(e)) = 1$,
G5. G is canonicalizable under the chosen equivalence $\sim_G$ (declared by the model class).

(Defined) Graph equivalence. A model class declares $\sim_G$ (e.g. labeled isomorphism, canonical labeling equivalence). Canonicalization MUST collapse $\sim_G$-equivalent graphs.

F4. Derived statements

(Derived) Typing constraints enable local operator admissibility. If all graph edits performed by an operator preserve Allow_E, then WF_G is preserved under that operator.

Proof. An operator may add/remove/rewire edges. If after each edit all edges remain within V and satisfy Allow_E, then conditions G2–G4 remain satisfied. If the operator does not break finiteness (G1), WF_G is preserved. □

F5. Proof or status

All statements are Defined or Derived with explicit proofs.

F6. Model notes

Model note. Specific node/edge type vocabularies are model-specific and belong to Part VII. This node fixes only the typing mechanism and well-formedness schema.

F7. Examples (non-definitional)

Example (informative). A node type universe may include `Operator`, `State`, `Artifact`, `Constraint`, while edge types may include `depends_on`, `emits`, `measures`. Such vocabularies are not fixed here.

F8. Limitations

Limitations. This node does not require acyclicity. Acyclicity constraints, if needed, are introduced by specific operator programs or model classes.

F9. Local DoD

Local DoD. Complete iff: (i) typed multigraph structure is defined, (ii) WF_G is fixed with explicit conditions, (iii) equivalence and canonicalization obligations are stated.

0.2.3 II.3 Measurement vector z and profiles (P-II.C3)

F1. Purpose

Purpose. This node defines the global measurement vector z and the profile mechanism by which z is computed from G and other state components. It fixes the minimal constraints required for determinism and comparability across runs.

F2. Dependencies

Dependencies. Requires §0.2.1, §0.1.4, and §0.1.7.

F3. Definitions (Defined)

(Defined) Measurement domain. $\mathbb{D}$ is the numeric domain used for measurements (e.g. fixed-point or real). The chosen $\mathbb{D}$ MUST be fixed in RD.

(Defined) Measurement vector. $z \in \mathbb{D}^n$ is a finite vector of normalized measurement coordinates.

(Defined) Profile. A profile is a deterministic measurement map

$$m : \Sigma \to \mathbb{D}^n,$$

and z is defined by $z := m(\Sigma)$ for the active profile m.

(Defined) Profile registry. A PSP-conformant system MUST identify the active profile by an identifier m_{id} stored in RD. If multiple profiles are used, each MUST be named and versioned.

(Defined) Normalization discipline. A profile MUST declare its normalization rule. Unless stated otherwise, each coordinate is normalized to $[0, 1]$. Normalization parameters (min/max, scaling constants) MUST be recorded in RD.

(Defined) Measurement well-formedness. $\mathrm{WF}_z(z) = 1$ iff $z \in \mathbb{D}^n$ and all coordinates satisfy the declared normalization constraints.

F4. Derived statements

(Derived) Profile identity is required for meaningful comparisons. If two runs use different profiles (or different profile versions), then equality/ordering comparisons of z values are not meaningful unless an explicit translation map is provided.

Proof. A profile defines the semantics of each coordinate in z. Different profiles induce different coordinate meanings, so numeric comparisons are not semantically preserved in general. Therefore cross-profile comparisons require an explicit translation mechanism, otherwise they are undefined. □

(Derived) Deterministic profiles preserve replay stability. If m is deterministic and all its parameters are fixed in RD, then z is replay-stable under the replay contract.

Proof. Replay-stability follows from determinism: under replay the same Σ and the same RD recur, hence $m(\Sigma)$ is identical, yielding identical z. □

F5. Proof or status

All statements are Defined or Derived with explicit proofs.

F6. Model notes

Model note. Specific coordinate semantics (e.g. tripolar coordinates, resonance axes, codematrix axes) are introduced later as model-specific profiles. This node fixes only the profile mechanism and determinism constraints.

F7. Examples (non-definitional)

Example (informative). A profile may measure structural complexity, symmetry signatures, stability margins, or gate evidence metrics, provided it is deterministic and recorded in RD.

F8. Limitations

Limitations. This node does not define any concrete profile coordinates. It defines the registry, determinism, and normalization discipline.

F9. Local DoD

Local DoD. Complete iff: (i) z and the measurement domain $\mathbb{D}$ are defined, (ii) profiles are defined as deterministic maps with registry identity in RD, (iii) normalization and WF_z are fixed, (iv) replay and comparability properties are derived and proved.

0.2.4 II.4 Projection/embedding set Π (P-II.C4)

F1. Purpose

Purpose. This node defines the projection/embedding apparatus Π used to move between representations of the same semantic state (e.g. graph, tensor, boundary message, profile space). It fixes typing, admissibility, and determinism rules for representation transitions.

F2. Dependencies

Dependencies. Requires §0.2.1 and §0.1.7.

F3. Definitions (Defined)

(Defined) Projection/embedding family. Π is a finite set of maps of the form

$$\pi_j : \mathcal{X}_j \to \mathcal{Y}_j,$$

where $\mathcal{X}_j, \mathcal{Y}_j$ are declared representation domains (graph space, tensor space, message space, etc.).

(Defined) Typing of projections. Each π_j MUST declare: (i) its domain and codomain $(\mathcal{X}_j, \mathcal{Y}_j)$, (ii) an admissibility predicate $\mathrm{Adm}_{\pi_j}(\Sigma, RD) \in \{0, 1\}$, (iii) a canonicalization compatibility statement:

$$\sigma \equiv \sigma' \Rightarrow \pi_j(\sigma) \equiv_{\mathcal{Y}_j} \pi_j(\sigma'),$$

where $\equiv_{\mathcal{Y}_j}$ is semantic equivalence in the codomain representation.

(Defined) Determinism requirement. Every π_j MUST be deterministic with respect to (Σ, RD).

(Defined) Projection well-formedness. $\mathrm{WF}_\Pi(\Pi) = 1$ iff:

P1. Π is finite,
P2. each π_j has declared $(\mathcal{X}_j, \mathcal{Y}_j)$,
P3. each π_j has Adm_{π_j},
P4. each π_j declares canonicalization compatibility,
P5. each π_j is deterministic under RD.

F4. Derived statements

(Derived) Projection determinism preserves replay comparability. If all projections used in a run are deterministic and recorded via identifiers in RD, then representation transitions do not introduce replay drift.

Proof. Replay fixes (Σ, RD) and the projection identifiers. Deterministic π_j yields identical outputs for identical inputs, hence representation transitions are replay-stable and cannot introduce nondeterminism. □

(Derived) Canonicalization compatibility prevents representation leakage. If each π_j respects semantic equivalence, then proofs and gates written over codomain representations cannot be triggered purely by representational variance in the domain.

Proof. If two domain states are semantically equivalent, their canonical forms coincide. By compatibility, codomain canonical forms also coincide, so any predicate that depends only on codomain canonical forms cannot separate equivalent domain states. Hence representational leakage is prevented. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Concrete projection families (e.g. graph→tensor, state→boundary message, state→profile) are introduced later. This node fixes only the admissibility/determinism contract.

F7. Examples (non-definitional)

Example (informative). A projection $\pi : \Sigma \to \mathbb{D}^n$ can serve as a profile map; a projection $\pi : \Sigma \to \Sigma_B$ can serve as a boundary encoder, provided admissibility and determinism are satisfied.

F8. Limitations

Limitations. This node does not define any particular projection formulas. It fixes the type and determinism constraints required for PSP.

F9. Local DoD

Local DoD. Complete iff: (i) Π is defined as a finite typed map family, (ii) admissibility and determinism are mandatory, (iii) canonicalization compatibility is mandatory, (iv) replay-stability properties are derived and proved.

0.2.5 II.5 Potential functional μ and monotonic descent interface (P-II.C5)

F1. Purpose

Purpose. This node defines the potential functional μ used to express monotone descent, stability, and convergence claims in a model-agnostic way. It does not fix a particular physics interpretation; it provides an abstract descent interface.

F2. Dependencies

Dependencies. Requires §0.2.1 and §0.1.9.

F3. Definitions (Defined)

(Defined) Potential functional. A potential functional is a deterministic map

$$\mu : \Sigma \to \mathbb{R},$$

recorded by identifier in RD, together with declared evaluation conditions (e.g. numeric domain, normalization).

(Defined) Descent operator. An operator Ω is a *descent operator* with respect to μ if whenever $\mathrm{Adm}_\Omega(\Sigma, RD) = 1$,

$$\mu(\Omega(\Sigma)) \leq \mu(\Sigma).$$

(Defined) Strict descent region. A set $\mathcal{D} \subseteq \Sigma$ is a strict descent region for Ω if for all $\Sigma \in \mathcal{D}$,

$$\mu(\Omega(\Sigma)) < \mu(\Sigma),$$

unless Σ is a fixed point of Ω.

(Defined) Potential well-formedness. $\mathrm{WF}_\mu(\mu) = 1$ iff μ is deterministic under RD and its evaluation parameters (scales, normalization, numeric type) are recorded in RD.

F4. Derived statements

(Derived) Descent implies bounded non-increase along trajectories. If every step operator in a run is a descent operator for the same μ, then $\mu(\Sigma_i)$ is non-increasing along the trajectory.

Proof. By induction: if $\mu(\Sigma_i) \leq \mu(\Sigma_{i-1})$ for each step, then the sequence is non-increasing. □

(Derived) Strict descent plus finite lower bound prevents infinite strict decrease. Assume μ is bounded below on the reachable set, and a strict descent region $\mathcal{D}$ is entered infinitely often. Then μ cannot strictly decrease at every such visit.

Proof. A real-valued sequence bounded below cannot decrease strictly infinitely often without violating lower boundedness. Therefore strict decrease cannot occur at every visit; eventually the system must leave the strict descent regime or approach a fixed point. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Concrete μ choices are model-specific (e.g. resonance energy, constraint violation norm, entropy proxy). This node fixes only the interface required to use μ in admissibility, gating, and convergence claims.

F7. Examples (non-definitional)

Example (informative). A gate may require both $q(\Sigma) = 1$ and $\Delta\mu := \mu(\Sigma_i) - \mu(\Sigma_{i-1}) \leq 0$ over a persistence window.

F8. Limitations

Limitations. This node does not guarantee convergence. It provides a uniform scalar witness that can support convergence arguments when combined with other conditions.

F9. Local DoD

Local DoD. Complete iff: (i) μ is defined as deterministic and RD-bound, (ii) descent and strict descent are defined, (iii) basic descent consequences are derived and proved.

0.2.6 II.6 History object H and evidence payloads (P-II.C6)

F1. Purpose

Purpose. This node defines the history object H as the carrier for trace, evidence, proof objects, deltas, and audit material. It fixes minimal requirements for replay, provenance, and gate evidence.

F2. Dependencies

Dependencies. Requires §0.1.4 and §0.1.5.

F3. Definitions (Defined)

(Defined) History object. H is a finite record composed of:

$$H := (\mathcal{T},\ \mathcal{E},\ \mathcal{M},\ \mathcal{D}),$$

where:

- $\mathcal{T}$ is the step trace (operator IDs, input/output digests),
- $\mathcal{E}$ is an evidence set (gate evidence, measurement snapshots),
- $\mathcal{M}$ is an artifact manifest set (artifact digests, provenance bindings),
- $\mathcal{D}$ is a delta set describing state edits (optional, model-specific).

(Defined) Evidence payload. An evidence payload is a finite object attached to a trace step, used to justify a gate decision or an admissibility decision. Evidence payloads MUST be deterministic functions of canonicalized states and RD.

(Defined) History well-formedness. $\mathrm{WF}_H(H) = 1$ iff:

H1. $\mathcal{T}$ is finite and each entry contains operator identifier, input digest, output digest,
H2. $\mathcal{E}$ is finite and each evidence item is tied to a trace index or artifact ID,
H3. $\mathcal{M}$ is finite and each manifest item binds an artifact digest to (Σ_0, P, RD) and a producing operator segment,
H4. all digests in H use algorithms declared in RD.

F4. Derived statements

(Derived) Deterministic evidence preserves audit reproducibility. If all evidence payloads are deterministic functions of canonicalized states and RD, then the audit trail is replay-stable.

Proof. Replay reproduces the same canonicalized states at each trace step. Deterministic evidence functions therefore reproduce the same evidence payloads. Hence the audit trail is stable under replay. □

(Derived) Manifest binding implies artifact provenance verifiability. If each manifest item binds artifact digests to the producing run descriptor and operator segment, then provenance claims are verifiable by replay.

Proof. Replay produces identical trace digests and artifact digests. The manifest binds artifacts to the generating run and segment, which can be checked by comparing reproduced digests and segment identifiers. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Concrete delta encodings and proof-pack schemas are introduced later. This node fixes the minimal audit carrier structure.

F7. Examples (non-definitional)

Example (informative). A gate evidence item may include $(m_{\mathrm{id}}, z, \eta, \mathrm{MCI}, \Delta\mu)$ snapshots over a window, all tied to specific trace indices.

F8. Limitations

Limitations. This node does not prescribe storage formats (JSON, CBOR, etc.). It prescribes semantic requirements for what must be recorded.

F9. Local DoD

Local DoD. Complete iff: (i) $H = (\mathcal{T}, \mathcal{E}, \mathcal{M}, \mathcal{D})$ is defined, (ii) evidence and manifest determinism requirements are fixed, (iii) replay/audit consequences are derived and proved.

0.2.7 II.7 Canonicalization and state equivalence for Σ (P-II.C7)

F1. Purpose

Purpose. This node defines how canonicalization and semantic equivalence apply to the full PSP state tuple Σ, including component-wise normalization and the treatment of history.

F2. Dependencies

Dependencies. Requires §0.1.4, §0.1.7, and §0.2.1–§0.2.6.

F3. Definitions (Defined)

(Defined) State equivalence relation. Define $\Sigma \sim_\Sigma \Sigma'$ iff:

E1. $G \sim_G G'$ (graph equivalence declared by the model),
E2. $z = z'$ after applying declared normalization to each,
E3. Π contains the same projection identifiers and versions (order irrelevant),
E4. μ identifiers and parameterizations match in RD,
E5. $RD = RD'$ (same determinism contract),
E6. H may differ only by redundant encodings that do not alter the trace/manifest identity (declared by the model).

(Defined) Canonicalization of Σ. Define

$$\mathrm{Canon}(\Sigma) := \big(\mathrm{Canon}_G(G),\ \mathrm{Canon}_z(z),\ \mathrm{Canon}_\Pi(\Pi),\ \mu,\ \mathrm{Canon}_H(H),\ RD\big),$$

where each component canonicalizer is deterministic and idempotent, and uses only information fixed in RD.

(Defined) History canonicalization constraint. $\mathrm{Canon}_H(H)$ MUST preserve the trace and manifest identities (digests, operator IDs, artifact bindings). It MAY drop redundant encodings (e.g. alternative formatting) but MUST NOT change any digest-bearing field.

F4. Derived statements

(Derived) Canonicalization collapses $\sim_\Sigma$. If each component canonicalizer collapses its component equivalence and is deterministic/idempotent, then

$$\Sigma \sim_\Sigma \Sigma' \Rightarrow \mathrm{Canon}(\Sigma) = \mathrm{Canon}(\Sigma').$$

Proof. By definition of $\sim_\Sigma$, each component is equivalent under its declared equivalence. Each component canonicalizer collapses that equivalence, yielding equal canonical components. Therefore the canonical tuples coincide. □

(Derived) Canonicalized histories preserve replay identity. If Canon_H preserves digest-bearing fields, then replay identity checks based on trace and artifact digests are invariant under history canonicalization.

Proof. Replay identity depends on the trace/manifest digests and their binding fields. If these are preserved, any history canonicalization cannot alter replay identity checks. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Specific choices for $G \sim_G G'$ and history redundancy allowances are model-specific; they must not alter replay-critical identity fields.

F7. Examples (non-definitional)

Example (informative). Two histories may be considered equivalent if they store the same trace digests but differ in auxiliary human-readable annotations.

F8. Limitations

Limitations. This node does not define a concrete canonical labeling for graphs or a concrete digest algorithm. It fixes the canonicalization structure and constraints.

F9. Local DoD

Local DoD. Complete iff: (i) $\sim_\Sigma$ is defined, (ii) Canon(Σ) is defined component-wise, (iii) history canonicalization preserves digest-bearing identity, (iv) collapse and replay-invariance properties are derived and proved.

0.2.8 II.8 Run descriptor RD as a typed object (P-II.C8)

F1. Purpose

Purpose. This node formalizes the run descriptor RD as a typed object inside Σ, making determinism, replay, canonicalization, and tie-break rules first-class and auditable.

F2. Dependencies

Dependencies. Requires §0.1.4 and §0.2.1.

F3. Definitions (Defined)

(Defined) Run descriptor domain. $RD \in \mathcal{R}$ is a finite record with required fields:

$$RD := (rd\_ver,\ seed,\ canon\_id,\ params,\ env,\ tie,\ hash,\ profile\_id,\ budgets).$$

(Defined) Required fields.

RD1. rd_ver: specification version identifier.
RD2. $seed$: deterministic seed value (may be constant).
RD3. $canon\_id$: identifier for canonicalization rules.
RD4. $params$: finite map of constants/thresholds used by operators and gates.
RD5. env: finite map of execution-relevant constraints (numeric domain, rounding mode, etc.).
RD6. tie: deterministic tie-break rule identifier and parameters.
RD7. $hash$: digest algorithm identifiers used in trace and manifest.
RD8. $profile\_id$: active measurement profile identifier and version.
RD9. $budgets$: termination/convergence budgets (cf. §0.1.9).

(Defined) RD well-formedness. $\mathrm{WF}_{RD}(RD) = 1$ iff: (i) all required fields exist, (ii) each referenced identifier resolves to a declared rule set in the document or in the model boundary statement, (iii) $params$ contains all constants required by executed operators, (iv) env fully fixes numeric semantics relevant to determinism.

(Defined) RD immutability. Within a PSP run, RD MUST be immutable. Any change to RD defines a different run.

F4. Derived statements

(Derived) RD immutability prevents mid-run semantic drift. If RD is immutable, then no operator can silently change replay semantics mid-run.

Proof. Replay semantics are fixed by RD (canonicalization, hash, tie-break, numeric domain). If RD cannot change during execution, then any operator outcome is evaluated under the same determinism contract throughout, preventing mid-run drift. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Model classes may extend RD with additional fields, but MUST preserve required fields and immutability.

F7. Examples (non-definitional)

Example (informative). A model may add $rd.$`float_mode` or $rd.$`fixed_q` to pin down numeric evaluation exactly.

F8. Limitations

Limitations. This node does not specify concrete rule sets; it specifies the RD schema and immutability requirements.

F9. Local DoD

Local DoD. Complete iff: (i) RD is formalized with required fields, (ii) WF_{RD} is fixed, (iii) immutability is stated normatively and its consequence proved.

0.2.9 II.9 State merge operator $\oplus$ and monotone merge discipline (P-II.C9)

F1. Purpose

Purpose. This node defines a canonical merge operator $\oplus$ for composing partial results (parallel operator paths, multi-candidate synthesis, dual-fabric evaluation) into a single PSP state while preserving determinism and well-formedness.

F2. Dependencies

Dependencies. Requires §0.2.1–§0.2.8.

F3. Definitions (Defined)

(Defined) Merge operator. A merge operator is a deterministic function

$$\oplus : \Sigma \times \Sigma \to \Sigma$$

that is admissible only when both inputs are well-formed and share the same RD.

(Defined) Merge admissibility. Define

$$\mathrm{Adm}_{\oplus}(\Sigma_a, \Sigma_b, RD) = 1 \iff \mathrm{WF}(\Sigma_a) = \mathrm{WF}(\Sigma_b) = 1 \ \wedge \ RD_a = RD_b = RD.$$

(Defined) Component-wise merge schema. Let $\Sigma_a = (G_a, z_a, \Pi_a, \mu_a, H_a, RD)$ and $\Sigma_b = (G_b, z_b, \Pi_b, \mu_b, H_b, RD)$. A canonical merge has the form:

$$\Sigma_a \oplus \Sigma_b := \big(G_{ab},\ z_{ab},\ \Pi_{ab},\ \mu_{ab},\ H_{ab},\ RD\big),$$

where each component merge rule is deterministic and declared:

- $G_{ab} := \mathrm{Merge}_G(G_a, G_b; RD)$,
- $z_{ab} := \mathrm{Merge}_z(z_a, z_b; RD)$,
- $\Pi_{ab} := \mathrm{Merge}_\Pi(\Pi_a, \Pi_b; RD)$,
- $\mu_{ab} := \mathrm{Merge}_\mu(\mu_a, \mu_b; RD)$,
- $H_{ab} := \mathrm{Merge}_H(H_a, H_b; RD)$.

(Defined) Merge determinism requirement. Each Merge$_\bullet$ MUST be a pure function of canonicalized inputs and RD and MUST use deterministic ordering.

(Defined) Merge monotonicity (optional). A merge is monotone w.r.t. a potential μ iff

$$\mu(\Sigma_a \oplus \Sigma_b) \leq \max\{\mu(\Sigma_a), \mu(\Sigma_b)\}.$$

Monotonicity is model-specific; if claimed, it MUST be proved or labeled External.

F4. Derived statements

(Derived) Deterministic merge preserves replay stability. If all component merges are deterministic under RD, then $\oplus$ is replay-stable.

Proof. Replay reproduces canonicalized inputs. Deterministic component merges produce identical merged components under replay, hence the merged state is identical under replay. □

(Derived) Merge admissibility prevents cross-contract blending. If merge requires identical RD, then it is impossible to silently combine states from incompatible determinism contracts.

Proof. By definition, $\mathrm{Adm}_\oplus = 1$ only when $RD_a = RD_b$. Therefore incompatible contracts cannot be merged. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs. Merge monotonicity is optional and not asserted here.

F6. Model notes

Model note. Concrete merge semantics depend on the model (graph union vs. graph selection, vector averaging vs. Pareto merge, etc.). This node fixes only admissibility and determinism requirements.

F7. Examples (non-definitional)

Example (informative). A dual-fabric run may produce $\Sigma^{(1)}$ and $\Sigma^{(2)}$; a merge may attach both as audited branches in H_{ab} while selecting a canonical committed branch by deterministic tie-break.

F8. Limitations

Limitations. This node does not mandate a particular merge behavior beyond determinism and RD compatibility.

F9. Local DoD

Local DoD. Complete iff: (i) $\oplus$ is defined as deterministic and RD-compatible, (ii) component-wise merge schema is fixed, (iii) replay stability and contract separation are derived and proved.

0.2.10 II.10 State pseudo-distance d and measurement compatibility (P-II.C10)

F1. Purpose

Purpose. This node defines a deterministic pseudo-distance d over Σ used for convergence/termination and robustness claims. It links d to measurement profiles and canonicalization.

F2. Dependencies

Dependencies. Requires §0.1.9, §0.2.3, and §0.2.7.

F3. Definitions (Defined)

(Defined) Pseudo-distance. A pseudo-distance is a deterministic map

$$d : \Sigma \times \Sigma \to \mathbb{R}_{\geq 0}$$

satisfying:

D1. $d(\Sigma, \Sigma) = 0$ (reflexivity),
D2. $d(\Sigma, \Sigma') = d(\Sigma', \Sigma)$ (symmetry),
D3. triangle inequality MAY hold (if claimed, it must be proved or labeled External).

(Defined) Canonical compatibility. d is canonical-compatible iff

$$\Sigma \equiv \Sigma' \Rightarrow d(\Sigma, \Sigma') = 0.$$

(Defined) Profile-induced distance. Given a profile $m : \Sigma \to \mathbb{D}^n$ and a norm $\|\cdot\|$ on $\mathbb{D}^n$, define

$$d_m(\Sigma, \Sigma') := \|m(\Sigma) - m(\Sigma')\|.$$

If $\mathbb{D}$ is fixed-point, the norm evaluation MUST be defined deterministically in *RD*.

(Defined) Distance identifier. The active distance d MUST be identified and parameterized in *RD* (including tolerance ε and window N).

F4. Derived statements

(Derived) Canonical-compatible distances preserve semantic convergence. If d is canonical-compatible, then convergence in d implies stabilization up to semantic equivalence.

Proof. If Conv_ε holds with sufficiently small ε and d collapses semantic equivalence to 0, then the trajectory stabilizes in the equivalence class induced by canonicalization. Thus convergence reflects semantic, not representational, stabilization. □

(Derived) Profile-induced distances are replay-stable under deterministic profiles. If m is deterministic under *RD* and the norm evaluation is deterministic, then d_m is replay-stable.

Proof. Replay reproduces the same $m(\Sigma)$ values and the same deterministic norm evaluation, hence d_m yields identical results. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs. Triangle inequality is not claimed.

F6. Model notes

Model note. Graph-edit distances and hybrid distances are model-specific; they must still be deterministic and RD-bound to be admissible.

F7. Examples (non-definitional)

Example (informative). A run may use d_m for termination, while using a separate gate evidence metric (e.g. MCI) for acceptance.

F8. Limitations

Limitations. This node does not require d to be a metric. It requires determinism, RD identity, and canonical compatibility when semantic convergence is claimed.

F9. Local DoD

Local DoD. Complete iff: (i) pseudo-distance is defined with minimal laws, (ii) canonical compatibility is defined, (iii) profile-induced distance is defined, (iv) replay/semantic stabilization properties are derived and proved.

0.2.11 II.11 Numeric domains and rounding discipline (P-II.C11)

F1. Purpose

Purpose. This node fixes numeric-domain obligations required for determinism. It does not mandate a specific numeric type, but it specifies the constraints any numeric domain must satisfy to avoid replay drift.

F2. Dependencies

Dependencies. Requires §0.1.4 and §0.2.3.

F3. Definitions (Defined)

(Defined) Numeric domain. The numeric domain $\mathbb{D}$ used for z and for numeric operator evaluations MUST be declared in *RD*.

(Defined) Rounding discipline. If $\mathbb{D}$ requires rounding (fixed-point, finite precision floats), the rounding mode MUST be fixed in *RD* and MUST be applied consistently.

(Defined) Deterministic arithmetic. Arithmetic operations on $\mathbb{D}$ used by PSP MUST be deterministic functions of operands and *RD*. If platform-dependent behavior exists, it MUST be eliminated by explicit normalization or by choosing a deterministic numeric domain.

(Defined) Numeric evidence. Any gate or proof object that relies on numeric thresholds MUST record the thresholds and the numeric domain identifiers in *RD*.

F4. Derived statements

(Derived) Fixed rounding is necessary for replay stability under finite precision. If numeric evaluations use finite precision without fixed rounding discipline, replay stability can fail due to platform variation.

Proof. Finite precision arithmetic can vary by implementation (evaluation order, rounding). If rounding mode and evaluation rules are not fixed, the same semantic expression may yield different numeric results across runs or platforms, breaking determinism. Fixing rounding and evaluation eliminates this source. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. A model may choose Q16.16 fixed-point, rational arithmetic, or constrained floating-point. The choice must be recorded in *RD*.

F7. Examples (non-definitional)

Example (informative). If a profile uses $\|z\|_2$, the summation order and rounding must be fixed (or computed in a deterministic accumulator) to avoid drift.

F8. Limitations

Limitations. This node does not impose performance constraints; it imposes determinism constraints.

F9. Local DoD

Local DoD. Complete iff: (i) numeric domain declaration is mandatory, (ii) rounding/evaluation determinism is mandatory, (iii) threshold evidence must bind to RD, (iv) replay-stability requirement is derived and proved.

0.3 Part III — Operator Calculus (P-III)

0.3.1 III.1 Operators: typing, admissibility, and registry (P-III.C1)

F1. Purpose

Purpose. This node defines PSP operators as typed state transformers and fixes the operator registry discipline: every operator must have a declared signature, admissibility predicate, determinism requirements, and an identity for traceability.

F2. Dependencies

Dependencies. Requires Part I (SI2, determinism, status taxonomy) and Part II (Σ, RD, canonicalization).

F3. Definitions (Defined)

(Defined) Operator. An operator is a function $\Omega : \Sigma \to \Sigma$ together with: (i) an identifier Ω_{id} (name+version), (ii) admissibility predicate $\mathrm{Adm}_\Omega : \Sigma \times RD \to \{0,1\}$, (iii) evidence payload schema Ev_Ω (may be empty), (iv) a declared determinism class (pure / RD-pure / external-IO).

(Defined) Operator well-formedness. An operator is well-formed iff:

O1. $\Omega : \Sigma \to \Sigma$ is typed and total on admissible inputs,
O2. Adm_Ω is defined and deterministic under RD,
O3. Ω preserves well-formedness: $\mathrm{WF}(\Sigma) = 1 \wedge \mathrm{Adm}_\Omega(\Sigma, RD) = 1 \Rightarrow \mathrm{WF}(\Omega(\Sigma)) = 1$,
O4. Ω_{id} and its parameters are recorded in RD or in the trace,
O5. if Ω emits evidence, it is a deterministic function of canonicalized states and RD.

(Defined) Operator registry. The operator registry $\mathcal{R}_\Omega$ is the set of all operators used in the system, each entry containing:

$$(\Omega_{\mathrm{id}},\ \mathrm{sig},\ \mathrm{Adm}_\Omega,\ \mathrm{DetClass},\ \mathrm{EvSchema},\ \mathrm{Params}).$$

All entries MUST be versioned. If an operator changes semantics, its version MUST change.

(Defined) Operator identity binding. Every trace step MUST record Ω_{id} and sufficient parameter identity to replay the step deterministically.

F4. Derived statements

(Derived) Registry discipline implies operator traceability. If every executed operator step records Ω_{id} and parameter identity, then every state transition in the trace can be attributed to a unique operator instance.

Proof. A trace entry contains $(\Omega_{\mathrm{id}}, \mathrm{in}, \mathrm{out})$. If Ω_{id} and its parameter identity uniquely select an operator instance from $\mathcal{R}_\Omega$, then the transition is attributable to that instance. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Model classes may define additional operator categories (e.g. geometric operators, boundary operators), but the registry discipline and admissibility/determinism requirements are invariant.

F7. Examples (non-definitional)

Example (informative). A "measure" operator may compute $z := m(\Sigma)$ and attach (m_{id}, z) as evidence payload.

F8. Limitations

Limitations. This node does not prescribe a specific operator set. It fixes the universal requirements for any operator to be admissible in PSP.

F9. Local DoD

Local DoD. Complete iff: (i) operators are defined as typed transformers with admissibility, (ii) operator well-formedness criteria are fixed, (iii) operator registry and traceability rules are fixed.

0.3.2 III.2 Program forms: sequence, DAG, and HDAG (P-III.C2)

F1. Purpose

Purpose. This node defines admissible PSP program forms: linear operator sequences and operator graphs (DAG/HDAG). It fixes execution determinism and forbids semantic dependence on unspecified scheduling.

F2. Dependencies

Dependencies. Requires §0.3.1 and determinism/tie-break discipline from §0.1.4.

F3. Definitions (Defined)

(Defined) Sequential program. A sequential program is a finite composition

$$P := \Omega_k \circ \cdots \circ \Omega_1.$$

(Defined) Operator DAG program. An operator DAG program is a directed acyclic graph

$$\mathcal{P} := (U, A, \lambda),$$

where U is a finite node set, $A \subseteq U \times U$ is an acyclic edge set, and $\lambda : U \to \mathcal{R}_\Omega$ labels each node with an operator registry entry. Edges encode data/state dependencies.

(Defined) Operator HDAG program. An operator HDAG program is an operator DAG equipped with additional structure for hierarchical or hyperdimensional scheduling, but MUST still admit a deterministic topological evaluation order.

(Defined) Deterministic evaluation order. For DAG/HDAG programs, evaluation proceeds in a topological order. If multiple nodes are simultaneously schedulable, the tie-break rule $\prec$ from RD MUST select a unique next node deterministically as a function of canonicalized state and RD.

(Defined) Program well-typedness. A program is well-typed iff every executed operator application satisfies its admissibility predicate on the current state and preserves WF.

F4. Derived statements

(Derived) Acyclicity plus deterministic tie-break yields deterministic schedules. If $\mathcal{P}$ is acyclic and tie-break is deterministic, then the sequence of executed nodes is deterministic (given the same initial state and RD).

Proof. Acyclicity guarantees that at each step, there exists at least one schedulable node whose predecessors have completed. If multiple nodes are schedulable, deterministic tie-break selects one uniquely as a function of state and RD. Thus the schedule is uniquely determined. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. HDAG-specific scheduling metadata is model-specific; determinism requirements are not.

F7. Examples (non-definitional)

Example (informative). A program may run two independent operator subchains in parallel conceptually and then merge their states via $\oplus$, as long as the scheduling and merge are deterministic.

F8. Limitations

Limitations. This node does not define parallel runtime execution. It defines semantic program forms and deterministic evaluation rules.

F9. Local DoD

Local DoD. Complete iff: (i) sequence and DAG/HDAG program forms are defined, (ii) deterministic evaluation order is defined, (iii) determinism of schedules is derived and proved.

0.3.3 III.3 Purity, side effects, and IO boundaries (P-III.C3)

F1. Purpose

Purpose. This node classifies operator purity and fixes how side effects are treated so that determinism and replay are preserved. It defines the only admissible way in which IO can influence semantics: via explicit boundary operators and recorded evidence.

F2. Dependencies

Dependencies. Requires determinism contract §0.1.4 and operator registry §0.3.1.

F3. Definitions (Defined)

(Defined) Pure operator. An operator Ω is pure iff $\Omega(\Sigma)$ depends only on Canon(Σ) and RD and produces no external side effects.

(Defined) RD-pure operator. An operator is RD-pure iff it is pure except that it may read parameters/constants from RD (already part of purity).

(Defined) Boundary operator. A boundary operator is an operator that interacts with an external environment. Boundary operators MUST: (i) record all external inputs as canonicalized evidence in H, (ii) treat external inputs as part of the effective run input (thus replay requires reusing that evidence), (iii) declare their admissibility conditions explicitly.

(Defined) Side-effect prohibition. Operators MUST NOT rely on hidden side effects (ambient time, ambient randomness, undefined IO order). Any interaction must occur via a declared boundary operator and must be recorded.

F4. Derived statements

(Derived) Boundary evidence converts external IO into replayable inputs. If a boundary operator records all external inputs as canonicalized evidence, then replay can reproduce identical operator outcomes by reusing recorded evidence rather than re-querying the environment.

Proof. During replay, the operator reads the same canonicalized evidence instead of querying the environment. Therefore the operator's effective input is identical, yielding identical outputs under determinism. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Model families may define specialized boundary states (messages, headers, masked payloads), but must still treat IO via boundary operators.

F7. Examples (non-definitional)

Example (informative). A compilation step that calls an external compiler can be modeled as a boundary operator if the exact compiler version, inputs, and outputs are recorded and replayed deterministically.

F8. Limitations

Limitations. This node does not define a concrete serialization format for evidence. It fixes the semantic rule: external inputs must be recorded as replayable evidence.

F9. Local DoD

Local DoD. Complete iff: (i) purity classes are defined, (ii) boundary operators and recording requirements are defined, (iii) hidden side effects are prohibited, (iv) replay conversion property is derived and proved.

0.3.4 III.4 Minimal kernel operator set (P-III.C4)

F1. Purpose

Purpose. This node defines the minimal operator kernel sufficient to express PSP runs: ingest, canon, measure, solve, morph, gate, select, merge, and emit. It fixes their roles without committing to specific implementations.

F2. Dependencies

Dependencies. Requires Parts I–II and operator discipline §0.3.1–§0.3.3.

F3. Definitions (Defined)

(Defined) Kernel operator roles. The kernel consists of role-operators (each may be instantiated by multiple concrete operators):

- Ω_{ingest}: external domain input $\to$ initial state construction,
- Ω_{canon}: canonicalization enforcement (may wrap Canon),
- Ω_{measure}: compute z via profile m and attach evidence,
- Ω_{solve}: constraint satisfaction / monotone improvement step (may use μ),
- Ω_{morph}: structure expansion / transformation within admissibility,
- Ω_{gate}: evaluate acceptance predicates and attach gate evidence,
- Ω_{select}: deterministic selection among candidates (tie-break via RD),
- $\Omega_{\oplus}$: deterministic merge (Part II),
- Ω_{emit}: artifact emission + manifest binding into H.

(Defined) Kernel completeness. A PSP system is kernel-complete iff every run can be expressed as a composition (sequence or DAG) of operators that instantiate these roles.

(Defined) Role separation. An operator may implement multiple roles, but role separation MUST remain observable in the trace via role tags or step labels, so that evidence and admissibility can be audited per role.

F4. Derived statements

(Derived) Kernel roles suffice for closed-loop PSP runs. Given (i) ingest to create Σ_0, (ii) repeated solve/morph/measure with gates, and (iii) emit to produce artifacts with manifests, a PSP run can be expressed without requiring any additional semantic primitive.

Proof. The run lifecycle consists of state construction, iterative transformation with measurement and gating, and artifact emission with provenance. Each phase maps directly to kernel roles. Any additional operator can be treated as a specialization of solve/morph/measure/gate/select, hence the kernel is semantically sufficient. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Model classes may introduce specialized operator families (e.g. lattice operators, boundary masking operators), but they must still reduce to kernel roles for audit and conformance.

F7. Examples (non-definitional)

Example (informative). A dual-fabric evaluation may be expressed as two parallel subprograms sharing ingest/canon, each with its own solve/morph chain, followed by merge $\oplus$ and a final gate.

F8. Limitations

Limitations. This node does not prescribe particular constraints or objective functions; it fixes the minimal semantic roles.

F9. Local DoD

Local DoD. Complete iff: (i) kernel roles are enumerated and defined, (ii) kernel completeness is defined, (iii) role separation observability is required, (iv) kernel sufficiency is derived and proved.

0.3.5 III.5 Constraint operators, feasibility, and monotone improvement (P-III.C5)

F1. Purpose

Purpose. This node defines constraint operators and the notion of feasibility in PSP. It formalizes the "solve" role as a state transformer that reduces constraint violation and/or decreases a potential functional without breaking determinism.

F2. Dependencies

Dependencies. Requires §0.2.5 (potential μ), §0.1.9 (termination), and §0.3.1 (operator discipline).

F3. Definitions (Defined)

(Defined) Constraint family. A constraint family is a finite set $\mathcal{K} = \{K_j\}_{j=1}^{m}$ of predicates

$$K_j : \Sigma \to \{0, 1\}.$$

A run MUST identify the active constraint family $\mathcal{K}_{\text{id}}$ in RD.

(Defined) Feasibility. A state Σ is feasible w.r.t. $\mathcal{K}$ iff

$$\text{Feas}_{\mathcal{K}}(\Sigma) = 1 \iff \bigwedge_{j=1}^{m} K_j(\Sigma) = 1.$$

(Defined) Violation score. A violation score is a deterministic map

$$v_{\mathcal{K}} : \Sigma \to \mathbb{R}_{\geq 0},$$

with $v_{\mathcal{K}}(\Sigma) = 0 \Rightarrow \text{Feas}_{\mathcal{K}}(\Sigma) = 1$. If a stronger equivalence is claimed ($\Leftrightarrow$), it MUST be proved or labeled External.

(Defined) Solve operator. A solve operator Ω_{solve} is an operator that is admissible only under a declared constraint family and aims to reduce violation and/or descend in μ. Two admissible solve contracts are:

S1. **Violation contract:** $v_{\mathcal{K}}(\Omega_{\text{solve}}(\Sigma)) \leq v_{\mathcal{K}}(\Sigma)$.
S2. **Potential contract:** $\mu(\Omega_{\text{solve}}(\Sigma)) \leq \mu(\Sigma)$.

Any claimed strict improvement condition MUST be stated explicitly and proved or labeled External.

(Defined) Evidence obligation for solve. If a solve operator claims S1 or S2, it MUST attach evidence sufficient to verify the inequality using canonicalized inputs and RD.

F4. Derived statements

(Derived) Monotone solve steps prevent oscillation in the witness scalar. If each solve step satisfies S1 (or S2), then the corresponding witness sequence $v_{\mathcal{K}}(\Sigma_i)$ (or $\mu(\Sigma_i)$) is non-increasing.

Proof. Immediate from the inequality contract applied at each step. □

(Derived) Feasibility is stable under feasibility-preserving solves. If $\text{Feas}_{\mathcal{K}}(\Sigma) = 1$ and Ω_{solve} is designed to preserve feasibility (i.e. $v_{\mathcal{K}}(\Sigma) = 0 \Rightarrow v_{\mathcal{K}}(\Omega_{\text{solve}}(\Sigma)) = 0$), then feasibility is preserved.

Proof. If $v_{\mathcal{K}}(\Sigma) = 0$ implies feasibility and the operator preserves zero violation, then the post-state has zero violation and is feasible. □

F5. Proof or status

All claims are Defined or Derived with explicit proofs. No external theorems.

F6. Model notes

Model note. Concrete constraints and violation scores are model- or domain-specific. This node fixes only the admissible contracts.

F7. Examples (non-definitional)

Example (informative). A constraint family may require type-correct edges in G, bounded measurement coordinates in z, and deterministic manifest bindings in H.

F8. Limitations

Limitations. This node does not guarantee convergence of solve steps; it formalizes monotone improvement witnesses used for later convergence arguments.

F9. Local DoD

Local DoD. Complete iff: (i) constraints and feasibility are defined, (ii) solve operator contracts are defined, (iii) evidence obligations are fixed, (iv) monotone consequences are derived and proved.

0.3.6 III.6 Morph operators: expansion, rewrites, and structure preservation (P-III.C6)

F1. Purpose

Purpose. This node defines morph operators that transform the internal structure of the state (typically G and related components). It fixes admissibility, determinism, and preservation obligations to prevent uncontrolled growth or semantic drift.

F2. Dependencies

Dependencies. Requires §0.2.2 (typed graph), §0.1.4 (determinism), and §0.3.1 (operator discipline).

F3. Definitions (Defined)

(Defined) Morph operator. A morph operator Ω_{morph} is an operator that modifies structural components of Σ (e.g. G, Π, H) while preserving well-formedness.

(Defined) Structure preservation. A morph operator MUST preserve:

M1. WF_G (typing and admissible edges),
M2. RD immutability,
M3. trace/audit integrity (any structural edit must be representable in $H.\mathcal{D}$ or equivalent evidence),
M4. canonicalization compatibility (post-state must remain canonicalizable).

(Defined) Expansion budget. If a morph operator can grow the graph (add nodes/edges), it MUST be subject to an expansion budget contained in RD. Budget violation MUST render the operator inadmissible.

(Defined) Rewrite schema. A morph operator may be specified by a finite rewrite schema:

$$(\text{pre\_pattern},\ \text{post\_pattern},\ \text{side\_conditions}),$$

where matching and replacement are deterministic under canonicalization and RD.

F4. Derived statements

(Derived) Budgeted morphs prevent unbounded growth in admissible runs. If every graph-expanding morph is bounded by a finite expansion budget and budget exhaustion prevents further expansions, then any admissible run can contain only finitely many expansions.

Proof. A finite budget can be decremented only finitely often. If expansions require remaining budget, then expansions occur only finitely many times. □

(Derived) Deterministic rewrite schemas preserve replay identity. If pattern matching, tie-breaking among matches, and replacement are deterministic under RD, then morph steps are replay-stable.

Proof. Replay reproduces the same canonicalized graph and the same deterministic match selection. Therefore the chosen rewrite and resulting graph are identical. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Concrete rewrite grammars are model- and domain-specific. This node fixes only determinism, typing preservation, and budgeting obligations.

F7. Examples (non-definitional)

Example (informative). A morph may expand G by inserting a standardized "operator macro node" with typed edges to its dependencies and outputs, recorded as a delta.

F8. Limitations

Limitations. This node does not assert that morphing improves feasibility or potential. Such claims must be expressed via solve/gate operators or explicit contracts.

F9. Local DoD

Local DoD. Complete iff: (i) morph operators are defined, (ii) structure preservation obligations are fixed, (iii) expansion budgeting is fixed, (iv) replay-stability of deterministic rewrites is derived and proved.

0.3.7 III.7 Gate operators: predicates, evidence, and staged acceptance (P-III.C7)

F1. Purpose

Purpose. This node defines gate operators as the admissible mechanism for acceptance/rejection, termination gates, and staged admissibility checks. It fixes gate evidence requirements so that acceptance decisions are replayable and auditable.

F2. Dependencies

Dependencies. Requires §0.2.6 (history/evidence), §0.1.4 (replay), and §0.1.8 (robust acceptance).

F3. Definitions (Defined)

(Defined) Gate predicate. A gate predicate is a deterministic map $q : \Sigma \to \{0, 1\}$ whose parameters are fixed in RD.

(Defined) Gate operator. A gate operator Ω_{gate} evaluates one or more gate predicates and attaches evidence items to $H.\mathcal{E}$. Gate operators MUST NOT change semantic state except for appending evidence/annotations consistent with H well-formedness.

(Defined) Staged gate. A staged gate is an ordered tuple $(q_1, \ldots, q_r)$ with an acceptance rule

$$q_{\text{stage}}(\Sigma) = 1 \iff \bigwedge_{i=1}^{r} q_i(\Sigma) = 1.$$

If short-circuit evaluation is used, it MUST be deterministic and recorded (e.g. first failing stage index).

(Defined) Gate evidence. For each evaluated predicate q_i, evidence MUST include: (i) predicate identifier and version, (ii) all threshold parameters used, (iii) any measured values used by the predicate (e.g. profile snapshots), (iv) the boolean outcome. All evidence MUST be deterministic functions of canonicalized state and RD.

(Defined) Robust gate. A robust gate for task τ is a gate operator that applies the robust acceptance rule from §0.1.8 using two declared fabrics and an MCI threshold.

F4. Derived statements

(Derived) Gate decisions are replay-stable under deterministic evidence. If gate predicates and all measurements used by them are deterministic under RD and evidence is recorded deterministically, then gate outcomes and evidence items are replay-stable.

Proof. Replay reproduces identical canonicalized states and identical deterministic measurements. Therefore predicates evaluate identically, and recorded evidence matches identically. □

(Derived) Gates separate optimization from acceptance. If gates do not modify the semantic state (except audit metadata), then acceptance decisions cannot be confounded with optimization steps.

Proof. Optimization changes G, z, μ etc.; gates only read and record. Therefore acceptance is a predicate over state, not an implicit mutation. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Specific gates (PoR, phase conductance, mirror consistency, budget checks) are introduced later as model-specific predicates. This node fixes the abstract gate/evidence mechanism.

F7. Examples (non-definitional)

Example (informative). A three-stage gate may check: (i) feasibility $\mathrm{Feas}_{\mathcal{K}}$, (ii) potential descent over a window, (iii) robust dual-fabric agreement with MCI $\geq \eta$.

F8. Limitations

Limitations. This node does not assert that any particular predicate is sufficient for correctness; it defines how predicates become auditable decisions.

F9. Local DoD

Local DoD. Complete iff: (i) gate predicate and gate operator are defined, (ii) staged gates are defined, (iii) evidence obligations are fixed, (iv) replay-stability and separation properties are derived and proved.

0.3.8 III.8 Selection operators: ordering, Pareto sets, and tie-break determinism (P-III.C8)

F1. Purpose

Purpose. This node defines selection operators that choose among candidates (states, rewrites, artifacts) and fixes deterministic ordering requirements. It enables multi-objective optimization without nondeterministic drift.

F2. Dependencies

Dependencies. Requires §0.1.4 (tie-break) and §0.2.3 (measurement profiles).

F3. Definitions (Defined)

(Defined) Candidate set. A candidate set is a finite set $\mathcal{C} \subseteq \Sigma$ (or a finite set of candidate actions) produced by upstream operators.

(Defined) Score map. A score map is a deterministic function $s : \mathcal{C} \to \mathbb{D}^p$ producing one or more objective coordinates.

(Defined) Total order selection. A total order selection chooses

$$\mathrm{Select}_{\prec}(\mathcal{C}) := \arg\min_{\Sigma \in \mathcal{C}} \langle s(\Sigma), RD \rangle_{\prec},$$

where $\prec$ is a deterministic total order derived from RD (including tie-break rules).

(Defined) Pareto selection. For multi-objective scores, define Pareto dominance $\Sigma \prec_P \Sigma'$ iff $s(\Sigma)$ is no worse in all coordinates and strictly better in at least one. The Pareto set $\mathrm{Pareto}(\mathcal{C})$ is the set of non-dominated candidates. If a single candidate must be chosen from the Pareto set, deterministic tie-break MUST be applied.

(Defined) Selection admissibility. Selection is admissible only if $\mathcal{C}$ is finite and the tie-break rule yields a unique output.

F4. Derived statements

(Derived) Deterministic tie-break yields unique selection. If the tie-break order $\prec$ is total and deterministic, then selection from any finite candidate set yields a unique result.

Proof. A total order on a finite set has a unique minimal element. Determinism ensures the same order is used under replay. □

(Derived) Pareto selection is replay-stable under deterministic scoring. If s is deterministic under RD, then the Pareto set and any tie-broken selection from it are replay-stable.

Proof. Deterministic scoring yields identical score vectors under replay, hence identical dominance relations and Pareto sets. Tie-break determinism yields identical chosen element. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Score coordinates may come from profiles, potentials, feasibility margins, or gate evidence. This node fixes only the deterministic selection logic.

F7. Examples (non-definitional)

Example (informative). A selector may choose the candidate that minimizes violation score and then maximizes MCI, breaking remaining ties by digest order.

F8. Limitations

Limitations. This node does not prescribe which objectives are correct. It prescribes how objectives are used without nondeterminism.

F9. Local DoD

Local DoD. Complete iff: (i) selection and scoring are defined, (ii) Pareto selection is defined, (iii) deterministic uniqueness and replay-stability are derived and proved.

0.4 Part IV — Measurement Profiles (P-IV)

0.4.1 IV.1 Profile taxonomy and coordinate semantics (P-IV.C1)

F1. Purpose

Purpose. This node classifies measurement profiles as first-class semantic instruments in PSP. It fixes the requirement that every coordinate in z has declared meaning, normalization, and admissible range, and that profile changes are explicit and versioned.

F2. Dependencies

Dependencies. Requires §0.2.3 (profiles), §0.2.11 (numeric discipline), and §0.1.4 (RD).

F3. Definitions (Defined)

(Defined) Coordinate semantics declaration. For a profile $m : \Sigma \to \mathbb{D}^n$, each coordinate z_i MUST be declared as a triple

$$(z_i.\text{name},\ z_i.\text{meaning},\ z_i.\text{range}),$$

where meaning is a precise definition (not a metaphor) and range is a declared admissible interval (e.g. $[0, 1]$ after normalization).

(Defined) Profile classes. A profile MUST declare its class label from the set:

$$\mathcal{C}_m := \{\text{Structure},\ \text{Symmetry},\ \text{Topology},\ \text{Resonance},\ \text{Feasibility},\ \text{Audit}\}.$$

A profile may declare multiple class labels.

(Defined) Profile versioning. A profile is identified by $(m_{\text{id}}, m_{\text{ver}})$ and MUST be recorded in RD. Any change to coordinate meaning, normalization, or evaluation rules MUST increment m_{ver}.

(Defined) Profile well-formedness. A profile is well-formed iff:

PM1. m is deterministic under RD,
PM2. each coordinate semantics declaration exists,
PM3. normalization rules are explicit and RD-bound,
PM4. the numeric domain and rounding rules are fixed in RD,
PM5. the profile has an admissibility predicate $\text{Adm}_m(\Sigma, RD)$ (may be trivially 1).

F4. Derived statements

(Derived) Explicit coordinate semantics prevents implicit meaning drift. If each coordinate has declared meaning and profiles are versioned in RD, then changes to measurement meaning cannot occur silently.

Proof. Any meaning change requires a version increment, and RD records the active profile id/version. Therefore a run's coordinate semantics are fixed and auditable. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Concrete coordinate meanings (e.g. tripolar axes, operator-energy axes, codematrix axes) are introduced in model sections and must remain Model-Specific unless proved portable.

F7. Examples (non-definitional)

Example (informative). A "Resonance" profile may include coordinates for stability margin, phase coherence, and gate-evidence strength, each normalized to $[0, 1]$.

F8. Limitations

Limitations. This node does not prescribe which coordinates are "correct". It prescribes how coordinates become explicit, deterministic, and auditable.

F9. Local DoD

Local DoD. Complete iff: (i) coordinate semantics declaration is required, (ii) profile classes are defined, (iii) versioning rules are fixed, (iv) profile well-formedness is fixed.

0.4.2 IV.2 Profile admissibility, test vectors, and calibration (P-IV.C2)

F1. Purpose

Purpose. This node defines admissibility conditions for profiles and fixes the notion of calibration via test vectors. It ensures that profiles are not arbitrary numbers but reproducible measurement instruments.

F2. Dependencies

Dependencies. Requires §0.4.1, §0.1.4, and §0.2.6.

F3. Definitions (Defined)

(Defined) Profile admissibility. A profile m is admissible for a run iff:

$$\mathrm{Adm}_m(\Sigma, RD) = 1$$

and all calibration parameters (normalization constants, thresholds, numeric domain) are recorded in RD.

(Defined) Test vector set. A profile test vector set is a finite set

$$\mathcal{V}_m := \{(\Sigma^{(j)}, z^{(j)})\}_{j=1}^{N_m},$$

where each $\Sigma^{(j)}$ is a canonicalized state fixture and $z^{(j)}$ is the expected measurement output under m and RD.

(Defined) Calibration. A profile is calibrated w.r.t. $\mathcal{V}_m$ iff for all $(\Sigma^{(j)}, z^{(j)})$:

$$m(\Sigma^{(j)}) = z^{(j)}$$

under the declared numeric domain and rounding rules.

(Defined) Calibration evidence. A run MAY record calibration checks in $H.\mathcal{E}$ as evidence items:

$$(\, m_{\mathrm{id}}, m_{\mathrm{ver}},\ \mathrm{Dig}(\Sigma^{(j)}),\ m(\Sigma^{(j)}),\ z^{(j)},\ \text{pass/fail}\,).$$

F4. Derived statements

(Derived) Calibration yields profile regression detectability. If a profile is calibrated on a fixed test vector set and the run records $m_{\mathrm{id}}, m_{\mathrm{ver}}$, then any change to m that affects outputs is detectable by test failure.

Proof. A meaning-changing modification to m changes at least one $m(\Sigma^{(j)})$ unless it is behaviorally identical on all fixtures. The calibration check compares measured and expected outputs, thus flags deviations. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Fixtures may be model-specific (graphs/tensors/messages). The requirement is determinism and canonicalization of fixtures, not a specific fixture format.

F7. Examples (non-definitional)

Example (informative). A test vector may include a minimal graph with one node and no edges, expecting a fixed "complexity" coordinate of 0 after normalization.

F8. Limitations

Limitations. Calibration does not prove that a profile is semantically correct for all domains; it proves reproducibility and detects drift.

F9. Local DoD

Local DoD. Complete iff: (i) admissibility and calibration are defined, (ii) test vectors are defined as canonical fixtures, (iii) regression detectability is derived and proved.

0.4.3 IV.3 Profile translation maps and cross-profile comparability (P-IV.C3)

F1. Purpose

Purpose. This node defines when and how measurements from different profiles can be compared. It introduces translation maps between profiles and forbids undefined comparisons.

F2. Dependencies

Dependencies. Requires §0.2.3 and §0.4.1.

F3. Definitions (Defined)

(Defined) Translation map. Given two profiles $m_a : \Sigma \to \mathbb{D}^{n_a}$ and $m_b : \Sigma \to \mathbb{D}^{n_b}$, a translation map is a deterministic function

$$T_{a\to b} : \mathbb{D}^{n_a} \to \mathbb{D}^{n_b}$$

such that, for an admissible class of states $\mathcal{S}_{ab} \subseteq \Sigma$,

$$T_{a\to b}(m_a(\Sigma)) \approx m_b(\Sigma) \quad \forall\, \Sigma \in \mathcal{S}_{ab},$$

where the approximation tolerance is fixed in RD.

(Defined) Translation admissibility. A translation map is admissible iff:

TR1. it is deterministic under RD,
TR2. it declares its valid state class $\mathcal{S}_{ab}$,
TR3. it declares an error bound (tolerance) recorded in RD,
TR4. it is versioned and identified in RD when used.

(Defined) Cross-profile comparability. Cross-profile numeric comparisons are admissible only if a translation map is declared and applied, or if the two profiles share identical coordinate semantics.

F4. Derived statements

(Derived) Undefined cross-profile comparisons cause semantic ambiguity. If m_a and m_b have different coordinate meanings and no translation is declared, then comparisons of $m_a(\Sigma)$ and $m_b(\Sigma)$ are semantically undefined.

Proof. Measurements are meaningful only relative to their coordinate semantics. Without a translation or identical semantics, there is no defined mapping between coordinates. Hence numeric comparisons lack semantic interpretation. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Translation maps are often model-specific and may be valid only on restricted state classes; such restrictions must be explicit.

F7. Examples (non-definitional)

Example (informative). A translation may map a "structure complexity" coordinate into a "feasibility slack" coordinate only on states within a fixed constraint family.

F8. Limitations

Limitations. This node does not guarantee existence of translation maps; it fixes the admissibility conditions for using them.

F9. Local DoD

Local DoD. Complete iff: (i) translation maps are defined with admissibility rules, (ii) cross-profile comparability is restricted accordingly, (iii) semantic ambiguity of undefined comparisons is derived and proved.

0.4.4 IV.4 Profile-induced distances and robustness metrics (P-IV.C4)

F1. Purpose

Purpose. This node links profiles to distances and robustness metrics used in convergence and dual-fabric consistency. It formalizes how measurement maps induce replay-stable distances and how tolerances are bound to *RD*.

F2. Dependencies

Dependencies. Requires §0.2.10 (distance), §0.1.8 (MCI), and §0.4.1–§0.4.2.

F3. Definitions (Defined)

(Defined) Profile norm. A profile norm $\|\cdot\|_m$ is a deterministic evaluation rule for vectors in $\mathbb{D}^n$ under *RD*.

(Defined) Induced distance. Given profile m and norm $\|\cdot\|_m$, define the induced distance

$$d_m(\Sigma, \Sigma') := \|m(\Sigma) - m(\Sigma')\|_m.$$

The choice of $\|\cdot\|_m$ MUST be recorded in *RD*.

(Defined) Robustness tolerance. A robustness tolerance is a tuple (ϵ_m, η) where ϵ_m is the MCI normalization tolerance and η is the acceptance threshold. Both MUST be recorded in *RD*.

F4. Derived statements

(Derived) Induced distances are replay-stable. If m and $\|\cdot\|_m$ are deterministic under RD, then d_m is replay-stable.

Proof. Replay reproduces identical measurements and identical norm evaluation, so the resulting distance is identical. □

(Derived) RD-bound tolerances prevent hidden robustness drift. If (ϵ_m, η) are fixed in RD, then robust acceptance thresholds cannot shift across runs without changing RD.

Proof. Acceptance is computed from deterministic measurements and fixed thresholds. Any threshold change constitutes a different RD, hence drift is explicit. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Different models may use different profile norms (e.g. ℓ_1, ℓ_2, max norm) and must declare them explicitly.

F7. Examples (non-definitional)

Example (informative). A system may use d_m for convergence termination and MCI for dual-fabric agreement, both driven by the same profile m.

F8. Limitations

Limitations. This node does not assert that any particular profile is sufficient for correctness; it defines measurement-driven robustness machinery.

F9. Local DoD

Local DoD. Complete iff: (i) induced distances are defined with RD-bound norms, (ii) robustness tolerances are defined and RD-bound, (iii) replay-stability and no-drift properties are derived and proved.

0.5 Part V — Constraint Lattice and Dynamics (P-V)

0.5.1 V.1 Constraint lattice: objects, orders, and feasibility surfaces (P-V.C1)

F1. Purpose

Purpose. This node defines the constraint lattice view of PSP: constraints as structured objects with an order, enabling monotone reasoning about feasibility, tightening, relaxation, and staged admissibility.

F2. Dependencies

Dependencies. Requires §0.3.5 (constraints), §0.2.3 (profiles), and §0.2.5 (potential interface).

F3. Definitions (Defined)

(Defined) Constraint object. A constraint object is a pair $K := (K^{\text{pred}}, K^{\text{meta}})$ where $K^{\text{pred}} : \Sigma \to \{0,1\}$ is the predicate and K^{meta} is metadata (id, version, scope, severity, proofs/evidence requirements).

(Defined) Constraint set. A constraint set is a finite set $\mathcal{K} = \{K_1, \dots, K_m\}$ identified in RD by $(\mathcal{K}_{\text{id}}, \mathcal{K}_{\text{ver}})$.

(Defined) Constraint order. Define a preorder $\preceq$ on constraints by logical implication:

$$K_a \preceq K_b \iff \forall \Sigma \in \Sigma : \; K_b^{\text{pred}}(\Sigma) = 1 \Rightarrow K_a^{\text{pred}}(\Sigma) = 1.$$

Intuitively, K_b is at least as strong as K_a.

(Defined) Lattice closure (partial). If for some constraints K_a, K_b there exists a constraint $K_{a\wedge b}$ with $K_{a\wedge b}^{\text{pred}}(\Sigma) = K_a^{\text{pred}}(\Sigma) \wedge K_b^{\text{pred}}(\Sigma)$, then $K_{a\wedge b}$ is the meet. Analogously, if $K_{a\vee b}$ exists with predicate OR, it is a join. Existence is not assumed globally.

(Defined) Feasibility surface. Given $\mathcal{K}$, the feasibility surface is the set

$$\mathcal{F}_{\mathcal{K}} := \{\Sigma \in \Sigma : \text{Feas}_{\mathcal{K}}(\Sigma) = 1\}.$$

F4. Derived statements

(Derived) Stronger constraints shrink feasibility surfaces. If $\mathcal{K}'$ is obtained from $\mathcal{K}$ by replacing some constraints with stronger ones under $\preceq$ (or by adding constraints), then $\mathcal{F}_{\mathcal{K}'} \subseteq \mathcal{F}_{\mathcal{K}}$.

Proof. Adding constraints or strengthening constraints can only remove feasible states: any state feasible for $\mathcal{K}'$ satisfies all constraints in $\mathcal{K}'$, hence also satisfies all weaker constraints in $\mathcal{K}$. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Concrete constraint vocabularies are domain- or model-specific. This node defines only the abstract order and feasibility-surface reasoning.

F7. Examples (non-definitional)

Example (informative). A gate may implement a staged constraint policy by moving upward in $\preceq$ (tightening) as evidence strengthens.

F8. Limitations

Limitations. This node does not require that all meets/joins exist. It defines implication order and uses meet/join only when explicitly declared.

F9. Local DoD

Local DoD. Complete iff: (i) constraint objects and order $\preceq$ are defined, (ii) feasibility surface $\mathcal{F}_{\mathcal{K}}$ is defined, (iii) monotonic shrink property is derived and proved.

0.5.2 V.2 Lattice dynamics: monotone tightening, relaxation, and hysteresis (P-V.C2)

F1. Purpose

Purpose. This node defines admissible dynamics over constraint sets: tightening, relaxation, hysteresis windows, and staged feasibility. It provides a formal basis for "post-symbolic stabilizers" without model-specific operator details.

F2. Dependencies

Dependencies. Requires §0.5.1 and §0.3.7 (staged gates).

F3. Definitions (Defined)

(Defined) Constraint policy. A constraint policy is a deterministic function

$$\Pi_{\mathcal{K}} : \Sigma \times RD \to \mathcal{K}$$

that selects (or updates) the active constraint set based on state and RD.

(Defined) Tightening step. A tightening step transforms $\mathcal{K} \mapsto \mathcal{K}^+$ such that

$$\mathcal{F}_{\mathcal{K}^+} \subseteq \mathcal{F}_{\mathcal{K}}.$$

(Defined) Relaxation step. A relaxation step transforms $\mathcal{K} \mapsto \mathcal{K}^-$ such that

$$\mathcal{F}_{\mathcal{K}} \subseteq \mathcal{F}_{\mathcal{K}^-}.$$

(Defined) Hysteresis band. A hysteresis band is a pair of thresholds $(\tau_{\text{on}}, \tau_{\text{off}})$ with $\tau_{\text{on}} > \tau_{\text{off}}$ applied to a scalar witness $w(\Sigma)$ (evidence, margin, or potential decrease) to avoid oscillatory switching.

(Defined) Hysteretic switching rule. A policy MAY use:

$$\text{activate tightening if } w(\Sigma) \geq \tau_{\text{on}}, \quad \text{deactivate tightening if } w(\Sigma) \leq \tau_{\text{off}}.$$

All thresholds MUST be in RD and evidence MUST be recorded in H.

F4. Derived statements

(Derived) Hysteresis prevents rapid toggling under bounded noise. If $w(\Sigma)$ varies within a bounded interval of width $< (\tau_{\text{on}} - \tau_{\text{off}})$ over a persistence window, then the hysteretic switching rule cannot toggle on/off repeatedly within that window.

Proof. To toggle from on to off and back on, w must cross both thresholds in opposite directions. If the variation is smaller than the gap, such crossings cannot both occur; hence repeated toggling is prevented. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. The witness w may be MCI, feasibility margin, potential decrease, or profile coordinate; this node is agnostic.

F7. Examples (non-definitional)

Example (informative). A system may tighten constraints when dual-fabric agreement is high and relax when agreement collapses, using hysteresis to avoid thrashing.

F8. Limitations

Limitations. This node does not guarantee that tightening leads to better artifacts; it defines admissible switching logic and required evidence.

F9. Local DoD

Local DoD. Complete iff: (i) tightening/relaxation are defined via feasibility-surface inclusion, (ii) hysteresis band and switching rule are defined, (iii) anti-toggling property is derived and proved.

0.5.3 V.3 Operator lattices and closure regimes (P-V.C3)

F1. Purpose

Purpose. This node introduces the concept of an operator lattice: families of operators organized by roles and admissibility, and defines closure regimes that characterize when operator compositions remain inside an admissible class.

F2. Dependencies

Dependencies. Requires Part III (operators, solve/morph/gate) and Part II (canonicalization, RD, determinism).

F3. Definitions (Defined)

(Defined) Operator family. An operator family is a finite set $\mathcal{O} = \{\Omega^{(1)}, \ldots, \Omega^{(r)}\} \subseteq \otimes$ with shared role and shared admissibility schema.

(Defined) Closure regime. A closure regime for $\mathcal{O}$ is a predicate

$$\mathrm{Closed}_{\mathcal{O}}(\Sigma, RD) = 1$$

meaning: for all admissible compositions of operators from $\mathcal{O}$ under the declared evaluation policy, the resulting state remains well-formed and within a declared invariant subset $\Sigma_{\mathcal{O}} \subseteq \Sigma$.

(Defined) Invariant subset. A subset $\Sigma_{\mathcal{O}} \subseteq \Sigma$ is invariant under $\mathcal{O}$ iff for all $\Omega \in \mathcal{O}$:

$$\Sigma \in \Sigma_{\mathcal{O}} \wedge \mathrm{Adm}_{\Omega}(\Sigma, RD) = 1 \Rightarrow \Omega(\Sigma) \in \Sigma_{\mathcal{O}}.$$

(Defined) Operator lattice (informal-to-formal bridge). An operator lattice is a finite partially ordered set $(\mathcal{O}_1, \ldots, \mathcal{O}_L, \preceq_{\Omega})$ of operator families, ordered by admissibility strength (e.g. stricter constraints or stricter budgets yield higher families). The order MUST be declared by explicit rules.

F4. Derived statements

(Derived) Invariant subsets yield compositional closure. If $\Sigma_{\mathcal{O}}$ is invariant under every operator in $\mathcal{O}$, then any finite composition of admissible operators in $\mathcal{O}$ maps $\Sigma_{\mathcal{O}}$ to itself.

Proof. By induction on composition length using invariance under each step operator. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Concrete operator lattices (e.g. DK/SW/PI/WT families) are introduced as model-specific instantiations later; this node provides the abstract closure scaffold.

F7. Examples (non-definitional)

Example (informative). A "strict" family may require feasibility preservation and decreasing μ, while a "loose" family may allow exploratory morphs under a larger budget.

F8. Limitations

Limitations. This node does not assert existence of meaningful invariant subsets; it defines the concept and how closure claims must be stated.

F9. Local DoD

Local DoD. Complete iff: (i) closure regimes and invariant subsets are defined, (ii) compositional closure is derived and proved, (iii) operator-lattice ordering is constrained to explicit rules.

0.5.4 V.4 Contraction conditions and fixed-point closure (P-V.C4)

F1. Purpose

Purpose. This node defines sufficient conditions under which an iterative PSP subprogram admits fixed-point closure: a contraction regime over a deterministic distance witness. It enables rigorous termination claims when contraction is established.

F2. Dependencies

Dependencies. Requires §0.2.10 (distance), §0.1.9 (convergence), and §0.3.5 (solve contract).

F3. Definitions (Defined)

(Defined) Contraction operator (relative). An operator Ω is a contraction w.r.t. d on a subset $\mathcal{S} \subseteq \Sigma$ if there exists $0 \leq \lambda < 1$ (fixed in RD) such that for all $\Sigma, \Sigma' \in \mathcal{S}$:

$$d(\Omega(\Sigma), \Omega(\Sigma')) \leq \lambda\, d(\Sigma, \Sigma').$$

(Defined) Contractive subprogram. A subprogram $\mathcal{F} = (P, \mathcal{E})$ is contractive on $\mathcal{S}$ if its induced one-step map $T_{\mathcal{F}} : \Sigma \to \Sigma$ (the map corresponding to one full evaluation of P) is a contraction on $\mathcal{S}$.

(Defined) Fixed-point closure claim. A fixed-point closure claim for a contractive subprogram is the statement:

$$\exists!\ \Sigma^\star \in \mathcal{S} \text{ s.t. } T_{\mathcal{F}}(\Sigma^\star) \equiv \Sigma^\star,$$

together with a convergence statement for iterating $T_{\mathcal{F}}$ starting from any $\Sigma_0 \in \mathcal{S}$.

F4. Derived statements

(Derived) Contraction implies convergence in the witness distance. Assume $T_{\mathcal{F}}$ is a contraction on $\mathcal{S}$ with factor $\lambda < 1$, and assume $\mathcal{S}$ is closed under $T_{\mathcal{F}}$. Then for any $\Sigma_0 \in \mathcal{S}$, the iterates $\Sigma_{n+1} := T_{\mathcal{F}}(\Sigma_n)$ form a Cauchy sequence in d.

Proof. For $n > m$, repeated contraction yields $d(\Sigma_n, \Sigma_m) \leq \lambda^m \cdot C$ for some finite C depending on Σ_1, Σ_0. Since $\lambda^m \to 0$, the sequence is Cauchy in d. □

(External) Existence/uniqueness of fixed points. Existence and uniqueness of a fixed point for contractions in complete metric spaces is standard (Banach fixed-point theorem). If a model asserts completeness of $(\mathcal{S}, d)$, it MAY import Banach's theorem as External to justify existence/uniqueness.

F5. Proof or status

The Cauchy property is Derived with explicit proof. Fixed-point existence/uniqueness is labeled External unless proved in-document.

F6. Model notes

Model note. Completeness of $(\mathcal{S}, d)$ and the choice of d are model-specific. This node provides the admissible structure for contraction-based closure claims.

F7. Examples (non-definitional)

Example (informative). A lattice-dynamics model may claim that a "solve/coagula" operator is contractive in a profile-induced distance on a bounded region, thereby enabling fixed-point closure for that region.

F8. Limitations

Limitations. This node does not guarantee contraction. It defines how contraction claims must be stated and what they imply.

F9. Local DoD

Local DoD. Complete iff: (i) contraction and contractive subprogram are defined, (ii) Cauchy convergence under contraction is derived and proved, (iii) Banach fixed-point use is correctly labeled External unless proved.

0.5.5 V.5 Fundamental operator families: DK/SW/PI/WT as a lattice instance (P-V.C5)

F1. Purpose

Purpose. This node introduces a canonical *operator-family instance* used in PSP literature: four fundamental operator families (DK, SW, PI, WT). The node is careful about status: it defines the families abstractly and fixes admissible axioms, while leaving concrete realizations to model classes.

F2. Dependencies

Dependencies. Requires §0.5.3 (operator families/closure regimes) and Part III (operator registry).

F3. Definitions (Defined)

(Defined) Fundamental family instance. Define four operator families as subsets of $\otimes$:

$$\mathcal{O}_{DK},\ \mathcal{O}_{SW},\ \mathcal{O}_{PI},\ \mathcal{O}_{WT}\ \subseteq \otimes,$$

each with its own admissibility schema and evidence obligations.

(Defined) Family roles (informative-to-normative bridge). The families MAY be used with the following role interpretations:

- DK: condensation / stabilization transforms (typically norm-preserving or contractive maps),
- SW: switching / gating transforms (often discrete regime selection),
- PI: accumulation / path integration transforms (often aggregations along graph paths),
- WT: weaving / stitching transforms (often topology-altering joins under constraints).

These role labels are informative; admissibility is governed by the contracts below.

(Defined) Admissible family axioms. A model class may assert any subset of the following axioms, but each asserted axiom MUST be proved (Derived within the model) or labeled External:

A1 (Well-formedness preservation). Every operator in each family preserves WF on admissible inputs.

A2 (Determinism). Every operator in each family is deterministic under RD.

A3 (Budget discipline). Any expansion effect is budgeted (cf. §0.3.6).

A4 (Descent compatibility). DK or PI operators may be declared descent operators w.r.t. μ (cf. §0.2.5).

A5 (Closure regime). Each family may declare an invariant subset $\Sigma_{\mathcal{O}}$ and closure regime (cf. §0.5.3).

(Defined) Family registry obligations. If a model class uses the DK/SW/PI/WT instance, it MUST: (i) register each concrete operator under one family, (ii) declare which axioms A1–A5 it asserts, (iii) provide proof-status labels for those assertions.

F4. Derived statements

(Derived) Family axioms induce compositional safety envelopes. If a family asserts A1–A3 and declares a closure regime $\Sigma_{\mathcal{O}}$, then any admissible composition within the family is safe w.r.t. well-formedness, determinism, and budget constraints on $\Sigma_{\mathcal{O}}$.

Proof. A1 gives WF preservation, A2 gives determinism, A3 gives bounded growth, and the closure regime provides invariance under composition (by §0.5.3). Hence admissible compositions remain inside the declared safety envelope. □

F5. Proof or status

All claims are Defined or Derived with explicit proofs in this node. Any concrete axiom instantiation is deferred to model classes.

F6. Model notes

Model note. This node defines the family instance as an admissible PSP pattern, not as a universal truth. Concrete DK/SW/PI/WT definitions belong to Part VII.

F7. Examples (non-definitional)

Example (informative). A model may define DK as a rotation/condensation operator on a tensor representation, PI as an aggregation along HDAG paths, SW as a regime switch based on hysteresis, and WT as a constrained graph-merge operator.

F8. Limitations

Limitations. This node does not define numerical formulas for DK/SW/PI/WT. It defines admissible scaffolding and registry obligations.

F9. Local DoD

Local DoD. Complete iff: (i) the four families are defined as operator-family instances, (ii) admissible axioms A1–A5 are fixed with status discipline, (iii) family registry obligations are fixed, (iv) compositional safety envelope property is derived and proved.

0.5.6 V.6 Solve/Coagula composition as a stabilization template (P-V.C6)

F1. Purpose

Purpose. This node defines a canonical stabilization template: alternating *Solve* (constraint improvement) and *Coagula* (state consolidation). The purpose is to formalize a common PSP control pattern without committing to any single model.

F2. Dependencies

Dependencies. Requires §0.3.5 (solve operators), §0.2.9 (merge), and §0.2.5 (potential interface).

F3. Definitions (Defined)

(Defined) Solve step. A solve step is an application of an operator Ω_{solve} satisfying the monotone improvement contract (S1 or S2) from §0.3.5.

(Defined) Coagula step. A coagula step is an operator $\Omega_{\text{coag}} : \Sigma \to \Sigma$ that consolidates state structure by reducing redundancy or merging equivalent substructures, subject to:

C1. Ω_{coag} is deterministic under RD,
C2. Ω_{coag} preserves WF,
C3. Ω_{coag} does not increase the violation score $v_{\mathcal{K}}$ (if a violation witness is active),
C4. if μ is active, Ω_{coag} is a descent operator or is explicitly exempted by a gate rule.

(Defined) Solve/Coagula cycle. A Solve/Coagula cycle is the two-step map

$$T_{SC}(\Sigma) := \Omega_{\text{coag}}\big(\Omega_{\text{solve}}(\Sigma)\big),$$

executed iteratively with termination/convergence criteria from §0.1.9.

(Defined) Cycle evidence. Each cycle MUST record: (i) violation/potential witness values before and after each step, (ii) any consolidation decisions (e.g. merge keys) in H.

F4. Derived statements

(Derived) If both steps are descent operators, the cycle is descent. If Ω_{solve} and Ω_{coag} are descent operators w.r.t. μ, then T_{SC} is a descent map: $\mu(T_{SC}(\Sigma)) \le \mu(\Sigma)$ on admissible inputs.

Proof. Apply descent inequality for Ω_{solve} then for Ω_{coag}: $\mu(\Omega_{\text{solve}}(\Sigma)) \le \mu(\Sigma)$ and $\mu(\Omega_{\text{coag}}(\Omega_{\text{solve}}(\Sigma))) \le \mu(\Omega_{\text{solve}}(\Sigma))$. Chaining yields $\mu(T_{SC}(\Sigma)) \le \mu(\Sigma)$. □

(Derived) Deterministic cycles yield replay-stable stabilization traces. If both steps are deterministic and evidence is recorded deterministically, then the entire cycle trace is replay-stable.

Proof. Immediate from the replay contract: deterministic steps produce identical state transitions and evidence under replay. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs. No external theorems.

F6. Model notes

Model note. Concrete definitions of "redundancy" and consolidation strategies are model-specific. This node fixes the admissible contracts.

F7. Examples (non-definitional)

Example (informative). A coagula operator may merge isomorphic subgraphs under canonical labeling, preserving edge typing and recording the merge map in $H.\mathcal{D}$.

F8. Limitations

Limitations. This node does not assert convergence. Convergence requires either contraction conditions (V.4) or additional model-specific properties.

F9. Local DoD

Local DoD. Complete iff: (i) coagula is defined with determinism and non-regression requirements, (ii) Solve/Coagula cycle is defined, (iii) descent and replay properties are derived and proved, (iv) evidence obligations are fixed.

0.5.7 V.7 Stitching cycles, weaving constraints, and topology-safe joins (P-V.C7)

F1. Purpose

Purpose. This node defines stitching cycles as topology-safe join mechanisms: repeated application of weaving/merge operators under explicit constraints, with evidence for each join decision.

F2. Dependencies

Dependencies. Requires §0.3.6 (morph operators), §0.2.9 (merge), and §0.5.1 (constraint lattice).

F3. Definitions (Defined)

(Defined) Stitch operator. A stitch operator Ω_{stitch} is a morph operator that introduces join edges or join structures in G, subject to: (i) typing preservation (WF_G), (ii) join admissibility constraints $\mathcal{K}_{\text{join}} \subseteq \mathcal{K}$, (iii) deterministic join selection and tie-break.

(Defined) Weaving constraint family. A weaving constraint family $\mathcal{K}_{\text{join}}$ is a finite subset of constraints dedicated to topology safety (e.g. forbidding cycles of certain types, forbidding forbidden edge-type adjacencies, bounding degree growth).

(Defined) Stitching cycle. A stitching cycle is an iterative subprogram that alternates:

$$\Sigma \xrightarrow{\Omega_{\text{stitch}}} \Sigma' \xrightarrow{\Omega_{\text{coag}}} \Sigma'',$$

where Ω_{coag} consolidates and normalizes the new topology. Termination is by budget, gate, or convergence.

(Defined) Join evidence. Each stitch action MUST record: (i) chosen join endpoints/keys in canonical form, (ii) which constraints in $\mathcal{K}_{\text{join}}$ were checked, (iii) resulting local topology summary (deterministic snapshot or digest).

F4. Derived statements

(Derived) Constraint-checked stitching preserves topology safety invariants. If Ω_{stitch} checks and enforces $\mathcal{K}_{\text{join}}$ deterministically before applying a join, then any invariant expressed by $\mathcal{K}_{\text{join}}$ is preserved by stitching.

Proof. By construction, joins are applied only when all join constraints evaluate to 1. Therefore no stitch step can violate any constraint in $\mathcal{K}_{\text{join}}$. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Topology summaries and particular invariants are model-specific; the enforcement mechanism is universal.

F7. Examples (non-definitional)

Example (informative). A weaving constraint may forbid introducing an edge of type `depends_on` that would create a dependency cycle among operator nodes.

F8. Limitations

Limitations. This node does not guarantee that stitching improves artifact quality; it guarantees only constraint-preserving join safety.

F9. Local DoD

Local DoD. Complete iff: (i) stitch operator and weaving constraints are defined, (ii) stitching cycle is defined, (iii) constraint-preservation property is derived and proved, (iv) join evidence obligations are fixed.

0.5.8 V.8 Dual-fabric consensus as a lattice stabilizer (P-V.C8)

F1. Purpose

Purpose. This node links dual-fabric consistency (Part I) to constraint-lattice dynamics: constraint tightening and acceptance decisions may be gated by dual-fabric agreement, thereby stabilizing post-symbolic evolution against one-sided drift.

F2. Dependencies

Dependencies. Requires §0.1.8 (dual fabrics), §0.3.7 (gates), and §0.5.2 (hysteresis policies).

F3. Definitions (Defined)

(Defined) Consensus witness. A consensus witness is a scalar

$$w_{\text{cons}}(\Sigma_0) := \text{MCI}(\Sigma_0; \mathcal{F}_1, \mathcal{F}_2)$$

computed under *RD* for two declared fabrics.

(Defined) Consensus-driven tightening rule. A constraint policy MAY tighten constraints if:

$$w_{\text{cons}}(\Sigma_0) \geq \tau_{\text{on}} \ \wedge \ \text{Accept}_{\tau}(\Sigma_0) = 1,$$

and MAY relax if $w_{\text{cons}}(\Sigma_0) \leq \tau_{\text{off}}$, where $(\tau_{\text{on}}, \tau_{\text{off}})$ are in RD.

(Defined) Consensus gate. A consensus gate is a staged gate that includes (i) a task acceptance predicate for each fabric and (ii) an MCI threshold check.

F4. Derived statements

(Derived) Consensus-driven tightening inherits hysteresis stability. If the tightening/relaxation policy uses hysteresis thresholds with $\tau_{\text{on}} > \tau_{\text{off}}$, then constraint policy switching inherits the anti-toggling property from §0.5.2.

Proof. Immediate: the witness is w_{cons} and the switching rule is hysteretic. Therefore repeated toggling within bounded variation is prevented. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Concrete fabric choices and the measurement map driving MCI are model-specific; the consensus policy mechanism is universal.

F7. Examples (non-definitional)

Example (informative). A system may tighten weave constraints only when two alternative operator decompositions produce consistent measurements and both pass feasibility gates.

F8. Limitations

Limitations. Consensus does not prove correctness; it provides a stability criterion that reduces one-sided acceptance errors.

F9. Local DoD

Local DoD. Complete iff: (i) consensus witness is defined via MCI, (ii) consensus-driven tightening rule is defined, (iii) hysteresis inheritance is derived and proved, (iv) consensus gate is defined.

0.6 Part VI — Gating and Proof Objects (P-VI)

0.6.1 VI.1 Proof objects: evidence bundles and verification semantics (P-VI.C1)

F1. Purpose

Purpose. This node defines PSP proof objects as deterministic evidence bundles that justify gate decisions, constraint claims, and artifact acceptance. Proof objects are not "human proofs" by default; they are machine-verifiable records tied to a run, an operator segment, and a declared verification procedure.

F2. Dependencies

Dependencies. Requires §0.2.6 (history/evidence), §0.3.7 (gates), and §0.1.4 (replay).

F3. Definitions (Defined)

(Defined) Proof object. A proof object is a tuple

$$\mathcal{P} := (\mathcal{P}_{id},\ \mathcal{P}_{ver},\ \mathcal{B},\ \mathcal{V},\ RD),$$

where:

- $\mathcal{B}$ is an evidence bundle (finite set of evidence items from $H.\mathcal{E}$ and trace anchors),
- $\mathcal{V}$ is a verification procedure identifier and parameters,
- $(\mathcal{P}_{id}, \mathcal{P}_{ver})$ identify the proof-object schema version,
- RD binds determinism, numeric semantics, and thresholds.

(Defined) Evidence bundle. An evidence bundle $\mathcal{B}$ is a finite set of records of the form

$$(\text{ref},\ \text{kind},\ \text{payload},\ \text{dig}),$$

where ref ties the item to a trace step or artifact id, kind names the evidence type, payload is the deterministic data, and dig is a digest over canonicalized payload.

(Defined) Verification semantics. A proof object is *valid* iff

$$\text{Verify}(\mathcal{P}) = 1,$$

where Verify is the deterministic verification procedure specified by $\mathcal{V}$. Verification MUST be replayable: identical $\mathcal{P}$ under identical RD yields identical verification outcomes.

(Defined) Proof-object admissibility. A proof object is admissible iff:

PO1. all evidence items are deterministic and RD-bound,
PO2. all digests are computed by algorithms declared in RD,
PO3. $\mathcal{V}$ is declared and versioned,
PO4. verification is deterministic under RD.

F4. Derived statements

(Derived) Proof objects are replay-stable under deterministic verification. If $\mathcal{P}$ is admissible and Verify is deterministic, then Verify($\mathcal{P}$) is replay-stable.

Proof. Replay reproduces identical canonical evidence and identical deterministic verification computation under the same RD. Hence the outcome is invariant under replay. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Verification procedures may include constraint checks, recomputation of profile values, replay of subgraphs, or external tool verification. Any external dependence MUST be modeled as a boundary operator and recorded as evidence, otherwise the proof object is non-admissible.

F7. Examples (non-definitional)

Example (informative). A gate proof object may bundle: profile snapshot z, MCI computation inputs, thresholds, the gate predicate id, and the boolean outcome, plus a verification rule "recompute and compare".

F8. Limitations

Limitations. This node does not claim that proof objects establish universal mathematical truth; it establishes deterministic, auditable justification for PSP decisions.

F9. Local DoD

Local DoD. Complete iff: (i) proof objects and evidence bundles are defined, (ii) verification semantics and admissibility are defined, (iii) replay-stability is derived and proved.

0.6.2 VI.2 Gate evidence schema: mandatory fields and minimal completeness (P-VI.C2)

F1. Purpose

Purpose. This node fixes the minimal gate evidence schema required for any acceptance, rejection, or termination decision. It makes gate decisions machine-checkable and prevents "opaque boolean gates".

F2. Dependencies

Dependencies. Requires §0.3.7 (gate operator) and §0.2.6 (history).

F3. Definitions (Defined)

(Defined) Gate evidence item. A gate evidence item is a record

$$E_q := (q_{id}, q_{ver},\ \theta,\ \mathsf{inputs},\ \mathsf{outputs},\ \mathsf{outcome},\ t,\ \mathrm{dig}),$$

where:

- (q_{id}, q_{ver}) identify the predicate version,
- θ contains all thresholds/parameters used (from $RD.params$),
- inputs contains canonical references to required measurements (profile id, z snapshot digests, etc.),
- $\mathsf{outputs}$ contains computed witness values (e.g. margins, MCI, $\Delta\mu$),
- $\mathsf{outcome} \in \{0, 1\}$ is the gate result,
- t is the trace index (or window indices) to which the decision applies,
- dig is a digest of the canonicalized evidence item.

(Defined) Evidence completeness. A gate evidence item is complete iff inputs and outputs are sufficient to recompute the predicate deterministically under RD.

(Defined) Gate proof object. A gate proof object is a proof object $\mathcal{P}$ whose evidence bundle includes all gate evidence items used in a decision plus the verification rule

$$\text{Verify}_{gate} : \text{“recompute predicate under } RD \text{ and compare to outcome}''.$$

F4. Derived statements

(Derived) Evidence completeness implies re-verifiability. If gate evidence is complete and deterministic, then Verify_{gate} can re-evaluate the predicate and verify the outcome.

Proof. Completeness guarantees that all predicate inputs are available in canonical form and that all parameters are RD-bound. Therefore recomputation is deterministic and yields a unique truth value, which can be compared to the recorded outcome. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Models may add additional evidence fields (topology summaries, constraint lists, branch selections), but MUST include the mandatory fields above.

F7. Examples (non-definitional)

Example (informative). A staged gate stores one evidence item per stage q_i and records the first failing stage index for deterministic short-circuit evaluation.

F8. Limitations

Limitations. This node does not define any concrete predicate formulas; it defines the minimal evidence schema to support verification.

F9. Local DoD

Local DoD. Complete iff: (i) gate evidence item schema is defined, (ii) completeness is defined, (iii) re-verifiability is derived and proved.

0.6.3 VI.3 Proof-of-Resonance style predicates as admissible model predicates (P-VI.C3)

F1. Purpose

Purpose. This node defines a general predicate pattern often used in PSP model families: a weighted-threshold predicate over measurement coordinates. It is presented as an admissible schema, not as a universal foundational law.

F2. Dependencies

Dependencies. Requires Part IV (profiles) and Part VI.2 (gate evidence).

F3. Definitions (Defined)

(Defined) Weighted-threshold predicate schema. Let $m : \Sigma \to \mathbb{D}^n$ be the active profile and let $z = m(\Sigma)$. A weighted-threshold predicate has the form

$$q_{W,\theta}(\Sigma) = 1 \iff \langle W, z \rangle \geq \theta,$$

where $W \in \mathbb{D}^n$ and $\theta \in \mathbb{D}$ are fixed in $RD.params$.

(Defined) Deterministic evaluation. Evaluation of $\langle W, z \rangle$ MUST be deterministic under the numeric discipline (Part II.11) and MUST record all required intermediate values if needed for verification.

(Defined) Evidence obligation. Gate evidence MUST include: (W, θ) identifiers/values, the input z digest or snapshot, the computed inner product value, and the boolean outcome.

F4. Derived statements

(Derived) Weighted-threshold predicates are re-verifiable from evidence. If evidence includes (W, θ) and the z snapshot (or digest plus reconstructible snapshot), then Verify_{gate} can recompute the predicate.

Proof. All inputs are available and deterministic under RD. Therefore recomputation is unique and comparable to recorded outcome. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. The choice of profile coordinates and weight vectors is model-specific. This node provides an admissible predicate pattern.

F7. Examples (non-definitional)

Example (informative). A model may choose $z = (\psi, \rho, \omega, \chi, \eta)$ and define a predicate using weights $(\alpha, \beta, \gamma, 0, 0)$ and threshold θ_c.

F8. Limitations

Limitations. This schema does not capture non-linear gates (hysteresis, multi-window checks). Those are introduced as staged or composite predicates.

F9. Local DoD

Local DoD. Complete iff: (i) weighted-threshold schema is defined, (ii) determinism and evidence obligations are fixed, (iii) re-verifiability is derived and proved.

0.6.4 VI.4 Dual-fabric gate proof packs and consensus certificates (P-VI.C4)

F1. Purpose

Purpose. This node defines proof packs for dual-fabric acceptance: certificates that justify that two declared fabrics agree on a task predicate and meet robustness thresholds (MCI, tolerances), with replayable evidence.

F2. Dependencies

Dependencies. Requires §0.1.8 (dual fabrics), §0.6.1–§0.6.2 (proof objects), and Part IV (profile metrics).

F3. Definitions (Defined)

(Defined) Consensus certificate. A consensus certificate is a proof object

$$\mathcal{C}_\tau := (\mathcal{C}_{id}, \mathcal{C}_{ver},\ \mathcal{B}_\tau,\ \mathcal{V}_\tau,\ RD)$$

whose evidence bundle $\mathcal{B}_\tau$ includes:

CC1. fabric identifiers for $\mathcal{F}_1, \mathcal{F}_2$ (program ids + evaluation policies),
CC2. final-state digests for both runs (or windowed digests),
CC3. task predicate outcomes for both fabrics,
CC4. the measurement snapshots used for MCI and the computed MCI value,
CC5. thresholds (ϵ_m, η) used for robustness.

(Defined) Verification rule. $\mathcal{V}_\tau$ MUST verify: (i) both fabrics are declared and RD-compatible, (ii) both runs are replayable under the same RD, (iii) recomputation of predicate outcomes matches recorded outcomes, (iv) recomputation of MCI matches recorded MCI within tolerance.

F4. Derived statements

(Derived) Consensus certificates are replay-stable. If both fabric runs are replayable and all evidence is deterministic, then the consensus certificate verification result is replay-stable.

Proof. Replay reproduces the same fabric trajectories and the same measurements; recomputation therefore yields identical predicate outcomes and MCI values. Hence verification outcome is invariant under replay. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. Concrete dualization strategies (how $\mathcal{F}_1$ and $\mathcal{F}_2$ differ) are model-specific. This node defines only the admissible certificate structure and verification obligations.

F7. Examples (non-definitional)

Example (informative). A certificate may bind a "linear chain" fabric and an "operator DAG" fabric, both passing feasibility and weighted-threshold gates, with high MCI.

F8. Limitations

Limitations. Consensus certificates establish robustness, not truth. They guarantee that acceptance is not an artifact of a single decomposition.

F9. Local DoD

Local DoD. Complete iff: (i) consensus certificate evidence requirements are defined, (ii) verification obligations are defined, (iii) replay-stability is derived and proved.

0.6.5 VI.5 External dependencies and boundary-verification protocol (P-VI.C5)

F1. Purpose

Purpose. This node defines how PSP handles external theorems, external tools, and external verification procedures without breaking determinism and auditability. It fixes a boundary-verification protocol: external dependencies are admissible only if their inputs/outputs are captured as replayable evidence.

F2. Dependencies

Dependencies. Requires §0.3.3 (boundary operators), §0.1.3 (External status), and §0.6.1 (proof objects).

F3. Definitions (Defined)

(Defined) External dependency. An external dependency is any verification-relevant component not proved or implemented within the PSP system, including: standard mathematical theorems, third-party tools, compilers, SMT solvers, oracles, and external services.

(Defined) Boundary-verification protocol. A proof object that relies on an external dependency is admissible only if:

BV1. the dependency is explicitly declared (name, version, identity),
BV2. all external inputs are recorded as canonical evidence,
BV3. all external outputs are recorded as canonical evidence,
BV4. verification replays by reusing recorded outputs (or by re-running in a captured environment whose identity is fixed in $RD.env$),
BV5. the proof-status of the claim is labeled External (unless the dependency is eliminated by an in-document proof).

(Defined) Captured environment identity. If a proof pack claims re-execution of an external tool is replayable, the environment identity MUST be fixed: tool version, platform constraints, and any nondeterminism sources.

F4. Derived statements

(Derived) Boundary-verification preserves replay semantics. If BV1–BV4 hold, then external verification outcomes are replay-stable in the sense of PSP auditability.

Proof. Either (i) replay reuses recorded outputs, yielding identical outcomes, or (ii) re-execution occurs under a captured environment identity fixed in RD, with all inputs fixed and recorded. In both cases, the verification outcome is determined by canonical evidence and RD, hence replay-stable. □

F5. Proof or status

All non-definitional claims are Derived with explicit proof. External dependencies remain labeled External by rule BV5.

F6. Model notes

Model note. Models may tighten BV requirements (e.g. prohibit re-execution and require output reuse only). They may not weaken BV1–BV5.

F7. Examples (non-definitional)

Example (informative). If a proof pack uses an SMT solver, the solver version, the exact query, and the solver output must be recorded; replay verifies by comparing recorded outputs.

F8. Limitations

Limitations. This protocol does not make external dependencies "true"; it makes them explicit, bounded, and audit-compatible.

F9. Local DoD

Local DoD. Complete iff: (i) external dependencies are defined, (ii) BV1–BV5 protocol is fixed, (iii) replay preservation is derived and proved.

0.6.6 VI.6 Proof obligation registry and closure tracking (P-VI.C6)

F1. Purpose

Purpose. This node defines a proof-obligation registry: a machine-readable ledger of claims that require proof, evidence, or explicit External/Open classification. It enables deterministic closure tracking of PSP specifications and model instances.

F2. Dependencies

Dependencies. Requires §0.1.3 (status taxonomy) and §0.6.1 (proof objects).

F3. Definitions (Defined)

(Defined) Proof obligation. A proof obligation is a tuple

$$\mathcal{O} := (id,\ node,\ claim,\ status,\ prereq,\ verifier,\ state),$$

where:

- id is a unique obligation identifier,
- $node$ is the section label in which the claim appears,
- $claim$ is a normalized statement (machine-checkable form or structured text),
- $status \in \mathcal{S}$ is its proof-status label,
- $prereq$ lists required premises (by obligation IDs or section references),
- $verifier$ identifies the verification procedure (proof object schema or external protocol),
- $state \in \{\mathsf{Open}, \mathsf{Satisfied}, \mathsf{External}, \mathsf{Quarantined}\}$ is closure state.

(Defined) Obligation registry. The obligation registry $\mathcal{R}_{\mathcal{O}}$ is a finite set of obligations maintained as part of the document's appendix apparatus.

(Defined) Closure rule. An obligation is Satisfied iff: (i) it is labeled Derived and a proof is present and admissible, or (ii) it is backed by an admissible proof object whose verification passes deterministically under RD. An obligation labeled External may be marked External if it obeys the boundary-verification protocol. An obligation labeled Open must be Open or Quarantined and MUST NOT be used as a premise for Derived obligations.

F4. Derived statements

(Derived) Registry closure prevents hidden proof gaps. If every claim that carries downstream load is represented as a proof obligation with an explicit closure state, then no proof gap can remain hidden without appearing as an Open/External obligation.

Proof. A load-bearing claim must appear as a premise in some derivation or gate. If it is registered, its closure state is explicit. If it is not satisfied, it cannot be treated as Derived without violating admissibility. Hence gaps surface as Open/External. □

F5. Proof or status

All non-definitional claims are Derived with explicit proof.

F6. Model notes

Model note. Model instances may maintain separate obligation registries for model-specific claims, but must preserve the same status/closure semantics.

F7. Examples (non-definitional)

Example (informative). A model claiming contraction in a region must register: (i) the contraction factor $\lambda < 1$, (ii) the region invariance, (iii) the distance witness d, and (iv) the verification method (proof or proof object).

F8. Limitations

Limitations. The registry does not guarantee satisfiability; it guarantees explicit accounting.

F9. Local DoD

Local DoD. Complete iff: (i) proof obligations are defined, (ii) registry and closure rules are defined, (iii) anti-hidden-gap property is derived and proved.

0.6.7 VI.7 Minimal theorem discipline: claim normalization and dependency hygiene (P-VI.C7)

F1. Purpose

Purpose. This node defines a minimal theorem discipline for PSP: how to normalize claims, how to reference dependencies, and how to prevent ambiguous "theorem-like" prose from carrying untracked semantic load.

F2. Dependencies

Dependencies. Requires §0.1.3 (status taxonomy) and §0.6.6 (obligation registry).

F3. Definitions (Defined)

(Defined) Normalized claim form. A normalized claim is represented as:

$$\text{Claim} := (\text{pre},\ \text{stmt},\ \text{post},\ \text{tags}),$$

where pre are explicit premises/assumptions, stmt is the statement, post are explicit consequences, and tags include status, model boundary, and verifier id.

(Defined) Dependency hygiene. Every Derived proof MUST list its dependencies explicitly as obligation IDs or section references. Implicit "it follows" without referenced premises is non-admissible for Derived status.

(Defined) Theorem-like label discipline. A statement may be labeled Lemma/Proposition/Theorem only if it is registered in $\mathcal{R}_{\mathcal{O}}$ with: (i) explicit premises, (ii) explicit proof-status, (iii) explicit closure state.

F4. Derived statements

(Derived) Dependency hygiene ensures proof replayability at the spec level. If every derived claim lists dependencies explicitly, then the validity of a proof can be audited by checking closure states of its dependencies.

Proof. A derived proof is valid only if all its premises are admissible and satisfied. If dependencies are explicit, an auditor can map each dependency to an obligation and check that it is Satisfied (or appropriately External). Thus proof validity becomes mechanically checkable. □

F5. Proof or status

All non-definitional claims are Derived with explicit proof.

F6. Model notes

Model note. Model-specific theorems must carry model boundary tags and must not be imported into foundations without boundary propagation.

F7. Examples (non-definitional)

Example (informative). A statement "the system converges" is non-admissible unless it specifies (i) the distance witness, (ii) the region, (iii) the contraction factor or other sufficient condition, and (iv) the closure state of the supporting obligations.

F8. Limitations

Limitations. This discipline does not force deep mathematical content; it forces explicit bookkeeping and removes ambiguity.

F9. Local DoD

Local DoD. Complete iff: (i) normalized claim form is defined, (ii) dependency hygiene is fixed, (iii) theorem-like labeling is tied to the obligation registry, (iv) audit property is derived and proved.

0.6.8 VI.8 Residual risk register: open items quarantine and bounded use (P-VI.C8)

F1. Purpose

Purpose. This node defines a residual risk register: a finite list of open or quarantined items and the rules governing their bounded use. It prevents open conjectures and unproven bridges from influencing acceptance, proofs, or conformance claims.

F2. Dependencies

Dependencies. Requires §0.1.3 (Open status) and §0.6.6 (obligation registry).

F3. Definitions (Defined)

(Defined) Residual risk item. A residual risk item is a tuple

$$R := (rid,\ node,\ description,\ impact,\ mitigation,\ state),$$

where $state \in \{\mathsf{Open}, \mathsf{Quarantined}, \mathsf{Retired}\}$.

(Defined) Residual risk register. The residual risk register $\mathcal{R}_{RR}$ is a finite set of residual risk items maintained in the appendices.

(Defined) Quarantine rule. If an item is Open or Quarantined, then it MUST NOT be used as a premise for any Derived claim and MUST NOT be used as the sole justification for any gate acceptance. It MAY be referenced in informative text and MAY motivate further work.

F4. Derived statements

(Derived) Quarantine preserves conformance soundness. If conformance claims and gates do not depend on Open/Quarantined items, then conformance remains independent of unresolved theory gaps.

Proof. Conformance is defined by normative requirements and satisfied obligations. If Open/Quarantined items are excluded from derived proofs and gates, then unresolved gaps cannot change acceptance or conformance outcomes, preserving soundness with respect to stated requirements. □

F5. Proof or status

All non-definitional claims are Derived with explicit proof.

F6. Model notes

Model note. Model instances may carry their own residual risk registers. Any model-specific open items remain quarantined to that model boundary.

F7. Examples (non-definitional)

Example (informative). An unproven claim "profile translation is invertible" must be quarantined; systems may still use a translation map operationally, but must not treat invertibility as a derived guarantee.

F8. Limitations

Limitations. This register does not eliminate risk; it makes risk explicit and prevents it from contaminating foundations.

F9. Local DoD

Local DoD. Complete iff: (i) residual risk items and register are defined, (ii) quarantine rule is fixed, (iii) conformance-soundness property is derived and proved.

0.7 Part VII — Model Classes and Instantiations (P-VII)

0.7.1 VII.1 Model-class declaration template (P-VII.C1)

F1. Purpose

Purpose. This node fixes the mandatory declaration template for any PSP model class. It ensures that a model instance is admissible (Part I.6), that its boundary is explicit, and that it cannot leak model-specific assumptions into foundations.

F2. Dependencies

Dependencies. Requires §0.1.6 (admissible model classes) and Part II–VI (state, operators, profiles, proof objects).

F3. Definitions (Defined)

(Defined) Model declaration record. A model class declaration MUST provide:

MD1. **Model identifier:** $(\mathfrak{M}_{id}, \mathfrak{M}_{ver})$.
MD2. **State encoding:** concrete domains for (G, z, Π, μ, H, RD) and WF definitions.
MD3. **Equivalence and canonicalization:** $\sim_G$, $\sim_\Sigma$, and explicit Canon$_\bullet$ procedures.
MD4. **Operator families:** $\Omega_{\mathfrak{M}}$ and family memberships (e.g. DK/SW/PI/WT), plus admissibility predicates.
MD5. **Profiles:** declared measurement profiles m with coordinate semantics, calibration fixtures, and norms.
MD6. **Gates:** declared predicates (staged gates allowed) with complete evidence schema.
MD7. **Budgets:** explicit budget policies for morph/expansion and termination.
MD8. **Boundary statement:** explicit list of Model-Specific claims and forbidden generalizations.
MD9. **Proof obligations:** a model-local obligation registry (subset) referencing the global registry.

(Defined) Model admissibility criterion. A model class is admissible iff it satisfies MD1–MD9 and all required invariants SI1–SI9 (Part I.5).

F4. Derived statements

(Derived) Template completeness implies auditability of model assumptions. If MD1–MD9 are present, then every model-specific assumption is explicitly declared and traceable.

Proof. MD8 forces an explicit boundary list; MD9 forces proof obligations; MD3–MD6 bind equivalence, canonicalization, operators, profiles, and gates to identifiers. Thus assumptions cannot remain implicit without violating the template. □

F5. Proof or status

All non-definitional claims are derived with explicit proofs.

F6. Model notes

Model note. This node is itself model-agnostic. It defines how models must be declared, not which models are preferred.

F7. Examples (non-definitional)

Example (informative). A boundary-calculus model may declare a boundary state representation and masking operators and list "stealth" claims as Model-Specific.

F8. Limitations

Limitations. This node does not supply any concrete model. It supplies the admissible declaration format.

F9. Local DoD

Local DoD. Complete iff: (i) MD1–MD9 template is fixed, (ii) admissibility criterion is fixed, (iii) auditability of assumptions is derived and proved.

0.7.2 VII.2 Scorpio boundary calculus model class (P-VII.C2)

F1. Purpose

Purpose. This node defines an admissible *boundary-calculus* model class, denoted $\mathfrak{M}_{\text{Scorpio}}$, characterized by a boundary state Σ_B and boundary operators for serialization, masking, and phase control. All properties beyond the PSP contract are explicitly Model-Specific.

F2. Dependencies

Dependencies. Requires §0.1.6 (model admissibility), §0.3.3 (boundary operators), and Part VI (proof objects).

F3. Definitions (Model-Specific unless stated otherwise)

(Defined) Boundary state. Define a boundary state space Σ_B as a typed record

$$\Sigma_B := (\text{msg},\ \text{hdr},\ \theta,\ \kappa,\ RD),$$

where msg is payload, hdr is header/metadata, and (θ, κ) are control parameters fixed by RD.

(Defined) Boundary encoder. Define a projection $\pi_B : \Sigma \to \Sigma_B$ (a member of Π) that is deterministic and admissible only at explicit IO boundaries.

(Defined) Boundary operator family. Define a boundary operator family $\mathcal{O}_B$ containing operators:

$$\Omega_{\text{gen}},\ \Omega_{\text{ser}},\ \Omega_{\text{mask}},\ \Omega_{\text{phase}} : \Sigma_B \to \Sigma_B,$$

each with admissibility predicates and evidence obligations.

(Model-Specific) Boundary gate. Define a boundary acceptance predicate $q_B : \Sigma_B \to \{0, 1\}$ with complete evidence schema (Part VI.2).

(Model-Specific) Boundary run. A boundary run is the composition

$$\Sigma \xrightarrow{\pi_B} \Sigma_B \xrightarrow{\Omega_{\text{ser}} \circ \Omega_{\text{mask}} \circ \Omega_{\text{phase}}} \Sigma'_B$$

with all IO recorded as evidence and replay performed by evidence reuse.

(Model-Specific) Boundary statement. All claims about "protocol mimicry", "stealth", "non-identifiability", or "invisibility" are Model-Specific and MUST NOT be treated as foundational. They are admissible only as boundary predicates whose evidence is explicitly declared and replayable.

F4. Derived statements

(Derived) Boundary calculus preserves PSP determinism if IO is evidence-captured. If all external inputs/outputs are captured as canonical evidence and replay reuses that evidence, then boundary runs are replay-stable.

Proof. This is the boundary-verification principle applied to boundary operators: by recording and reusing external IO, the effective operator inputs are fixed, so the boundary transformation is deterministic under replay. □

F5. Proof or status

The determinism preservation claim is Derived. Any "stealth" claims remain Model-Specific and require explicit evidence predicates.

F6. Model notes

Model note. $\mathfrak{M}_{\text{Scorpio}}$ is an admissible model family iff it satisfies the declaration template (VII.1) and SI1–SI9. This node defines the minimum structure, not a privileged implementation.

F7. Examples (non-definitional)

Example (informative). A boundary gate may check (i) syntactic header invariants, (ii) phase-window constraints, and (iii) masking entropy bounds, each recorded as evidence.

F8. Limitations

Limitations. This node does not define concrete masking or phase formulas. It defines admissible boundary structure and status discipline.

F9. Local DoD

Local DoD. Complete iff: (i) Σ_B and boundary operator family are defined, (ii) boundary statement forbids foundational leakage, (iii) replay-stability under evidence capture is derived and proved.

0.7.3 VII.3 AinSoft runtime-mesh model class (P-VII.C3)

F1. Purpose

Purpose. This node defines an admissible *runtime-mesh* model class, denoted $\mathfrak{M}_{\text{AinSoft}}$, in which the state includes a mesh-like or manifold-like internal representation and operators act as structured projections/rotations/rewires. All geometric semantics are Model-Specific unless proved portable.

F2. Dependencies

Dependencies. Requires Part II (state encoding), Part III (operators), Part IV (profiles), and Part V (dynamics/closure).

F3. Definitions (Model-Specific unless stated otherwise)

(Model-Specific) Mesh component. Let the graph component G be enriched with a mesh structure $\mathcal{M}$ encoded either as additional node/edge types or as auxiliary tensors accessible via Π:

$$G_{\mathcal{M}} := (V, E, \tau;\ \mathcal{M}).$$

(Defined) Geometric operator family. Define a geometric operator family $\mathcal{O}_{geo} \subseteq \otimes$ whose members are deterministic transforms of the mesh encoding, subject to WF preservation and budget discipline.

(Model-Specific) Operator archetypes. A model MAY define named operator archetypes (e.g. *Moebius*, *Metatron*, *Thunderbolt*) as members of $\mathcal{O}_{geo}$, but each such archetype MUST provide: (i) a typed signature, (ii) admissibility predicates, (iii) evidence obligations, and (iv) boundary statement about scope.

(Model-Specific) Runtime update. A runtime update step is a morph/solve composition that updates $G_{\mathcal{M}}$ and z while maintaining determinism and audit evidence.

F4. Derived statements

(Derived) AinSoft-class models remain PSP-admissible under determinism and WF preservation. If $\mathfrak{M}_{\text{AinSoft}}$ provides deterministic canonicalization, deterministic operators, and preserves WF and SI1–SI9, then it is an admissible model class.

Proof. Directly from the admissibility definition (Part I.6) and the model declaration template (VII.1): determinism, canonicalization, and invariants suffice. Geometric interpretation adds Model-Specific semantics but does not alter PSP foundations. □

F5. Proof or status

Admissibility claim is Derived. All geometric semantics remain Model-Specific unless independently proved portable.

F6. Model notes

Model note. This node defines a model family boundary. It does not assert physical truth of any geometry; it asserts PSP compatibility.

F7. Examples (non-definitional)

Example (informative). A model may define a Moebius operator as a deterministic reparameterization of mesh coordinates plus a constrained rewire, recorded as evidence.

F8. Limitations

Limitations. This node does not define mesh mathematics. It defines the admissible contract and status discipline for such a model.

F9. Local DoD

Local DoD. Complete iff: (i) runtime-mesh enrichment is declared as Model-Specific, (ii) geometric operator family obligations are fixed, (iii) PSP-admissibility is derived and proved.

0.7.4 VII.4 Metatron-cube operator calculus as a discrete symmetry model (P-VII.C4)

F1. Purpose

Purpose. This node defines an admissible model class $\mathfrak{M}_{\text{Metatron}}$ in which the state dynamics are organized by a discrete symmetry graph (a "Metatron cube" style archetype). The model is treated as a discrete symmetry/adjacency framework, not as a metaphysical claim.

F2. Dependencies

Dependencies. Requires Part II (typed graph), Part III (operators), and Part IV (profiles).

F3. Definitions (Model-Specific unless stated otherwise)

(Model-Specific) Symmetry graph. Define a fixed finite graph $G_M = (V_M, E_M)$ with a distinguished symmetry action group Γ acting on V_M by permutations. G_M is treated as a discrete scaffold embedded into the PSP graph component via a declared embedding $\iota_M : V_M \hookrightarrow V$.

(Model-Specific) Symmetry operators. Define symmetry operators $\mathcal{O}_\Gamma \subseteq \otimes$ that act by Γ-permutations on embedded nodes and preserve typing and canonicalization.

(Model-Specific) Adjacency-driven propagation. A propagation operator Ω_{prop} updates selected substructures in G based on adjacency relations in G_M, subject to deterministic scheduling and evidence recording.

F4. Derived statements

(Derived) Discrete symmetry models preserve determinism under explicit group action. If Γ actions are implemented as deterministic permutations of a canonical labeling and all propagation updates are deterministic, then symmetry-driven computation is replay-stable.

Proof. A permutation of a canonically labeled structure is deterministic. With deterministic scheduling and update rules, the resulting state transitions are uniquely determined by canonical inputs and RD, hence replay-stable. □

F5. Proof or status

Replay-stability claim is Derived. Any claims about "universality" or "physicality" of the symmetry graph remain Model-Specific or Open unless proved.

F6. Model notes

Model note. This model class is compatible with PSP as a discrete symmetry scaffold. It must still satisfy the declaration template and SI1–SI9.

F7. Examples (non-definitional)

Example (informative). A propagation step may update a neighborhood around an embedded node by deterministic rules and record the chosen orbit elements as evidence.

F8. Limitations

Limitations. This node does not define a particular G_M; it defines the admissible structure for using a fixed symmetry scaffold.

F9. Local DoD

Local DoD. Complete iff: (i) symmetry scaffold embedding is defined as Model-Specific, (ii) symmetry operators and propagation are defined with determinism/evidence obligations, (iii) replay-stability is derived and proved.

0.7.5 VII.5 Tripolar logic as a PSP profile + gate family (P-VII.C5)

F1. Purpose

Purpose. This node defines an admissible *tripolar* model instantiation within PSP as (i) a measurement profile family and (ii) a gate family. Tripolarity is treated operationally: as a stable three-region classification induced by deterministic measurement and gated dynamics, not as an ontological claim.

F2. Dependencies

Dependencies. Requires Part IV (profiles), Part VI (gates/proof objects), and Part V (dynamics/closure).

F3. Definitions (Model-Specific unless stated otherwise)

(Model-Specific) Tripolar profile. A tripolar profile is a deterministic map

$$m_{tri} : \Sigma \to [0,1]^3, \qquad m_{tri}(\Sigma) = (\psi(\Sigma), \rho(\Sigma), \omega(\Sigma)),$$

with explicit coordinate semantics and normalization declared per Part IV.

(Model-Specific) Tripolar classification. Define a classification map

$$\chi_{tri} : \Sigma \to \{L_0, L_D, L_1\}$$

using thresholds $(\theta_0, \theta_1, \delta)$ recorded in $RD.params$, such that:

- L_0 denotes a stable low region,
- L_1 denotes a stable high region,
- L_D denotes a dynamic/intermediate region.

The classification rule MUST be explicit, deterministic, and evidence-backed.

(Model-Specific) Tripolar gate family. A tripolar gate is a staged predicate family that MAY include: (i) a weighted-threshold predicate (Part VI.3) on (ψ, ρ, ω), (ii) a hysteresis band (Part V.2) to stabilize transitions, (iii) a persistence window N (Part I.9) to prevent flicker.

(Model-Specific) Tripolar evidence. Tripolar gates MUST record:

$$(\, m_{tri,id}, m_{tri,ver},\ (\psi, \rho, \omega),\ \theta_0, \theta_1, \delta,\ N,\ \text{outcome},\ \text{class}\,),$$

as complete gate evidence items.

F4. Derived statements

(Derived) Tripolar gates are replay-stable under deterministic profiles. If m_{tri} and all thresholds are RD-bound and deterministic, then χ_{tri} and tripolar gate outcomes are replay-stable.

Proof. Replay reproduces identical canonical states, hence identical (ψ, ρ, ω) values and identical threshold parameters from RD. Therefore classification and gate outcomes are identical under replay. □

F5. Proof or status

Replay-stability is Derived. Any claims about "optimality" or "universality" of tripolarity remain Model-Specific or Open unless proved.

F6. Model notes

Model note. This node is an admissible model instantiation: it does not alter PSP foundations, it supplies a concrete profile+gate family.

F7. Examples (non-definitional)

Example (informative). A system may interpret L_D as a "decision basin" requiring dual-fabric consensus before committing to L_0 or L_1.

F8. Limitations

Limitations. No dynamical-system theorems are asserted here. Only determinism, explicit semantics, and evidence requirements are fixed.

F9. Local DoD

Local DoD. Complete iff: (i) tripolar profile and classification are defined deterministically, (ii) tripolar gates are defined as staged/hysteretic predicates, (iii) evidence schema is complete, (iv) replay-stability is derived and proved.

0.7.6 VII.6 Pi-operator and projection-chain extensions (P-VII.C6)

F1. Purpose

Purpose. This node defines an admissible extension pattern: *projection-chain operators* (Pi-operators) that compose multiple representation projections/embeddings into a single typed operator with explicit admissibility and evidence.

F2. Dependencies

Dependencies. Requires §0.2.4 (projection set Π) and Part III (operator registry).

F3. Definitions (Model-Specific unless stated otherwise)

(Defined) Projection chain. A projection chain is a finite sequence of typed maps

$$\pi := \pi_r \circ \cdots \circ \pi_1$$

such that the codomain of π_i matches the domain of π_{i+1} for all i.

(Model-Specific) Pi-operator. A Pi-operator is an operator $\Omega_\Pi : \Sigma \to \Sigma$ that: (i) applies a declared projection chain π to some component(s) of Σ, (ii) applies a declared transform in the projected space, (iii) re-embeds into Σ via an inverse or left-inverse embedding where declared. All steps MUST be deterministic under RD.

(Model-Specific) Pi-admissibility. Ω_Π is admissible only if:

- Π1. each π_i is admissible and deterministic under RD,
- Π2. the chain domain/codomain types match,
- Π3. the intermediate transform preserves declared well-formedness constraints,
- Π4. evidence records the chain identifier, intermediate digests, and the re-embedding rule used.

F4. Derived statements

(Derived) Deterministic projection chains preserve replay stability. If each chain step is deterministic and evidence records intermediate digests, then the Pi-operator step is replay-stable.

Proof. Replay reproduces identical canonical inputs and identical deterministic projections and transforms. Intermediate digests match; therefore the final embedded state matches. □

F5. Proof or status

Replay-stability is Derived. Existence of inverse embeddings is Model-Specific unless proved or constructed.

F6. Model notes

Model note. Pi-operators are a general extension mechanism used to connect PSP spaces (graph space, profile space, boundary space) without conflation. They must obey the projection determinism and compatibility contracts (Part II.4).

F7. Examples (non-definitional)

Example (informative). A Pi-operator may project a graph neighborhood into a feature vector space, apply a deterministic normalization/rotation, and re-embed by a declared mapping.

F8. Limitations

Limitations. This node does not assert that any projection chain is information-preserving. Such claims must be stated as obligations (Part VI.6) and proved or marked Open.

F9. Local DoD

Local DoD. Complete iff: (i) projection chain concept is defined, (ii) Pi-operator admissibility and evidence obligations are fixed, (iii) replay-stability is derived and proved, (iv) inverse-existence claims are properly status-labeled.

0.7.7 VII.7 Dual-fabric tripolar stabilization (P-VII.C7)

F1. Purpose

Purpose. This node defines an admissible stabilization pattern that combines tripolar classification with dual-fabric consensus. It yields a robust decision discipline for committing a state from the dynamic region L_D into L_0 or L_1.

F2. Dependencies

Dependencies. Requires §0.7.5 (tripolar model) and §0.1.8 (dual fabrics), plus Part VI (consensus certificates).

F3. Definitions (Model-Specific unless stated otherwise)

(Model-Specific) Dynamic-region commit rule. Let $\chi_{tri}(\Sigma) = L_D$. A commit rule is a staged predicate that requires:

TC1. dual-fabric acceptance $\mathrm{Accept}_\tau(\Sigma_0) = 1$ for a declared task τ,
TC2. a consensus certificate $\mathcal{C}_\tau$ that verifies successfully,
TC3. persistence of classification (remain in L_D or converge to the target) over window N.

(Model-Specific) Commit operator. A commit operator Ω_{commit} is a gate+annotation operator that records a commitment event in H and MAY trigger constraint tightening policies (Part V.8). It MUST NOT alter semantic state beyond audited commitment metadata.

F4. Derived statements

(Derived) Commit decisions are replay-stable. If tripolar classification, dual-fabric runs, and certificate verification are replay-stable, then commit decisions are replay-stable.

Proof. The commit rule is a deterministic conjunction of replay-stable predicates and certificate verification outcomes. Therefore the decision is invariant under replay. □

F5. Proof or status

Replay-stability is Derived. Any claims about "optimal" stabilization remain Model-Specific unless proved.

F6. Model notes

Model note. This is an admissible decision discipline pattern. It is not required for all PSP systems.

F7. Examples (non-definitional)

Example (informative). A system may interpret L_D as an "undecided" regime and only allow emission of certain artifacts after a commit event backed by a consensus certificate.

F8. Limitations

Limitations. This node does not prove that commits correspond to external truth; it proves replay-stability and auditability.

F9. Local DoD

Local DoD. Complete iff: (i) commit rule is defined with certificate requirement, (ii) commit operator is defined as audit-only, (iii) replay-stability is derived and proved.

0.7.8 VII.8 AinSoft–Scorpio interoperability via boundary projections (P-VII.C8)

F1. Purpose

Purpose. This node defines an admissible interoperability pattern between runtime-mesh models (AinSoft class) and boundary-calculus models (Scorpio class) via explicit boundary projections and Pi-operator chains, without conflating internal semantics.

F2. Dependencies

Dependencies. Requires §0.7.2 (Scorpio), §0.7.3 (AinSoft), and §0.7.6 (Pi-operators).

F3. Definitions (Model-Specific unless stated otherwise)

(Model-Specific) Boundary projection interface. Define a projection $\pi_{A\to B} : \Sigma_{\text{AinSoft}} \to \Sigma_B$ that encodes a subset of the runtime state into a boundary state, and a projection $\pi_{B\to A} : \Sigma_B \to \Sigma_{\text{AinSoft}}$ that decodes admissible boundary inputs into runtime updates. Both MUST be deterministic and RD-bound.

(Model-Specific) Interop contract. Interop is admissible only if:

IO1. both projections are admissible members of Π and satisfy Part II.4,
IO2. boundary IO is evidence-captured (Part VI.5),
IO3. decoding updates preserve WF and budgets,
IO4. any claims about "stealth" or "protocol mimicry" remain Model-Specific and are never imported into AinSoft semantics.

F4. Derived statements

(Derived) Interop preserves replay stability under evidence capture. If $\pi_{A\to B}, \pi_{B\to A}$ are deterministic and boundary IO is evidence-captured, then AinSoft–Scorpio interop runs are replay-stable.

Proof. Evidence capture fixes boundary inputs under replay. Deterministic projections and updates yield identical decoded runtime updates and identical boundary encodings, so the composed interop behavior is replay-stable. □

F5. Proof or status

Replay-stability is Derived. Functional correctness claims remain Model-Specific unless proved.

F6. Model notes

Model note. This node defines a safe interoperability template. It does not claim that every AinSoft state can be encoded as a boundary message.

F7. Examples (non-definitional)

Example (informative). A runtime step may emit a boundary summary message for audit or external negotiation; incoming boundary commands may be decoded into constrained morph steps.

F8. Limitations

Limitations. This node does not define concrete message formats; it defines the admissible projection and evidence-capture contract.

F9. Local DoD

Local DoD. Complete iff: (i) interoperability projections are defined as deterministic and RD-bound, (ii) evidence-capture and model-boundary rules are fixed, (iii) replay-stability is derived and proved.

0.8 Part VIII — Verification & Adversarial Harness (P-VIII)

0.8.1 VIII.1 Verification layers: replay, recomputation, differential checks (P-VIII.C1)

F1. Purpose

Purpose. This node defines the verification layer stack for PSP: replay verification, recomputation verification, and differential verification. It fixes the admissible ways to increase confidence in PSP artifacts without introducing nondeterminism.

F2. Dependencies

Dependencies. Requires Part I (determinism), Part VI (proof objects), and Part III (boundary operators for external tools).

F3. Definitions (Defined)

(Defined) Replay verification. Replay verification checks that rerunning (or replaying) a run under the same (Σ_0, P, RD) reproduces: (i) the same trace digests and (ii) the same artifact digests.

(Defined) Recomputation verification. Recomputation verification checks that designated measurements, predicates, and derived witnesses can be recomputed from recorded evidence and match recorded values within tolerances fixed in RD.

(Defined) Differential verification. Differential verification compares two executions (e.g. dual fabrics, alternate decompositions, alternate projections) and checks agreement criteria (predicate agreement, MCI threshold, bounded divergence).

(Defined) Verification pack. A verification pack is a proof object bundle

$$\mathcal{VP} := (\mathcal{VP}_{id}, \mathcal{VP}_{ver},\ \{\mathcal{P}_j\}_{j=1}^{r},\ RD),$$

where each $\mathcal{P}_j$ is a proof object with a deterministic verifier.

F4. Derived statements

(Derived) Verification layer monotonicity. If replay verification passes, then any deterministic recomputation verification based on the same evidence is well-defined under replay.

Proof. Replay verification guarantees that the same evidence and trace anchors are reproduced. Deterministic recomputation functions on that evidence are therefore well-defined and replay-stable. □

F5. Proof or status

All non-definitional claims are Derived with explicit proof.

F6. Model notes

Model note. Models may add specialized verification packs, but must adhere to deterministic verifiers and evidence-capture rules.

F7. Examples (non-definitional)

Example (informative). A pack may include: replay digest checks, gate predicate recomputation checks, and dual-fabric consensus certificate verification.

F8. Limitations

Limitations. Passing verification increases internal consistency and replayability; it does not establish external truth or domain correctness.

F9. Local DoD

Local DoD. Complete iff: (i) replay/recomputation/differential verification are defined, (ii) verification packs are defined, (iii) layer monotonicity is derived and proved.

0.8.2 VIII.2 Null-point injection: mismatch channels and robustness probes (P-VIII.C2)

F1. Purpose

Purpose. This node defines null-point injection as an adversarial robustness probe: controlled perturbations that test whether acceptance, gating, and artifact outputs are stable under declared mismatch channels. The goal is to expose hidden dependencies and brittle operator chains.

F2. Dependencies

Dependencies. Requires Part II (distance d, profiles), Part VI (gates/proof objects), and Part I.8 (dual-fabric robustness).

F3. Definitions (Defined)

(Defined) Injection operator. An injection operator is a deterministic operator

$$\Omega_{\text{inj}}^{(\delta)} : \Sigma \to \Sigma$$

parameterized by an injection descriptor δ fixed in $RD.params$.

(Defined) Mismatch channel. A mismatch channel is a named perturbation mode applied by $\Omega_{\text{inj}}^{(\delta)}$, such as:

- representation perturbations (graph relabelings within $\sim_G$),
- measurement noise within declared tolerance,
- projection-chain perturbations (alternate admissible Π paths),
- controlled morph edits under budget,
- boundary-message perturbations (within admissible header/payload constraints).

Channels MUST be explicit and admissible under the active model boundary.

(Defined) Robustness probe. Given a task predicate Q_τ and baseline run output Σ_k, a robustness probe is the comparison of:

$$\Sigma_k \text{ vs. } \Sigma_k^{(\delta)} := \text{Run}(\Omega_{\text{inj}}^{(\delta)}(\Sigma_0), P, RD).\Sigma_k$$

evaluated by: (i) predicate stability, (ii) MCI threshold, and (iii) distance bound $d(\Sigma_k, \Sigma_k^{(\delta)}) \leq \epsilon$.

(Defined) Injection evidence. Injection evidence MUST record: channel id, δ, impacted components, and the measured divergence metrics.

F4. Derived statements

(Derived) Injection probes detect hidden dependence on non-semantic representation choices. If an injection channel changes only representation within $\sim_\Sigma$ and the system is canonical-compatible, then acceptance should remain invariant; deviations indicate hidden non-semantic dependence.

Proof. Canonical compatibility collapses representational variance. If the system is well-formed and operators respect semantic equivalence, then outputs should be semantically equivalent. If acceptance differs, then some step depends on representation artifacts not collapsed by canonicalization, revealing a hidden dependence. □

F5. Proof or status

All non-definitional claims are Derived with explicit proof. Model-specific channels remain Model-Specific by boundary.

F6. Model notes

Model note. Injection channels must obey model boundaries and must not violate SI1–SI9. Any "adversarial" behavior must still be deterministic and RD-bound.

F7. Examples (non-definitional)

Example (informative). Apply a relabeling injection to a graph representation, then verify that gate acceptance and artifact digests remain unchanged.

F8. Limitations

Limitations. Injection probes do not prove correctness; they expose brittleness and hidden channels and provide evidence for robustness.

F9. Local DoD

Local DoD. Complete iff: (i) injection operator and mismatch channels are defined, (ii) robustness probe criteria are defined, (iii) hidden dependence detection property is derived and proved, (iv) evidence obligations are fixed.

0.8.3 VIII.3 Differential testing: dualization suites and invariance checks (P-VIII.C3)

F1. Purpose

Purpose. This node defines differential testing as a systematic method: executing dual fabrics, alternate decompositions, or alternate projection chains and checking invariance conditions with explicit tolerances. It operationalizes dual-fabric discipline as a test harness.

F2. Dependencies

Dependencies. Requires Part I.8 (dual fabrics), Part IV (profiles/metrics), and Part VI.4 (consensus certificates).

F3. Definitions (Defined)

(Defined) Dualization suite. A dualization suite is a finite set of fabric pairs

$$\mathcal{S}_{dual} := \{(\mathcal{F}_1^{(j)}, \mathcal{F}_2^{(j)})\}_{j=1}^{N}$$

with declared task predicates and declared measurement maps used for MCI.

(Defined) Invariance check. An invariance check is a predicate requiring agreement across a fabric pair:

$$\mathrm{Inv}_\tau^{(j)}(\Sigma_0) = 1 \iff \mathrm{Accept}_\tau(\Sigma_0; \mathcal{F}_1^{(j)}, \mathcal{F}_2^{(j)}) = 1.$$

(Defined) Differential test pack. A differential test pack is a verification pack whose proof objects include consensus certificates for each suite pair and a summary evidence record (pass/fail per pair).

F4. Derived statements

(Derived) Differential testing reduces one-sided acceptance errors. If acceptance requires agreement across dual fabrics for a diverse suite, then acceptance is less sensitive to idiosyncrasies of any single decomposition.

Proof. Acceptance is conditioned on multiple independent realizations. Any idiosyncratic artifact that appears only in one realization fails agreement. Thus one-sided errors are filtered out by construction. □

F5. Proof or status

All non-definitional claims are Derived with explicit proof.

F6. Model notes

Model note. Suite diversity is model-specific; this node defines only the admissible structure and evidence requirements.

F7. Examples (non-definitional)

Example (informative). Compare a sequential operator chain to an operator DAG decomposition for the same task and require a consensus certificate.

F8. Limitations

Limitations. Differential testing does not guarantee external correctness; it enforces internal invariance and robustness.

F9. Local DoD

Local DoD. Complete iff: (i) dualization suites are defined, (ii) invariance checks are defined, (iii) robustness effect is derived and proved, (iv) differential test pack structure is defined.

0.8.4 VIII.4 Artifact diffs: structural, numeric, and semantic diff contracts (P-VIII.C4)

F1. Purpose

Purpose. This node defines artifact diffing contracts used to compare outputs across runs, versions, and dual fabrics. It fixes deterministic diff semantics: structural diffs, numeric diffs with tolerances, and semantic diffs via manifests.

F2. Dependencies

Dependencies. Requires Part II (canonicalization, numeric discipline), Part VI (manifests/proof objects), and Part I.4 (digests).

F3. Definitions (Defined)

(Defined) Artifact identity. An artifact a has identity by digest $\mathrm{Dig}(a)$ and manifest binding (run id, operator segment, RD).

(Defined) Structural diff. A structural diff is a deterministic function $\Delta_{str}(a, a')$ producing a finite edit script over a declared artifact schema.

(Defined) Numeric diff. A numeric diff is a deterministic function $\Delta_{num}(a, a')$ producing a set of coordinate-wise deviations with tolerance parameters fixed in RD.

(Defined) Semantic diff. A semantic diff is a comparison of manifest claims: provenance equality, gate outcomes, profile ids/versions, and proof-pack validity. Semantic diffs MUST be computed from canonical manifest fields and proof-object verification outcomes.

(Defined) Diff admissibility. A diff procedure is admissible iff it is deterministic under RD and declares all tolerance parameters explicitly.

F4. Derived statements

(Derived) Deterministic diffs enable regression detection. If diffs are deterministic and tolerances are RD-bound, then changes across runs are mechanically detectable and attributable.

Proof. Deterministic diffs yield stable outputs for the same inputs. RD-bound tolerances prevent threshold drift. Therefore any change in artifacts beyond tolerances produces a stable diff record that can be attributed to specific run/version changes. □

F5. Proof or status

All non-definitional claims are Derived with explicit proof.

F6. Model notes

Model note. Artifact schemas are domain-specific. This node defines diff contracts and determinism requirements, not schema content.

F7. Examples (non-definitional)

Example (informative). A regression suite may accept numeric diffs within ε while requiring semantic diffs (gate outcomes, proof-pack validity) to match exactly.

F8. Limitations

Limitations. Diffs do not assert correctness; they provide deterministic change detection.

F9. Local DoD

Local DoD. Complete iff: (i) structural/numeric/semantic diffs are defined, (ii) diff admissibility is fixed, (iii) regression detection property is derived and proved.

0.9 Part IX — Operationalization (P-IX)

0.9.1 IX.1 Execution manifests: run identity, provenance, and replay packs (P-IX.C1)

F1. Purpose

Purpose. This node defines the execution manifest as the canonical operational artifact of PSP: a deterministic run identity, a provenance binding, and a replay pack sufficient to reproduce trace and artifact digests.

F2. Dependencies

Dependencies. Requires Part I.4 (RD/replay), Part II.6 (history), Part VI.1 (proof objects), and Part VIII.1 (verification packs).

F3. Definitions (Defined)

(Defined) Run identity. Define a run identity as a digest over canonicalized run inputs:

$$\mathrm{RunID} := \mathrm{Dig}\big(\mathrm{Canon}(\Sigma_0) \parallel P_{\mathrm{id}} \parallel RD\big),$$

where P_{id} is a stable identifier of the program form (sequence/DAG) and operator versions.

(Defined) Execution manifest. An execution manifest is a record

$$\mathcal{M}_{exec} := (\mathrm{RunID},\ RD,\ P_{\mathrm{id}},\ \mathcal{T}_{dig},\ \mathcal{A}_{dig},\ \mathcal{VP}_{dig},\ \mathsf{meta}),$$

where $\mathcal{T}_{dig}$ is the digest of the canonical trace, $\mathcal{A}_{dig}$ is the set of artifact digests, and $\mathcal{VP}_{dig}$ is the digest (or set of digests) of verification packs.

(Defined) Replay pack. A replay pack is a finite bundle

$$\mathcal{RP} := (\mathrm{Canon}(\Sigma_0),\ P,\ RD,\ \mathcal{E}_{IO}),$$

where $\mathcal{E}_{IO}$ is the set of evidence items required to replay boundary operators without re-querying the environment.

(Defined) Manifest well-formedness. An execution manifest is well-formed iff:

EM1. RunID is computed from canonical inputs and recorded algorithms in RD,
EM2. RD is complete and immutable for the run,
EM3. P_{id} resolves to a fully versioned operator set (registry entries),
EM4. $\mathcal{T}_{dig}$ and $\mathcal{A}_{dig}$ correspond to canonical trace/artifact digests in H,
EM5. any boundary evidence required for replay is included in $\mathcal{RP}$ or referenced by digest.

F4. Derived statements

(Derived) Execution manifests enable deterministic run attribution. If RunID is computed from canonical inputs and RD, then two runs with identical semantic inputs and identical RD share the same RunID.

Proof. Canonicalization collapses representational equivalence; digest computation is deterministic under the same algorithm id. Therefore equal canonical inputs yield equal run identity. □

(Derived) Replay packs are sufficient for boundary-replay when IO evidence is captured. If boundary operators record all external inputs/outputs as canonical evidence and that evidence is included in $\mathcal{RP}$, then replay does not require environmental queries.

Proof. The effective boundary inputs are the captured evidence. Reusing it makes the boundary operator a deterministic computation on fixed inputs. Hence environmental queries are unnecessary for replay. □

F5. Proof or status

All non-definitional claims are Derived with explicit proofs.

F6. Model notes

Model note. Models may extend the manifest metadata, but MUST preserve the core identity/provenance/replay fields.

F7. Examples (non-definitional)

Example (informative). A manifest may include "first failing gate stage index" and a compact summary of constraint set versions used.

F8. Limitations

Limitations. A manifest certifies provenance and replayability, not domain correctness.

F9. Local DoD

Local DoD. Complete iff: (i) RunID is defined from canonical inputs, (ii) execution manifest schema is defined, (iii) replay pack schema is defined, (iv) attribution and boundary-replay sufficiency are derived and proved.

0.9.2 IX.2 Artifact manifests and provenance bundles (P-IX.C2)

F1. Purpose

Purpose. This node defines the artifact manifest: the per-artifact provenance bundle binding each artifact to (1) a producing run, (2) a producing operator segment, and (3) supporting gate and proof-object evidence.

F2. Dependencies

Dependencies. Requires Part II.6 (history/manifest), Part VI (proof objects), and Part IX.1 (execution manifest).

F3. Definitions (Defined)

(Defined) Artifact manifest. For an artifact a, define its manifest

$$\mathcal{M}_a := (a_{\mathrm{id}}, a_{\mathrm{ver}},\ \mathrm{Dig}(a),\ \mathrm{RunID},\ seg,\ gates,\ proofs,\ RD),$$

where:

- $(a_{\mathrm{id}}, a_{\mathrm{ver}})$ identify artifact schema/version,
- seg identifies the producing operator subsequence/subgraph (trace indices or node IDs),
- $gates$ references gate evidence items (digests),
- $proofs$ references proof objects / verification packs (digests).

(Defined) Provenance bundle. A provenance bundle is the pair $(\mathcal{M}_{exec}, \mathcal{M}_a)$ together with referenced evidence required to verify claims.

(Defined) Provenance verification. Provenance verification passes iff:

PV1. RunID in $\mathcal{M}_a$ matches $\mathcal{M}_{exec}$,

PV2. *seg* resolves to a producing segment in the trace,
PV3. gate evidence and proof object references verify successfully,
PV4. the artifact digest matches $\mathrm{Dig}(a)$ computed under $RD.hash$.

F4. Derived statements

(Derived) Provenance bundles enable per-artifact replay attribution. If PV1–PV4 hold and the run is replayable, then the artifact can be re-attributed to the same producing segment under replay.

Proof. Replay reproduces the trace digests and segment identity. The manifest binds the artifact digest to the segment and run identity, and verification checks match. Therefore the artifact is attributable to the same producing segment. □

F5. Proof or status

All non-definitional claims are Derived with explicit proofs.

F6. Model notes

Model note. Artifact schema specifics are domain-specific. The manifest/provenance binding is universal.

F7. Examples (non-definitional)

Example (informative). A generated API blueprint artifact may include proofs referencing compilation/test gates and a verification pack that recomputes all gate predicates.

F8. Limitations

Limitations. Provenance does not establish that the artifact is useful; it establishes reproducible lineage and verifiable acceptance evidence.

F9. Local DoD

Local DoD. Complete iff: (i) artifact manifest schema is defined, (ii) provenance verification conditions PV1–PV4 are defined, (iii) replay attribution property is derived and proved.

0.9.3 IX.3 Versioning, upgrades, and compatibility contracts (P-IX.C3)

F1. Purpose

Purpose. This node fixes versioning and compatibility contracts for PSP systems: how spec versions, operator versions, profile versions, and model versions interact without silently changing semantics.

F2. Dependencies

Dependencies. Requires Part I.3 (status discipline), Part II.8 (RD schema), Part III.1 (operator registry), and Part IV.1 (profile versioning).

F3. Definitions (Defined)

(Defined) Version identifiers. Each of the following MUST be versioned:

$$rd\_ver,\ (\Omega_{\mathrm{id}}, \Omega_{\mathrm{ver}}),\ (m_{\mathrm{id}}, m_{\mathrm{ver}}),\ (\mathfrak{M}_{id}, \mathfrak{M}_{ver}),\ (a_{\mathrm{id}}, a_{\mathrm{ver}}).$$

(Defined) Semantic change. A semantic change is any modification that can alter $\mathrm{Canon}(\Sigma_k)$ or any artifact digest for the same canonical inputs and the same declared *RD* fields.

(Defined) Upgrade rule. If a component undergoes a semantic change, its version MUST increment, and any run using the new component MUST record the new version in *RD* and manifests.

(Defined) Compatibility class. Define compatibility classes for upgrades:

- **Replay-compatible:** identical canonical outputs for all calibrated fixtures and declared domains.
- **Evidence-compatible:** outputs may change, but evidence schemas and verification procedures remain valid.
- **Breaking:** changes invalidate replay comparability or evidence verification.

A change MUST declare its compatibility class.

F4. Derived statements

(Derived) Version discipline prevents silent regressions. If semantic changes force version increments and manifests record versions, then regressions are detectable by comparing manifests and diffs.

Proof. Manifests expose component identities. A semantic change without version increment would be silent; the rule forbids that. Therefore changes become explicit, enabling deterministic diff/regression checks. □

F5. Proof or status

All non-definitional claims are Derived with explicit proof.

F6. Model notes

Model note. Models may impose stricter compatibility rules (e.g. only replay-compatible upgrades allowed). They may not weaken the semantic-change version rule.

F7. Examples (non-definitional)

Example (informative). Changing a profile coordinate meaning is a semantic change and must increment m_{ver} even if the numeric range stays $[0, 1]$.

F8. Limitations

Limitations. This node does not define a global semantic versioning scheme; it defines the required invariants for version discipline.

F9. Local DoD

Local DoD. Complete iff: (i) version identifiers are mandatory, (ii) semantic change and upgrade rule are defined, (iii) compatibility classes are defined, (iv) anti-silent-regression property is derived and proved.

0.9.4 IX.4 Governance: safety boundaries, misuse constraints, and publication discipline (P-IX.C4)

F1. Purpose

Purpose. This node defines governance constraints that bound PSP systems to auditable, deterministic, and non-deceptive operation. It fixes publication and usage discipline so that model-specific boundary behaviors cannot be misrepresented as universal guarantees.

F2. Dependencies

Dependencies. Requires Part I.3 (status taxonomy), Part VI.8 (residual risks), and Part VII (model boundaries).

F3. Definitions (Defined)

(Defined) Boundary integrity rule. Any claim about boundary behaviors (e.g. protocol mimicry, masking, stealth) MUST be labeled Model-Specific and MUST include: (i) explicit boundary statement, (ii) admissible evidence schema, (iii) residual risk item if unproven.

(Defined) Non-deception constraint. PSP-conformant publications MUST NOT present Model-Specific predicates as universal or foundational properties. Foundational PSP guarantees are limited to determinism, replayability, auditability, and explicit status discipline.

(Defined) Misuse constraint. PSP systems MUST document which model components could be misused and MUST provide configuration constraints that keep runs within auditable replay boundaries (e.g. evidence-capture requirement for boundary operators).

F4. Derived statements

(Derived) Governance constraints preserve interpretability of PSP claims. If model boundaries and proof-status labels are enforced and publications respect the non-deception constraint, then readers can mechanically distinguish foundational guarantees from model-specific behaviors.

Proof. Status labels and boundary statements classify claims. The publication constraint forbids mislabeling. Therefore the distinction remains explicit and checkable. □

F5. Proof or status

All non-definitional claims are Derived with explicit proof.

F6. Model notes

Model note. Governance is orthogonal to implementation. It constrains how claims are stated and how boundary behaviors are bounded and audited.

F7. Examples (non-definitional)

Example (informative). If a boundary model includes masking operators, it must declare evidence rules and must not claim "undetectable" behavior as a foundational guarantee.

F8. Limitations

Limitations. This node does not define legal policy. It defines specification-level discipline for truthful status classification and auditable boundaries.

F9. Local DoD

Local DoD. Complete iff: (i) boundary integrity and non-deception constraints are defined, (ii) misuse constraints are defined in audit terms, (iii) interpretability preservation is derived and proved.

.1 Appendix A1 — Definitions and notation registry (A1)

A1.1 Purpose

This appendix provides a single canonical registry of symbols, objects, and functions used in the specification. Every symbol used normatively in Parts I–IX MUST appear here exactly once with a stable definition.

A1.2 Core objects (canonical)

- Σ — PSP state space (Part II).
- $\Sigma = (G, z, \Pi, \mu, H, RD)$ — canonical PSP state tuple (II.1).
- $G = (V, E, \tau)$ — typed multigraph (II.2).
- $z \in \mathbb{D}^n$ — measurement vector (II.3).
- Π — projection/embedding set (II.4).
- $\mu : \Sigma \to \mathbb{R}$ — potential functional interface (II.5).
- $H = (\mathcal{T}, \mathcal{E}, \mathcal{M}, \mathcal{D})$ — history/evidence carrier (II.6).
- RD — run descriptor (II.8).
- $\otimes$ — admissible operator set (III.1).
- $\oplus$ — deterministic merge operator (II.9).
- $m : \Sigma \to \mathbb{D}^n$ — profile measurement map (II.3, Part IV).
- $d : \Sigma \times \Sigma \to \mathbb{R}_{\geq 0}$ — pseudo-distance witness (II.10).

A1.3 Equivalence and canonicalization

- $\sim_G$ — graph representation equivalence (II.2).
- $\sim_\Sigma$ — state representation equivalence (II.7).
- Canon : $\Sigma \to \Sigma$ — canonicalization map (I.4, II.7).
- $\equiv$ — semantic equality induced by canonicalization (I.7, II.1).

A1.4 Programs, fabrics, and verification

- P — operator program (sequence or DAG/HDAG) (III.2).
- $\mathcal{E}$ — evaluation policy (I.7, III.2).
- $\mathcal{F} = (P, \mathcal{E})$ — fabric (I.8).
- MCI — mirror-consistency index (I.8).
- $\mathcal{P}$ — proof object (VI.1).
- $\mathcal{VP}$ — verification pack (VIII.1).
- $\mathcal{M}_{exec}$ — execution manifest (IX.1).
- $\mathcal{M}_a$ — artifact manifest (IX.2).

A1.5 Status taxonomy (canonical)

- Axiomatic, Defined, Derived, Model-Specific, External, Open (I.3).

.2 Appendix A2 — Determinism and canonicalization rules (A2)

A2.1 Normative determinism rules

A2.1 All operators Ω MUST be deterministic functions of Canon(Σ) and RD unless explicitly declared as boundary operators (III.3).

A2.2 All selections MUST use deterministic tie-break rules derived solely from canonicalized objects and RD (I.4, III.8).

A2.3 RD MUST be immutable during a run (II.8).

A2.4 Numeric domain $\mathbb{D}$ and rounding/evaluation semantics MUST be fixed in $RD.env$ (II.11).

A2.5 Canonicalization MUST be idempotent and MUST collapse declared equivalence relations (I.4, II.7).

A2.6 Boundary operators MUST record all external IO as canonical evidence and replay MUST reuse that evidence (III.3, VI.5, IX.1).

A2.2 Canonicalization obligations

A2.a Canon_G MUST canonicalize G under $\sim_G$ (II.2, II.7).
A2.b Canon_z MUST normalize z according to the active profile's declared normalization (II.3, II.7).
A2.c Canon_Π MUST canonicalize projection identifiers and versions as an order-invariant set (II.4, II.7).
A2.d Canon_H MUST preserve digest-bearing trace and manifest identity fields (II.7).

.3 Appendix A3 — Operator registry schema (A3)

A3.1 Purpose

This appendix fixes the canonical schema for operator registry entries and the minimal fields required for traceability and replay.

A3.2 Operator registry entry (normative)

Each operator entry MUST include:

A3.1 $(\Omega_{\text{id}}, \Omega_{\text{ver}})$ — stable identifier and version.
A3.2 sig — typed signature (always $\Sigma \to \Sigma$; may include declared subdomain constraints).
A3.3 $\text{Adm}_\Omega(\Sigma, RD)$ — admissibility predicate (deterministic).
A3.4 DetClass — purity class: {Pure, Boundary}.
A3.5 EvSchema — evidence payload schema (possibly empty).
A3.6 Params — required parameters and their sources in $RD.params$.
A3.7 RoleTags — subset of kernel roles {ingest, canon, measure, solve, morph, gate, select, merge, emit}.
A3.8 Status — proof-status label if the operator claims formal properties (e.g. descent, contraction).

A3.3 Property claims and obligations

If an operator claims any of the following properties, it MUST register a proof obligation (Appendix A6 / A7):

- well-formedness preservation,
- descent in μ,
- contraction in d on a region $\mathcal{S}$,
- feasibility preservation under a constraint family,
- canonical-equivalence preservation.

.4 Appendix A4 — Profile registry schema (A4)

A4.1 Purpose

This appendix fixes the canonical schema for measurement profiles and their calibration artifacts.

A4.2 Profile registry entry (normative)

Each profile entry MUST include:

A4.1 $(m_{\text{id}}, m_{\text{ver}})$ — identifier and version.
A4.2 $\mathcal{C}_m$ — class labels (Structure/Symmetry/Topology/Resonance/Feasibility/Audit).
A4.3 Coordinate declarations $\{(z_i.\text{name}, z_i.\text{meaning}, z_i.\text{range})\}_{i=1}^{n}$.
A4.4 Normalization rule and parameters (RD-bound).
A4.5 Numeric domain and rounding discipline (RD-bound).

A4.6 Norm definition $\|\cdot\|_m$ if used for distances/MCI.
A4.7 Admissibility predicate $\mathrm{Adm}_m(\Sigma, RD)$ (may be trivial).
A4.8 Calibration fixture set $\mathcal{V}_m$ (or reference to it) and expected outputs.

A4.3 Cross-profile translation entries (optional)

If translation maps $T_{a\to b}$ are used, each entry MUST include:

A4.a (T_{id}, T_{ver}),
A4.b domain/codomain profile ids,
A4.c valid state class $\mathcal{S}_{ab}$,
A4.d error tolerance bound (RD-bound),
A4.e determinism declaration.

.5 Appendix A5 — Gate evidence and proof-pack schema (A5)

A5.1 Purpose

This appendix fixes the canonical evidence schema for gates and the minimal structure for gate proof objects and proof packs.

A5.2 Gate evidence item (normative)

Every evaluated gate predicate MUST emit a complete evidence item

$$E_q := (q_{id}, q_{ver},\ \theta,\ \mathsf{inputs},\ \mathsf{outputs},\ \mathsf{outcome},\ t,\ \mathrm{dig})$$

with the following mandatory fields:

A5.1 q_{id}, q_{ver} — predicate identifier and version.
A5.2 θ — all thresholds/parameters used, referenced to $RD.params$.
A5.3 inputs — canonical references to measurements and fixtures used (profile id/ver; z snapshot digest; window indices).
A5.4 outputs — computed witness values (margins, MCI, $\Delta\mu$, violation score, etc.).
A5.5 outcome — boolean result.
A5.6 t — trace index (or window range) to which the decision applies.
A5.7 dig — digest over canonicalized evidence item under $RD.hash$.

A5.3 Gate proof object (normative)

A gate proof object MUST include:

A5.a the evidence bundle containing all E_q items used in the decision,
A5.b the verifier identifier Verify_{gate}: recompute predicate under RD and compare to outcome,
A5.c the gate-stage ordering if staged gates are used (including short-circuit index).

A5.4 Consensus certificate (normative)

If dual-fabric acceptance is used, a consensus certificate MUST include:

A5.i fabric identifiers (program id + evaluation policy) for both fabrics,
A5.ii final-state digests (or windowed digests) for both runs,
A5.iii predicate outcomes for both fabrics,
A5.iv MCI inputs and computed MCI value,
A5.v robustness thresholds (ϵ_m, η) from RD,
A5.vi deterministic verifier that recomputes all values.

.6 Appendix A6 — Proof obligation registry (A6)

A6.1 Purpose

This appendix defines the canonical proof-obligation registry schema and the closure states.

A6.2 Obligation entry (normative)

Each obligation entry MUST be of the form:

$$\mathcal{O} := (id,\ node,\ claim,\ status,\ prereq,\ verifier,\ state)$$

with:

A6.1 *id* — unique obligation identifier.
A6.2 *node* — section label where claim appears.
A6.3 *claim* — normalized claim form (pre, stmt, post, tags).
A6.4 *status* — one of {Axiomatic, Defined, Derived, Model-Specific, External, Open}.
A6.5 *prereq* — dependency list (obligation ids or section refs).
A6.6 *verifier* — proof reference or proof-object verifier id.
A6.7 *state* — one of {Open,Satisfied,External,Quarantined}.

A6.3 Closure rule (normative)

A6.a A Derived obligation is Satisfied iff an admissible in-document proof exists, or an admissible proof object verifies successfully.
A6.b An External obligation is External iff it obeys the boundary-verification protocol (VI.5) where applicable.
A6.c An Open obligation MUST remain Open or be explicitly Quarantined and MUST NOT be used as a premise for Derived obligations.

.7 Appendix A7 — Residual risk register (A7)

A7.1 Purpose

This appendix defines the residual risk register used to quarantine open items and to bound their influence on conformance and acceptance.

A7.2 Residual risk item (normative)

Each residual risk item MUST be of the form:

$$R := (rid,\ node,\ description,\ impact,\ mitigation,\ state)$$

where $state \in \{\mathsf{Open}, \mathsf{Quarantined}, \mathsf{Retired}\}$.

A7.3 Quarantine rule (normative)

If $state \in \{\mathsf{Open}, \mathsf{Quarantined}\}$ then:

A7.1 the item MUST NOT be used as a premise in any Derived claim,
A7.2 the item MUST NOT be the sole justification for acceptance in any gate,
A7.3 the item MAY be referenced in informative sections and MAY motivate further work.

.8 Appendix A8 — Change log and version policy (A8)

A8.1 Purpose

This appendix fixes the document version policy and the minimum required change log fields.

A8.2 Version policy (normative)

A8.1 Any semantic change to a normative definition, operator contract, profile coordinate meaning, gate evidence schema, or manifest schema MUST increment the document version.
A8.2 Any change that affects replay comparability MUST be labeled **Breaking**.
A8.3 Any change that preserves evidence verification but may alter outputs MUST be labeled **Evidence-compatible**.
A8.4 Any change that preserves replay outputs on calibrated fixtures MUST be labeled **Replay-compatible**.

A8.3 Change log entry (normative)

Each change log entry MUST include:

A8.a date,
A8.b version,
A8.c change summary,
A8.d compatibility class,
A8.e affected nodes/sections,
A8.f updated obligation ids (if any).

.9 Appendix A9 — External dependencies ledger (A9)

A9.1 Purpose

This appendix enumerates all External dependencies referenced by any claim, proof object, verifier, or model boundary. It makes the External surface explicit and audit-compatible.

A9.2 Ledger entry schema (normative)

Each external dependency entry MUST be of the form:

$X := (xid, name, version, role, input\_contract, output\_contract, replay\_mode, boundary\_operator, status)$.

Mandatory fields:

A9.1 *xid* — unique dependency identifier.
A9.2 $name, version$ — dependency identity.
A9.3 *role* — {Theorem, Tool, Service, Oracle, Reference}.
A9.4 *input_contract* — canonical evidence required as inputs.
A9.5 *output_contract* — canonical evidence required as outputs.
A9.6 *replay_mode* — {ReuseRecordedOutputs, CapturedReExecution}.
A9.7 *boundary_operator* — the boundary operator id that mediates this dependency (or `None` for pure-theorem references).
A9.8 *status* — MUST be **External**.

A9.3 Minimal list (initially empty)

- **(reserved)** Standard fixed-point theorem dependencies (if used by any model) [External]

- **(reserved)** External verification tools (SMT/compilers) [External]

.10 Appendix A10 — Model compatibility matrix (A10)

A10.1 Purpose

This appendix records compatibility among model classes, profiles, gates, and verification packs. It prevents silent mixing of incompatible assumptions.

A10.2 Compatibility matrix schema (normative)

Each compatibility entry MUST be of the form:

$$C := (mid_a, mid_b,\ compat,\ shared\_RD,\ shared\_canon,\ shared\_profiles,\ boundary\_notes).$$

Fields:

A10.1 mid_a, mid_b — model identifiers (id+version).
A10.2 $compat$ — {Compatible, CompatibleWithAdapter, Incompatible}.
A10.3 $shared\_RD$ — whether RD schemas are compatible without loss.
A10.4 $shared\_canon$ — whether canonicalization/equivalence relations are compatible or require adapters.
A10.5 $shared\_profiles$ — whether profile semantics are directly comparable or require translation maps.
A10.6 $boundary\_notes$ — explicit constraints and quarantined items.

A10.3 Minimal matrix (initially empty)

Model A	Model B	Compat	Boundary notes
(to be filled)	*(to be filled)*	*(to be filled)*	*(to be filled)*

.11 Appendix A11 — Residual risk and open items register (A11)

A11.1 Purpose

This appendix is the single canonical quarantine register for Open/Quarantined items. It is the only permitted place to list unresolved bridges that must not influence conformance or derived claims.

A11.2 Register entry schema (normative)

Each item MUST be:

$$R := (rid,\ node,\ description,\ impact,\ mitigation,\ state),$$

with $state \in \{\mathsf{Open}, \mathsf{Quarantined}, \mathsf{Retired}\}$ (see Part VI.8).

A11.3 Initial items (empty by default)

- **RR-000** *(empty slot; insert only if a real Open item exists)* [Open]

.12 Appendix A12 — Conformance checklist (A12)

A12.1 Purpose

This appendix provides a normative conformance checklist. A system claiming PSP conformance MUST satisfy all items in A12.2.

A12.2 Checklist (normative)

A12.1 **State.** The system defines $\Sigma = (G, z, \Pi, \mu, H, RD)$ and a well-formedness predicate WF, and preserves WF for all admissible steps (SI1).

A12.2 **Operators.** Every executed operator is registered with $(\Omega_{id}, \Omega_{ver})$, has an admissibility predicate, and preserves WF (SI2, Appendix A3).

A12.3 **Determinism.** All selections/schedules/ties are deterministic functions of canonicalized objects and RD (SI3, Appendix A2).

A12.4 **Replay.** Runs are replayable under the same $(\text{Canon}(\Sigma_0), P, RD)$, reproducing trace and artifact digests (SI4, IX.1).

A12.5 **Canonicalization.** Canonicalization is idempotent and collapses declared equivalence relations; digest-bearing history fields are preserved (A2.2, II.7).

A12.6 **Status discipline.** All load-bearing claims are labeled by exactly one status in $\mathcal{S}$ and obey admissibility; Open items are quarantined (I.3, VI.8).

A12.7 **Model wall.** Model-specific assumptions do not leak into foundations without boundary propagation and Model-Specific labeling (SI6, I.6, VII.1).

A12.8 **No hidden channels.** No reliance on ambient time/randomness/undefined IO order; boundary IO is captured as evidence (SI7, III.3, VI.5).

A12.9 **Provenance.** Every artifact has an artifact manifest binding it to (RunID, seg, RD) and to verifying evidence/proof packs (SI8, IX.2).

A12.10 **Obligation ledger.** Any claim used as a premise in a Derived proof or as a gate justification is represented in the obligation registry with an explicit closure state (VI.6, A6).

A12.11 **Residual risk.** Open/Quarantined items are listed only in the residual risk register and do not drive conformance or acceptance (VI.8, A11).

.13 Appendix A13 — Canonical templates (A13)

A13.1 Purpose

This appendix defines canonical *schemas* used throughout PSP: node claims, gates, proof objects, manifests. Schemas are normative where indicated.

A13.2 Normalized claim template (normative)

A normalized claim MUST have the form:

$$\text{Claim} := (\text{pre},\ \text{stmt},\ \text{post},\ \text{tags})$$

where:

- pre lists explicit premises/assumptions,
- stmt is the statement,
- post lists explicit consequences,
- tags include $(status,\ node,\ model\_boundary,\ verifier\_id,\ obligation\_id)$.

A13.3 Gate predicate template (normative)

A gate predicate declaration MUST include:

- (q_{id}, q_{ver}) and a formal definition $q : \Sigma \to \{0, 1\}$,
- parameter set θ bound to $RD.params$,
- required inputs (measurements, windows, constraints),
- evidence schema reference (Appendix A5),
- proof-status label if the predicate claims properties beyond definitional use.

A13.4 Proof object template (normative)

A proof object MUST have:

$$\mathcal{P} := (\mathcal{P}_{id}, \mathcal{P}_{ver},\ \mathcal{B},\ \mathcal{V},\ RD)$$

with $\mathcal{B}$ (evidence bundle) and $\mathcal{V}$ (deterministic verifier id+params) per Part VI.

A13.5 Execution and artifact manifest template (normative)

Execution manifest:

$$\mathcal{M}_{exec} := (\text{RunID}, RD, P_{id}, \mathcal{T}_{dig}, \mathcal{A}_{dig}, \mathcal{VP}_{dig}, \mathsf{meta})$$

Artifact manifest:

$$\mathcal{M}_a := (a_{id}, a_{ver}, \text{Dig}(a), \text{RunID}, seg, gates, proofs, RD)$$

as defined in Part IX.

.14 Appendix A14 — Minimal bibliography (A14)

A14.1 Purpose

This appendix is a compact reference list for external theorems or standard results if and only if they are invoked as **External**. If no such results are invoked, this appendix may remain empty.

Bibliography

[1] *Reserved:* Banach fixed-point theorem reference (insert only if invoked by an External claim).

[2] *Reserved:* Stable manifold / dynamical systems reference (insert only if invoked by an External claim).

.15 Conclusion

This specification defines Post-Symbolic Programming (PSP) as a deterministic, state-first operator calculus with explicit conformance invariants, model-boundary discipline, auditable evidence, and replayable provenance. Its core result is a closed semantic frame in which (i) state evolution is defined by typed operators, (ii) acceptance decisions are justified by evidence bundles and verifiable proof objects, and (iii) model-specific instantiations (AinSoft-class runtimes, Scorpio boundary calculus, discrete symmetry scaffolds) are admissible only under explicit boundary statements and status discipline.

The remaining appendices provide the canonical registries required for implementation, audit, and long-term compatibility: operator registry schema, profile schema and calibration fixtures, gate evidence schema, proof-obligation registry and closure rules, external dependency ledger, compatibility matrix, and residual risk quarantine.